

从 C++ 到 AI 计算：第三册源码卷

LCQI 本地 CPU 推理引擎完整源码与阅读路径

CPU Performance Study

本卷把第三册贯穿项目 LCQI 的构建入口、模型资产、公共头文件、核心实现、工具脚本、benchmark、trace、报告生成器和 CTest 验收源码独立成册。原理卷负责解释机制，实践卷负责安排阶段任务；源码卷负责给出一份可完整审阅、可构建、可回归的代码证据。

前言

第三册原理卷讲 AI 推理系统的机器事实，实践卷把任务拆成阶段作业。本源码卷只处理一件事：把贯穿项目 [books/cpu-volume-3/labs/linux_cpu_inference](https://books.cpu-volume-3/labs/linux_cpu_inference) 以及相关工具脚本单独成册，让读者能从构建入口一路读到测试和报告，而不是在原理正文或实践任务后面翻一段过长的源码附录。

核心思想

源码卷不是“代码仓库的打印版”。每章先说明该组文件承担的工程责任，再给完整源码。阅读时要把函数签名、错误边界、测试断言和报告字段连起来看：一个推理引擎是否可信，不取决于某个单独函数看起来像模型，而取决于资产能校验、shape 能拒绝错误、kernel 能对照、trace 能定位、benchmark 能说明口径、CTest 能长期守住回归。

本卷收录的主线代码是 LCQI。它从 tiny MLP 教学闭环开始，逐步覆盖 tokenizer、safetensors manifest gate、int8 kernel、packed/AVX2 路径、reference decoder、KV cache trace、GPT-2 reference path、7B compute ledger、serving SLO probe、真实 small 开源模型 smoke、自研 GPT-2 smoke 和 benchmark compare。它不是生产级推理引擎，但每个能力都要求有源码、命令、测试或报告证据。

阅读顺序建议如下。第一章先看构建入口和模型资产，明确哪些目标一定构建、哪些对标依赖外部库。第二章读公共头文件，先把数据结构和函数合同固定下来。第三章读 tiny MLP、int8 kernel 和基础 CLI，建立最小推理闭环。第四章读 reference decoder、safetensors、tokenizer 和 GPT-2 代码，理解真实模型路径为什么比模板 demo 多出许多边界检查。第五章读工具脚本、benchmark、trace、ledger、serving probe 和测试，把代码和可复查证据连起来。

源码卷的 LaTeX 文件通过 [lstinputlisting](#) 直接读取第三册工程源码。也就是说，源码不在书中复制第二份；代码更新后重新构建本卷，PDF 和 EPUB 会读取当前仓库里的真实文件。

缩写与读码约定

LCQI Local CPU Quantized Inference, 本书第三册的本地 CPU 量化推理贯穿项目。

CLI Command Line Interface, 命令行入口。源码里通常只负责参数解析和结果打印, 不承载核心推理逻辑。

MLP Multi-Layer Perceptron, 多层感知机。tiny MLP 用来建立可手算、可测试的最小推理闭环。

KV Cache Key/Value Cache, decoder-only 模型跨 token 复用 attention key/value 的状态缓存。

BPE Byte Pair Encoding, GPT-2 使用的 byte-level 子词编码方法。

GGUF GPT-Generated Unified Format, llama.cpp 常用的模型文件格式。

TTFT Time To First Token, 服务请求从进入系统到输出第一个 token 的时间。

TPOT Time Per Output Token, 生成阶段每个输出 token 的平均或分位耗时。

SLO Service Level Objective, 服务等级目标。本书用它描述 TTFT、TPOT、拒绝率和取消回收等运行时约束。

PMU Performance Monitoring Unit, 硬件性能计数器单元。没有 PMU 权限时, 性能报告不能伪装成硬件级结论。

AVX2 Advanced Vector Extensions 2, x86-64 SIMD 指令集扩展。源码存在不等于当前机器可用, 报告要看运行时可用性。

读码时先看头文件中的数据结构和入口, 再看实现文件中的检查、循环和错误分支, 最后回到测试文件确认哪些行为已经被锁成回归合同。遇到 benchmark 或 smoke 脚本时, 要区分“同模型同精度对比”和“成熟引擎参考口径”, 不能把不同模型、不同量化格式的速度直接写成质量排名。

目录

前言	ii
缩写与读码约定	iii
第 1 章 构建入口、模型资产与项目边界	1
1.1 源码卷覆盖范围	1
1.2 根构建入口	1
1.3 LCQI 目标定义	2
1.4 项目说明与模型资产	4
第 2 章 公共合同：模型、Tokenizer、Kernel 与 Decoder	12
2.1 为什么先读头文件	12
2.2 Tiny MLP 与推理入口	12
2.3 Int8 Kernel、Tokenizer 与 Safetensors	13
2.4 Reference Decoder 与 GPT-2 合同	17
第 3 章 Tiny MLP、Int8 Kernel 与基础 CLI	23
3.1 最短闭环：从模型文件到 logits	23
3.2 模型加载与基础推理	23
3.3 Int8 Kernel 基础实现	30
第 4 章 Reference Decoder、GPT-2 与模型导入	40
4.1 从 tiny MLP 走向 decoder-only 模型	40
4.2 Tiny Reference Decoder	40
4.3 Tokenizer 与 Safetensors 实现	57
4.4 GPT-2 Reference Path	73
第 5 章 工具、报告、Benchmark 与回归测试	118
5.1 工具不是附属品	118
5.2 Benchmark、Trace、Ledger 与 Serving Probe	118
5.3 外部模型 Smoke 与对比脚本	133
5.4 完整测试	157
读码检查清单	174

第 1 章

构建入口、模型资产与项目边界

1.1 源码卷覆盖范围

LCQI 的源码边界由三个层次组成。第一层是仓库构建入口，它决定 CMake 怎样把第三册的实验代码纳入构建和测试。第二层是 `labs/linux_cpu_inference`，它包含真正的库、CLI、benchmark、trace、ledger、serving probe 和测试目标。第三层是工具脚本，它们下载或复用外部模型资产，生成 smoke 和 benchmark 报告。

源码阅读任务

先读本章的 CMake 文件，再读 README。读者要能回答三个问题：一是 `lcqi` 静态库包含哪些实现文件；二是哪些可执行文件只依赖自研代码，哪些依赖可选外部库；三是真实 GPT-2 和 SmolLM2 smoke 为什么不进入默认 CTest。

1.2 根构建入口

第三册根目录的 CMake 文件只做项目级选择：是否构建第三册 labs，以及如何把 `labs/linux_cpu_inference` 接入。它应该保持薄层，避免把具体目标定义分散到多个地方。

Listing 1.1 `cpu-volume-3/CMakeLists.txt`

```
1 cmake_minimum_required(VERSION 3.31)
2 project(CPUVolume3 LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 20)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6 set(CMAKE_CXX_EXTENSIONS OFF)
7 set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
8
9 option(CPU_VOLUME_3_BUILD_LABS "Build CPU Volume 3 labs" ON)
10
11 if(CPU_VOLUME_3_BUILD_LABS)
12     enable_testing()
13     add_subdirectory(labs/linux_cpu_inference)
14 endif()
```

实践卷也提供一个薄 CMake 入口，它把同一份 LCQI 源码作为实践卷可测试对象接入。这样实践卷不会复制实现，只复用第三册主工程。

Listing 1.2 `cpu-volume-3-practice/CMakeLists.txt`

```
1 cmake_minimum_required(VERSION 3.31)
2 project(CPUVolume3Practice LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 20)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6 set(CMAKE_CXX_EXTENSIONS OFF)
7 set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
8
```

```

9 enable_testing()
10 add_subdirectory(../cpu-volume-3/labs/linux_cpu_inference linux_cpu_inference)

```

1.3 LCQI 目标定义

`linux_cpu_inference/CMakeLists.txt` 是源码卷的第一份核心文件。这里能看到 `lcqi` 库、`lcqi_cli`、`lcqi_bench`、`lcqi_trace`、`lcqi_gpt2`、`lcqi_ledger`、`lcqi_serving_probe`、可选 `lcqi_onednn_bench` 和 `lcqi_tests` 的完整关系。

Listing 1.3 `linux_cpu_inference/CMakeLists.txt`

```

1 add_library(lcqi STATIC
2     src/gpt2_reference.cpp
3     src/inference.cpp
4     src/int8_kernels.cpp
5     src/model.cpp
6     src/reference_decoder.cpp
7     src/safetensors.cpp
8     src/tokenizer.cpp
9 )
10
11 if(CMAKE_SYSTEM_PROCESSOR MATCHES "x86_64|AMD64|i[3-6]86")
12     if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
13         target_sources(lcqi PRIVATE src/int8_kernels_avx2.cpp)
14         set_source_files_properties(src/int8_kernels_avx2.cpp PROPERTIES ↵
15             ↵ COMPILE_OPTIONS "-mavx2;-mfma")
16         target_compile_definitions(lcqi PRIVATE LCQI_ENABLE_AVX2=1)
17     endif()
18 endif()
19
20 if(NOT CMAKE_SYSTEM_NAME STREQUAL "Linux")
21     message(WARNING "LCQI is written for the book's local Linux CPU target; ↵
22         ↵ current system is ${CMAKE_SYSTEM_NAME}.")
23 endif()
24
25 target_include_directories(lcqi
26     PUBLIC
27     ${CMAKE_CURRENT_SOURCE_DIR}/include
28 )
29
30 target_compile_features(lcqi PUBLIC cxx_std_20)
31
32 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
33     target_compile_options(lcqi PRIVATE -Wall -Wextra -Wpedantic)
34 endif()
35
36 add_executable(lcqi_cli
37     src/main.cpp
38 )
39
40 target_link_libraries(lcqi_cli PRIVATE lcqi)

```

```
40 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
41     target_compile_options(lcqi_cli PRIVATE -Wall -Wextra -Wpedantic)
42 endif()
43
44 add_executable(lcqi_bench
45     src/bench.cpp
46 )
47
48 target_link_libraries(lcqi_bench PRIVATE lcqi)
49
50 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
51     target_compile_options(lcqi_bench PRIVATE -Wall -Wextra -Wpedantic)
52 endif()
53
54 add_executable(lcqi_trace
55     src/trace.cpp
56 )
57
58 target_link_libraries(lcqi_trace PRIVATE lcqi)
59
60 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
61     target_compile_options(lcqi_trace PRIVATE -Wall -Wextra -Wpedantic)
62 endif()
63
64 add_executable(lcqi_gpt2
65     src/gpt2_cli.cpp
66 )
67
68 target_link_libraries(lcqi_gpt2 PRIVATE lcqi)
69
70 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
71     target_compile_options(lcqi_gpt2 PRIVATE -Wall -Wextra -Wpedantic)
72 endif()
73
74 add_executable(lcqi_ledger
75     src/ledger.cpp
76 )
77
78 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
79     target_compile_options(lcqi_ledger PRIVATE -Wall -Wextra -Wpedantic)
80 endif()
81
82 add_executable(lcqi_serving_probe
83     src/serving_probe.cpp
84 )
85
86 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
87     target_compile_options(lcqi_serving_probe PRIVATE -Wall -Wextra -Wpedantic)
88 endif()
89
90 find_package(dnnl QUIET)
91 if(dnnl_FOUND)
```

```

92     add_executable(lcqi_onednn_bench
93         src/onednn_bench.cpp
94     )
95     target_link_libraries(lcqi_onednn_bench PRIVATE DNNL::dnnl)
96     target_compile_features(lcqi_onednn_bench PRIVATE cxx_std_20)
97     if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
98         target_compile_options(lcqi_onednn_bench PRIVATE -Wall -Wextra -↵
↵ Wpedantic)
99     endif()
100 endif()
101
102 add_executable(lcqi_tests
103     tests/lcqi_tests.cpp
104 )
105
106 target_link_libraries(lcqi_tests PRIVATE lcqi)
107
108 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
109     target_compile_options(lcqi_tests PRIVATE -Wall -Wextra -Wpedantic)
110 endif()
111
112 add_test(NAME lcqi_tests COMMAND ${<TARGET_FILE:lcqi_tests>})
113 add_test(NAME lcqi_gpt2_tiny COMMAND ${<TARGET_FILE:lcqi_gpt2>})
114 set_tests_properties(lcqi_gpt2_tiny PROPERTIES
115     PASS_REGULAR_EXPRESSION "generated_ids 1 2 3 3 3"
116 )
117 add_test(NAME lcqi_trace COMMAND ${<TARGET_FILE:lcqi_trace>})
118 set_tests_properties(lcqi_trace PROPERTIES
119     PASS_REGULAR_EXPRESSION "tokenizer,0,-1,5,1,i32,contiguous,10,4,0;1;2;3;4"
120 )
121 add_test(NAME lcqi_ledger COMMAND ${<TARGET_FILE:lcqi_ledger>})
122 set_tests_properties(lcqi_ledger PROPERTIES
123     PASS_REGULAR_EXPRESSION "bandwidth_limit,int4_at_100_gbps,23.602,tokens/s"
124 )
125 add_test(NAME lcqi_serving_probe COMMAND ${<TARGET_FILE:lcqi_serving_probe>})
126 set_tests_properties(lcqi_serving_probe PROPERTIES
127     PASS_REGULAR_EXPRESSION "request,req_too_long,0,memory_budget_exceeded"
128 )

```

1.4 项目说明与模型资产

README 是复现实验的入口。它写清楚当前能力、后续边界、构建命令、tiny 推理命令、benchmark 字段、reference trace 字段、GPT-2 smoke、7B ledger、serving SLO probe 和 SmolLM2 small smoke。读源码前先读 README，可以避免把 tiny fixture、GPT-2 reference path 和外部 llama.cpp smoke 混成同一个能力等级。

Listing 1.4 linux_cpu_inference/README.md

```

1 # Linux CPU Quantized Inference
2
3 这是第三册贯穿项目的第一阶段切片：Linux 本地 CPU 量化线性层和最小推理流程。第三↵
↵ 册的贯穿目标不是这个 MLP 本身，而是一台能推理开源 7B 左右 decoder-only 大↵

```


- ↪ 模型的本地引擎，并逐步覆盖 tokenizer、模型加载、Transformer 层、KV cache
- ↪ 、CPU/GPU 后端、正确性报告和性能对标。
- 4
- 5 目标平台是 x86-64 Linux 实验环境。完整性能报告应记录 CPU 型号、内核版本、编译器
 - ↪ 版本、是否能使用 PMU、NUMA 和线程亲和能力。若运行环境缺少 perf、NUMA、线
 - ↪ 程亲和性或硬件性能计数器权限，报告必须把相关结论降级为“未验证”，不能把源
 - ↪ 码路径存在当作实测性能证据。后续 GPU 后端会作为独立 backend 接入，不改变
 - ↪ 当前切片的语义合同。
- 6
- 7 当前版本支持：
- 8
- 9 - 教学文本模型格式 `LCQI\_MODEL\_V1`
- 10 - 最小 tokenizer 合同格式 `LCQI\_TOKENIZER\_V1`
- 11 - safetensors header manifest 解析：metadata、tensor name、dtype、shape、data
 - ↪ offsets、offset 边界和 dtype/shape 字节数校验
- 12 - 两层 MLP 教学切片
- 13 - int8 权重加 per-layer scale
- 14 - float 输入和激活
- 15 - 量化线性层、ReLU、分类 argmax
- 16 - int8 linear scalar baseline、packed layout、AVX2 路径和 shape sweep benchmark
- 17 - tiny reference decoder: RMSNorm、float linear、RoPE、GQA KV cache、decode
 - ↪ attention、SwiGLU、lm head
- 18 - GPT-2 reference path: byte-level BPE tokenizer、`config.json`、F32/F16/BF16
 - ↪ safetensors 张量读取、HuggingFace GPT-2 name mapping、LayerNorm、绝对位置
 - ↪ 嵌入、packed QKV、causal attention、GELU、tied lm head、full-prefix
 - ↪ greedy generation、KV cache greedy generation 和分阶段 benchmark
- 19 - reference trace CSV: 从 prompt/tokenizer 合同到 logits/sampler/token，逐
 - ↪ checkpoint 输出 shape、stride、dtype、layout、checksum、max_abs、values
 - ↪ 摘要
- 20 - 7B ledger CSV: 输出典型 7B shape 的 FLOPs、KV 容量、权重/KV 字节和 bandwidth
 - ↪ only tokens/s 上限
- 21 - serving SLO probe CSV: 输出 admission、batch state、KV 预算、queue wait、TTFT
 - ↪ 、TPOT、取消回收和拒绝率
- 22 - 自研 GPT-2 冒烟: 加载 `openai-community/gpt2` 的 `config.json`、`vocab.json`
 - ↪ 、`merges.txt`、`model.safetensors`，在 LCQI C++ reference path 上生成 1
 - ↪ 个 token，并把文件 hash、prompt ids、generated ids 和文本输出写入报告
- 23 - 自研 GPT-2 benchmark 对比: 同一个 GPT-2 F32 safetensors 模型分别跑 full
 - ↪ prefix 和 cached-KV，并在可用时附带 llama.cpp/SmolLM2 Q4_K_M 作为成熟引擎
 - ↪ 参考
- 24 - 真实 small 开源模型冒烟: 通过 llama.cpp 加载 SmolLM2-135M-Instruct 的 GGUF 量
 - ↪ 化文件，在本地 CPU 上生成一段 assistant 文本，并把模型 hash、命令、输出和
 - ↪ 性能行写入报告
- 25 - CLI 推理和简单 benchmark
- 26 - CTest 正确性测试
- 27
- 28 后续扩展方向：
- 29
- 30 - 真实 7B 级模型 metadata、tokenizer 和量化权重导入
- 31 - SentencePiece tokenizer、GGUF 权重导入、safetensors 分片加载和更多模型方言
 - ↪ name mapping
- 32 - GPT-2 reference path 的 Transformers/PyTorch golden logits 对齐报告
- 33 - KV cache 分页、内存规划和可选量化

```

34 - CPU packed weight、SIMD、线程和 NUMA 优化
35 - GPU backend adapter 和 device kernel
36 - 与现有框架的 prefill/decode/内存/误差对标报告
37
38 构建：
39
40 ```bash
41 cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↵
    ↵ DCMMAKE_BUILD_TYPE=Debug
42 cmake --build books/cpu-volume-3/build/lcqi-debug
43 ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
44 ```
45
46 运行：
47
48 ```bash
49 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_cli \
50   books/cpu-volume-3/labs/linux_cpu_inference/models/tiny_mlp_i8.txt \
51   1.0 2.0 -1.0 0.5 1000
52 ```
53
54 kernel benchmark：
55
56 ```bash
57 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_bench 200
58 ```
59
60 输出 CSV 字段：
61
62 ```text
63 target_arch,input_size,output_size,output_block,repeat,backend,available,↵
    ↵ average_us,max_abs_diff,checksum
64 ```
65
66 当前 benchmark 按 backend 逐行输出：`scalar`、`packed_scalar`、`avx2`。x86-64 ↵
    ↵ 构建会编译 AVX2 路径；不支持 AVX2/FMA 的机器会把该 backend 标为 `↵
    ↵ available=0`，避免把源码存在误报成当前环境实测性能。
67
68 这还不是生产级高性能 kernel。当前 benchmark 建立 scalar baseline、packed layout↵
    ↵ 、AVX2 基线正确性和 shape sweep 口径。要达到完整 AI Infra 证据，还必须继↵
    ↵ 续补反汇编分析、AVX-512、perf counter、oneDNN/OpenBLAS/llama.cpp/ggml 对↵
    ↵ 照、sanitizer、fuzz 和 CI。
69
70 safetensors manifest gate：
71
72 `lcqi_tests` 会运行时生成一个最小 safetensors fixture，验证 header 长度、↵
    ↵ `__metadata__`、tensor name、dtype、shape、data offsets、payload 边界和 ↵
    ↵ dtype/shape 推导出的字节数。它还会生成 offset 超出 payload、dtype/shape ↵
    ↵ 字节数不匹配的坏文件，确认加载器会拒绝。这个 gate 只覆盖模型资产目录表，↵
    ↵ 不等于已经支持完整 LLaMA/Qwen 权重映射、分片合并或 mmap 执行；后续真实模↵
    ↵ 型加载必须在这个 manifest 上继续接 name mapping、shape verifier、quant ↵
    ↵ descriptor 和 first token trace。

```

```

73
74 reference decoder trace:
75
76 ```bash
77 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_trace
78 ```
79
80 输出 CSV 字段:
81
82 ```text
83 name,token_position,layer_id,shape,stride,dtype,layout,checksum,max_abs,values
84 ```
85
86 这条 trace 固定 prompt fixture `tok1 tok2 tok3`, tokenizer 输出完整 `tokens`
    ↪ `=[0,1,2,3,4]`, 其中 `0/4` 是 BOS/EOS; decoder trace 会剥离 BOS/EOS 后进入
    ↪ tiny decoder。这样报告能同时固定 tokenizer 合同和 decoder 输入合同, 避免
    ↪ 把二者混成一个不可解释的 token 数组。随后 trace 把 embedding、RMSNorm、Q/
    ↪ K/V、RoPE、KV cache slot、attention、SwiGLU、layer output、final norm、
    ↪ logits、argmax sampler 和 generated token 串成可回归检查的硬链路。`
    ↪ lcqi_tests` 会检查 tokenizer contract hash、关键 checkpoint 的 shape、
    ↪ stride、dtype、layout 和 logits golden, CTest 也会直接运行 `lcqi_trace`。
87
88 当前 tokenizer fixture 的关键输出:
89
90 ```text
91 tokenizer,0,-1,5,1,i32,contiguous,10,4,0;1;2;3;4
92 tokenizer_contract,0,-1,scalar,scalar,u64,fnv1a,13452902845388333734,0,tiny-
    ↪ chat-template-v1
93 ```
94
95 GPT-2 自研 reference smoke:
96
97 ```bash
98 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_gpt2
99 make cpu3-gpt2-smoke
100 make cpu3-gpt2-benchmark-compare
101 ```
102
103 第一条命令不需要下载模型, 直接运行仓库内 tiny GPT-2 fixture, 输出固定 prompt
    ↪ ids、logits、predicted token 和 greedy generated ids。`lcqi_tests` 会覆盖
    ↪ tiny GPT-2 forward/generation、byte-level BPE 编码解码、F32/F16/BF16
    ↪ safetensors 读取、HuggingFace 风格 tiny GPT-2 checkpoint 加载和坏 shape
    ↪ 拒绝。
104
105 第二条命令显式依赖网络, 只下载 `openai-community/gpt2` 的 `config.json`、`vocab-
    ↪ .json`、`merges.txt`、`model.safetensors` 到 `~/cache/lcqi-gpt2-smoke/`
    ↪ `openai-community--gpt2`, 模型文件不进入 Git。脚本会校验四个文件的 bytes
    ↪ 和 SHA256, 构建 `lcqi_gpt2`, 运行:
106
107 ```text
108 Hello, my name is
109 ```

```

```

110
111 当前验证报告写到：
112
113 ```text
114 books/cpu-volume-3/results/lcqi-gpt2-smoke.txt
115 ```
116
117 最近一次本机报告的关键行是：
118
119 ```text
120 prompt_ids 15496 11 616 1438 318
121 generated_ids 15496 11 616 1438 318 1757
122 generated_text Hello, my name is John
123 validation=PASS
124 ```
125
126 这说明 LCQI 自研 C++ reference path 已经能跑标准 GPT-2 safetensors 的 tokenizer
    ↳ 、权重导入、forward 和 greedy generation。默认真实模型路径现在使用 `
    ↳ cached_kv`，也可以用 `--engine full` 复现旧的 full-prefix 重算路径：
127
128 ```bash
129 books/cpu-volume-3/build/lcqi-release/labs/linux_cpu_inference/lcqi_gpt2 \
130 --benchmark --engine cached \
131 ~/.cache/lcqi-gpt2-smoke/openai-community--gpt2 \
132 "Hello, my name is" 4
133 ```
134
135 `make cpu3-gpt2-benchmark-compare` 会生成：
136
137 ```text
138 books/cpu-volume-3/results/lcqi-gpt2-benchmark-compare.txt
139 ```
140
141 最近一次本机 aarch64/Qualcomm 报告中，LCQI full-prefix 在 GPT-2 F32 上生成 4 个
    ↳ token 的生成阶段为 `2522.81 ms`，约 `1.586 token/s`；LCQI cached-KV 同样
    ↳ 输出 `Hello, my name is John. I'm`，生成阶段为 `826.152 ms`，约 `4.842
    ↳ token/s`。cached-KV 的第一个新 token 来自 prefill 最后一步，后续 decode
    ↳ 实际执行 3 个 forward step，耗时 `309.524 ms`，约 `9.692 step/s`。同机
    ↳ llama.cpp 使用 SmolLM2-135M-Instruct Q4_K_M 的 `llama-bench -p 5 -n 4` 作
    ↳ 为成熟引擎参照，prefill `76.41 token/s`、decode `27.83 token/s`。这个对照
    ↳ 不是同模型质量排名：LCQI 跑的是 GPT-2 F32 reference path，llama.cpp 跑的
    ↳ 是 GGUF 量化 small instruct 模型。它揭示的是工程差距：LCQI 已经消除了“每
    ↳ 步重算前缀”的主要算法错误，但还缺 packed weight、量化 matmul、SIMD/SVE/
    ↳ NEON kernel、线程并行、mmap/lazy load、采样器和成熟的内存调度。
142
143 它仍然不是生产级推理引擎：当前 GPT-2 路径还没有把 logits 和 Transformers/
    ↳ PyTorch 逐数值对齐成外部 golden 报告，也没有 GGUF 自研导入、分页 KV cache
    ↳ 、量化权重执行和高性能多线程 kernel。
144
145 7B ledger：
146
147 ```bash

```

```

148 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_ledger \
149 > books/cpu-volume-3/results/lcqi-sevenb-ledger-2026-06-24.csv
150 ```
151
152 输出 CSV 字段：
153
154 ```text
155 section,item,value,unit,notes
156 ```
157
158 这份账本固定 `L=32,H=4096,n_q=32,n_kv=8,head_dim=128,context=4096`，报告 decode
    ↳ FLOPs、KV 容量、fp16/int8/int4 每 token 字节和不同有效带宽下的 tokens/s
    ↳ 上限。CTest 会检查 `int4_at_100_gbps=23.602 tokens/s` 这一行，防止账本公
    ↳ 式漂移。
159
160 serving SLO probe：
161
162 ```bash
163 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_serving_probe
    ↳ \
164 > books/cpu-volume-3/results/lcqi-serving-slo-probe-2026-06-24.csv
165 ```
166
167 输出 CSV 字段：
168
169 ```text
170 row_type,request_id,accepted,rejection_reason,prompt_tokens,reserved_tokens,
    ↳ kv_reserved_mib,queue_wait_ms,prefill_ms,ttft_ms,decode_step_p95_ms,
    ↳ tpot_ms,completion_tokens,finish_reason,finish_reason,metric_name,
    ↳ metric_value
171 ```
172
173 probe 固定四个请求：短请求、RAG 请求、取消请求和超长请求。它演示 admission 先按
    ↳ `prompt_tokens + max_new_tokens` 预留 KV，取消请求释放预算，超长请求返回
    ↳ `memory_budget_exceeded`，并输出 `ttft_p95_ms`、`queue_wait_p95_ms`、`
    ↳ rejection_rate` 等服务化指标。
174
175 真实 small 开源模型 smoke：
176
177 ```bash
178 make cpu3-smolllm2-smoke
179 ```
180
181 这条命令显式依赖网络和外部构建，因此不放进默认 CTest。脚本会把 llama.cpp 和模型
    ↳ 文件缓存到 `~/.cache/lcqi-smolllm2-small-smoke`，模型不进入 Git。固定模型
    ↳ 是 `HuggingFaceTB/SmolLM2-135M-Instruct` 的 GGUF 量化版本，来源仓库为 `
    ↳ bartowski/SmolLM2-135M-Instruct-GGUF`，文件为 `SmolLM2-135M-Instruct-
    ↳ Q4_K_M.gguf`。脚本会校验文件大小 `105454432` 字节和 SHA256：
182
183 ```text
184 2e8040ceae7815abe0dcb3540b9995eaa1fa0d2ca9e797d0a635ae4433c68c2d
185 ```

```

```

186
187 脚本构建固定 commit 的 `llama-simple-chat`，用 `-ngl 0` 强制 CPU 路径，输入短 ↵
    ↵ prompt，检查 assistant response 非空，并生成：
188
189 ```text
190 books/cpu-volume-3/results/lcqi-smolllm2-small-model-smoke.txt
191 ```
192
193 这份报告只能证明真实 GGUF small 模型已经完成加载、tokenizer/chat template、↵
    ↵ prefill、decode 和文本输出。它不能证明 LCQI 自研 C++ 引擎已经支持完整 ↵
    ↵ GGUF、不能证明回答质量正确，也不能替代 tiny fixture 的逐层 golden trace。↵
    ↵ tiny fixture 继续负责可手算语义和回归定位；SmolLM2 smoke 负责把“真实开源↵
    ↵ 模型能在本机跑起来”变成可复查证据。

```

tiny MLP 模型文件是最短闭环的资产。它决定输入维度、hidden 维度、输出维度、权重量化值、scale 和 bias；测试中的 logits golden 依赖这份文件。

Listing 1.5 models/tiny_mlp_i8.txt

```

1 LCQI_MODEL_V1
2 input_size 4
3 hidden_size 3
4 output_size 2
5 hidden_scale 0.1
6 hidden_weights
7 5 -3 2 1
8 -4 6 1 -2
9 3 2 -5 4
10 hidden_bias
11 0.1 -0.2 0.0
12 output_scale 0.05
13 output_weights
14 4 -6 2
15 -3 5 8
16 output_bias
17 -0.5 0.4

```

tiny tokenizer fixture 固定 BOS、EOS、UNK、词表和模板 hash。tokenizer 合同如果漂移，后面的 decoder trace 和测试都会被污染。

Listing 1.6 models/tiny_tokenizer.txt

```

1 LCQI_TOKENIZER_V1
2 bos_id 0
3 eos_id 4
4 unk_id -1
5 chat_template_hash tiny-chat-template-v1
6 vocab_size 5
7 vocab
8 0 <bos>
9 1 tok1
10 2 tok2
11 3 tok3

```

12 4 <eos>

第 2 章

公共合同：模型、Tokenizer、Kernel 与 Decoder

2.1 为什么先读头文件

头文件是系统对外承诺的最小合同。实现可以重写，kernel 可以替换，benchmark 可以补充平台路径，但头文件里暴露的数据结构、函数入口和返回字段决定了测试、CLI、工具脚本和章节讲解怎样调用这套工程。

核心思想

读 LCQI 时不要从最长的 `gpt2_reference.cpp` 开始。先读公共头文件，明确“模型如何表示”“推理返回什么”“trace 记录什么”“KV cache 怎样寻址”“tokenizer 的坏输入怎样处理”，再进入实现。这样读到长循环和 shape 检查时，知道它们在维护哪份合同。

2.2 Tiny MLP 与推理入口

`model.hpp` 定义 tiny MLP 的模型对象、权重、bias、scale 和加载入口。它的责任是把文本模型资产解释成 C++ 对象，不负责执行推理。

Listing 2.1 `include/lcqi/model.hpp`

```
1 #pragma once
2
3 #include <stdint>
4 #include <filesystem>
5 #include <string>
6 #include <vector>
7
8 namespace lcqi {
9
10 struct QuantizedLinearLayer {
11     std::int32_t input_size = 0;
12     std::int32_t output_size = 0;
13     float scale = 1.0F;
14     std::vector<std::int8_t> weights;
15     std::vector<float> bias;
16 };
17
18 struct TinyMlpModel {
19     std::int32_t input_size = 0;
20     std::int32_t hidden_size = 0;
21     std::int32_t output_size = 0;
22     QuantizedLinearLayer hidden;
23     QuantizedLinearLayer output;
24 };
25
26 TinyMlpModel load_model(const std::filesystem::path& path);
```



```

27
28 } // namespace lcqi

```

`inference.hpp` 定义推理结果和 `run_inference` 入口。返回值包含 hidden、logits 和 predicted class，测试可以用这些字段定位是加载错、kernel 错，还是 argmax 错。

Listing 2.2 include/lcqi/inference.hpp

```

1 #pragma once
2
3 #include <cstdint>
4 #include <span>
5 #include <vector>
6
7 #include <lcqi/model.hpp>
8
9 namespace lcqi {
10
11 struct InferenceResult {
12     std::vector<float> logits;
13     std::int32_t predicted_class = 0;
14 };
15
16 void linear_i8(const QuantizedLinearLayer& layer,
17               std::span<const float> input,
18               std::span<float> output);
19
20 void relu_inplace(std::span<float> values);
21
22 InferenceResult run_inference(const TinyMlpModel& model,
23                               std::span<const float> input);
24
25 } // namespace lcqi

```

2.3 Int8 Kernel、Tokenizer 与 Safetensors

int8 kernel 头文件定义 packed linear、deterministic fixture、scalar/workspace/AVX2 路径和误差比较。注意 AVX2 是运行时能力，不是源码存在就等于当前机器已验证。

Listing 2.3 include/lcqi/int8_kernels.hpp

```

1 #pragma once
2
3 #include <cstdint>
4 #include <span>
5 #include <vector>
6
7 #include <lcqi/model.hpp>
8
9 namespace lcqi {
10
11 struct PackedLinearI8 {
12     std::int32_t input_size = 0;

```

```

13     std::int32_t output_size = 0;
14     std::int32_t output_block_size = 0;
15     float scale = 1.0F;
16     std::vector<std::int8_t> packed_weights;
17     std::vector<float> bias;
18 };
19
20 struct KernelBenchmarkCase {
21     std::int32_t input_size = 0;
22     std::int32_t output_size = 0;
23     std::int32_t output_block_size = 0;
24     std::int32_t repeat = 0;
25 };
26
27 struct KernelBackendBenchmark {
28     const char* name = "";
29     bool available = false;
30     double average_us = 0.0;
31     float max_abs_diff = 0.0F;
32     float checksum = 0.0F;
33 };
34
35 struct KernelBenchmarkResult {
36     KernelBenchmarkCase benchmark_case;
37     std::vector<KernelBackendBenchmark> backends;
38 };
39
40 PackedLinearI8 pack_linear_i8(const QuantizedLinearLayer& layer,
41                               std::int32_t output_block_size);
42
43 void linear_i8_packed(const PackedLinearI8& layer,
44                      std::span<const float> input,
45                      std::span<float> output);
46
47 void linear_i8_packed_with_workspace(const PackedLinearI8& layer,
48                                     std::span<const float> input,
49                                     std::span<float> output,
50                                     std::span<float> accumulator_workspace);
51
52 bool linear_i8_packed_avx2_available() noexcept;
53
54 void linear_i8_packed_avx2(const PackedLinearI8& layer,
55                            std::span<const float> input,
56                            std::span<float> output);
57
58 QuantizedLinearLayer make_deterministic_i8_layer(std::int32_t input_size,
59                                                  std::int32_t output_size,
60                                                  float scale);
61
62 std::vector<float> make_deterministic_input(std::int32_t input_size);
63
64 float max_abs_diff(std::span<const float> lhs, std::span<const float> rhs);

```

```

65
66 KernelBenchmarkResult benchmark_linear_i8_case(const KernelBenchmarkCase& ↵
    ↵ benchmark_case);
67
68 } // namespace lcqi

```

tokenizer 头文件固定 BOS/EOS/UNK、词表和 template hash。prompt 进入模型前，必须先变成稳定 token id；未知 token 行为也必须被写进合同。

Listing 2.4 include/lcqi/tokenizer.hpp

```

1 #pragma once
2
3 #include <cstdint>
4 #include <filesystem>
5 #include <string>
6 #include <string_view>
7 #include <vector>
8
9 namespace lcqi {
10
11 struct TokenEntry {
12     std::string text;
13     std::int32_t id = 0;
14 };
15
16 struct TokenizerModel {
17     std::int32_t bos_id = -1;
18     std::int32_t eos_id = -1;
19     std::int32_t unk_id = -1;
20     std::string chat_template_hash;
21     std::vector<TokenEntry> vocab;
22 };
23
24 [[nodiscard]] TokenizerModel load_tokenizer(const std::filesystem::path& path);
25
26 [[nodiscard]] std::vector<std::int32_t> encode_prompt(
27     const TokenizerModel& tokenizer,
28     std::string_view prompt);
29
30 [[nodiscard]] std::uint64_t tokenizer_contract_hash(const TokenizerModel& ↵
    ↵ tokenizer);
31
32 } // namespace lcqi

```

safetensors 头文件固定 dtype、shape、offset 和 payload 边界。它负责模型字节怎样被解释，不应该和数学推理逻辑混在一起。

Listing 2.5 include/lcqi/safetensors.hpp

```

1 #pragma once
2
3 #include <cstdint>

```

```

4 #include <filesystem>
5 #include <span>
6 #include <string>
7 #include <string_view>
8 #include <utility>
9 #include <vector>
10
11 namespace lcqi {
12
13 struct SafeTensorEntry {
14     std::string name;
15     std::string dtype;
16     std::vector<std::int64_t> shape;
17     std::uint64_t data_begin = 0;
18     std::uint64_t data_end = 0;
19
20     [[nodiscard]] std::uint64_t byte_size() const;
21 };
22
23 struct SafeTensorManifest {
24     std::uint64_t header_size = 0;
25     std::uint64_t data_start_offset = 0;
26     std::vector<std::pair<std::string, std::string>> metadata;
27     std::vector<SafeTensorEntry> tensors;
28
29     [[nodiscard]] const SafeTensorEntry* find_tensor(std::string_view name) ↵
        ↵ const;
30 };
31
32 struct SafeTensorFile {
33     std::filesystem::path path;
34     SafeTensorManifest manifest;
35     std::vector<std::uint8_t> bytes;
36 };
37
38 [[nodiscard]] SafeTensorManifest load_safetensors_manifest(
39     const std::filesystem::path& path);
40
41 [[nodiscard]] SafeTensorFile load_safetensors_file(const std::filesystem::path& ↵
        ↵ path);
42
43 [[nodiscard]] std::vector<float> read_safetensor_f32_tensor(
44     const SafeTensorFile& file,
45     std::string_view name);
46
47 [[nodiscard]] std::span<const std::uint8_t> read_safetensor_tensor_bytes(
48     const SafeTensorFile& file,
49     std::string_view name);
50
51 } // namespace lcqi

```

2.4 Reference Decoder 与 GPT-2 合同

reference decoder 头文件定义张量、层配置、trace checkpoint、采样结果和 decode 函数。它是定位推理错误的主要合同，不只返回最终 token。

Listing 2.6 include/lcqi/reference_decoder.hpp

```

1 #pragma once
2
3 #include <cstdint>
4 #include <span>
5 #include <string>
6 #include <vector>
7
8 namespace lcqi {
9
10 struct LinearWeightsF32 {
11     std::int32_t input_size = 0;
12     std::int32_t output_size = 0;
13     std::vector<float> weights;
14     std::vector<float> bias;
15 };
16
17 struct DecoderConfig {
18     std::int32_t hidden_size = 0;
19     std::int32_t query_heads = 0;
20     std::int32_t kv_heads = 0;
21     std::int32_t head_dim = 0;
22     std::int32_t intermediate_size = 0;
23     std::int32_t vocab_size = 0;
24     std::int32_t max_context = 0;
25     float rms_epsilon = 1.0e-5F;
26     float rope_base = 10000.0F;
27 };
28
29 struct DecoderLayerWeightsF32 {
30     std::vector<float> rms_attention_weight;
31     LinearWeightsF32 wq;
32     LinearWeightsF32 wk;
33     LinearWeightsF32 wv;
34     LinearWeightsF32 wo;
35     std::vector<float> rms_mlp_weight;
36     LinearWeightsF32 w_gate;
37     LinearWeightsF32 w_up;
38     LinearWeightsF32 w_down;
39 };
40
41 struct ReferenceDecoderModel {
42     DecoderConfig config;
43     std::vector<float> token_embedding;
44     std::vector<DecoderLayerWeightsF32> layers;
45     std::vector<float> final_norm_weight;
46     LinearWeightsF32 lm_head;

```



```

99
100     DecoderConfig config_;
101     std::int32_t layer_count_ = 0;
102     std::int32_t filled_tokens_ = 0;
103     std::vector<float> keys_;
104     std::vector<float> values_;
105     std::vector<std::uint8_t> written_;
106 };
107
108 void rms_norm(std::span<const float> input,
109              std::span<const float> weight,
110              float epsilon,
111              std::span<float> output);
112
113 void linear_f32(const LinearWeightsF32& layer,
114                std::span<const float> input,
115                std::span<float> output);
116
117 void apply_rope(std::span<float> heads,
118                std::int32_t head_count,
119                std::int32_t head_dim,
120                std::int32_t model_position,
121                float rope_base);
122
123 void attention_decode(const DecoderConfig& config,
124                      const ReferenceKVCache& cache,
125                      std::int32_t layer_id,
126                      std::int32_t model_position,
127                      std::span<const float> query,
128                      std::span<float> output);
129
130 ReferenceDecodeResult run_reference_decode(const ReferenceDecoderModel& model,
131                                          std::span<const std::int32_t> ↵
132                                          ↵ token_ids);
133
134 ReferenceDecodeTraceResult run_reference_decode_with_trace(
135     const ReferenceDecoderModel& model,
136     std::span<const std::int32_t> token_ids);
137
138 ReferenceDecoderModel make_tiny_reference_decoder_model();
139 } // namespace lcqi

```

GPT-2 reference 头文件定义 GPT-2 配置、byte-level BPE tokenizer、HuggingFace safetensors 加载、forward、cached forward 和 greedy generation。它单独存在，是因为 GPT-2 使用 LayerNorm、绝对位置嵌入、packed QKV、GELU 和 tied lm head，不能直接塞进 LLaMA-like reference decoder。

Listing 2.7 include/lcqi/gpt2_reference.hpp

```

1 #pragma once
2
3 #include <cstdint>
4 #include <filesystem>

```

```

5  #include <span>
6  #include <string>
7  #include <string_view>
8  #include <unordered_map>
9  #include <vector>
10
11 namespace lcqi {
12
13 struct Gpt2Config {
14     std::int32_t hidden_size = 0;
15     std::int32_t head_count = 0;
16     std::int32_t layer_count = 0;
17     std::int32_t max_positions = 0;
18     std::int32_t vocab_size = 0;
19     std::int32_t intermediate_size = 0;
20     std::int32_t bos_token_id = -1;
21     std::int32_t eos_token_id = -1;
22     float layer_norm_epsilon = 1.0e-5F;
23     std::string activation_function = "gelu_new";
24 };
25
26 struct Gpt2LinearF32 {
27     std::int32_t input_size = 0;
28     std::int32_t output_size = 0;
29     std::vector<float> weights;
30     std::vector<float> bias;
31 };
32
33 struct Gpt2LayerWeightsF32 {
34     std::vector<float> ln_1_weight;
35     std::vector<float> ln_1_bias;
36     Gpt2LinearF32 c_attn;
37     Gpt2LinearF32 c_proj;
38     std::vector<float> ln_2_weight;
39     std::vector<float> ln_2_bias;
40     Gpt2LinearF32 c_fc;
41     Gpt2LinearF32 mlp_c_proj;
42 };
43
44 struct Gpt2ReferenceModel {
45     Gpt2Config config;
46     std::vector<float> token_embedding;
47     std::vector<float> position_embedding;
48     std::vector<Gpt2LayerWeightsF32> layers;
49     std::vector<float> final_ln_weight;
50     std::vector<float> final_ln_bias;
51     std::vector<float> lm_head_weight;
52     bool tie_lm_head_to_embedding = true;
53 };
54
55 struct Gpt2ForwardResult {
56     std::vector<float> logits;

```



```

57     std::int32_t predicted_token = 0;
58 };
59
60 class Gpt2KvCache {
61 public:
62     explicit Gpt2KvCache(const Gpt2Config& config);
63
64     void append(std::int32_t layer_id,
65                std::int32_t model_position,
66                std::span<const float> key,
67                std::span<const float> value);
68
69     [[nodiscard]] std::span<const float> key(std::int32_t layer_id,
70                                              std::int32_t model_position,
71                                              std::int32_t head) const;
72
73     [[nodiscard]] std::span<const float> value(std::int32_t layer_id,
74                                                std::int32_t model_position,
75                                                std::int32_t head) const;
76
77     [[nodiscard]] std::int32_t filled_tokens() const noexcept;
78     [[nodiscard]] std::size_t byte_size() const noexcept;
79
80 private:
81     [[nodiscard]] std::size_t base_offset(std::int32_t layer_id,
82                                           std::int32_t model_position,
83                                           std::int32_t head) const;
84     void validate_address(std::int32_t layer_id,
85                          std::int32_t model_position,
86                          std::int32_t head) const;
87     void validate_written(std::int32_t layer_id,
88                          std::int32_t model_position) const;
89
90     Gpt2Config config_;
91     std::int32_t filled_tokens_ = 0;
92     std::vector<float> keys_;
93     std::vector<float> values_;
94     std::vector<std::uint8_t> written_;
95 };
96
97 struct Gpt2Tokenizer {
98     std::int32_t bos_token_id = -1;
99     std::int32_t eos_token_id = -1;
100     std::unordered_map<std::string, std::int32_t> vocab;
101     std::vector<std::string> id_to_token;
102     std::unordered_map<std::string, std::int32_t> bpe_ranks;
103 };
104
105 [[nodiscard]] Gpt2Tokenizer load_gpt2_tokenizer(
106     const std::filesystem::path& vocab_json,
107     const std::filesystem::path& merges_txt,
108     std::int32_t bos_token_id,

```

```

109     std::int32_t eos_token_id);
110
111 [[nodiscard]] std::vector<std::int32_t> gpt2_encode(
112     const Gpt2Tokenizer& tokenizer,
113     std::string_view text);
114
115 [[nodiscard]] std::string gpt2_decode(
116     const Gpt2Tokenizer& tokenizer,
117     std::span<const std::int32_t> token_ids);
118
119 [[nodiscard]] Gpt2Config load_gpt2_config(const std::filesystem::path& ↵
    ↵ config_json);
120
121 [[nodiscard]] Gpt2ReferenceModel load_gpt2_reference_model(
122     const std::filesystem::path& config_json,
123     const std::filesystem::path& safetensors_path);
124
125 [[nodiscard]] Gpt2ReferenceModel load_gpt2_from_directory(
126     const std::filesystem::path& model_directory);
127
128 [[nodiscard]] Gpt2ForwardResult run_gpt2_forward(
129     const Gpt2ReferenceModel& model,
130     std::span<const std::int32_t> token_ids);
131
132 [[nodiscard]] Gpt2ForwardResult run_gpt2_forward_cached(
133     const Gpt2ReferenceModel& model,
134     Gpt2KvCache& cache,
135     std::int32_t token_id);
136
137 [[nodiscard]] std::vector<std::int32_t> gpt2_generate_greedy(
138     const Gpt2ReferenceModel& model,
139     std::span<const std::int32_t> prompt_token_ids,
140     std::int32_t max_new_tokens);
141
142 [[nodiscard]] std::vector<std::int32_t> gpt2_generate_greedy_cached(
143     const Gpt2ReferenceModel& model,
144     std::span<const std::int32_t> prompt_token_ids,
145     std::int32_t max_new_tokens);
146
147 [[nodiscard]] Gpt2ReferenceModel make_tiny_gpt2_reference_model();
148
149 } // namespace lcqi

```

第 3 章

Tiny MLP、Int8 Kernel 与基础 CLI

3.1 最短闭环：从模型文件到 logits

tiny MLP 路径不是为了证明 LCQI 已经是完整大模型引擎，而是为了建立最短的、可手算的、可测试的推理闭环。读者要把模型加载、输入检查、int8 linear、ReLU、第二层 logits、argmax 和 CLI 输出连起来看。

3.2 模型加载与基础推理

`model.cpp` 负责解析 tiny 文本模型、权重、scale、bias 和 shape。坏格式、坏维度和坏数值都应该在这里被拒绝，不能拖到 kernel 内部才崩溃。

Listing 3.1 src/model.cpp

```
1 #include <lcqi/model.hpp>
2
3 #include <fstream>
4 #include <limits>
5 #include <stdexcept>
6 #include <string>
7
8 namespace lcqi {
9 namespace {
10
11 constexpr std::int32_t LCQI_INT8_MIN_VALUE = -128;
12 constexpr std::int32_t LCQI_INT8_MAX_VALUE = 127;
13
14 std::string read_key(std::istream& input) {
15     std::string key;
16     if (!(input >> key)) {
17         throw std::runtime_error("unexpected end of model file");
18     }
19     return key;
20 }
21
22 void expect_key(std::istream& input, const std::string& expected) {
23     const std::string key = read_key(input);
24     if (key != expected) {
25         throw std::runtime_error("expected key '" + expected + "', got '" + key +
26     ↪ + "'");
27     }
28 }
29
30 std::int32_t read_i32(std::istream& input, const std::string& key) {
31     expect_key(input, key);
32     std::int32_t value = 0;
33     if (!(input >> value)) {
34         throw std::runtime_error("invalid int32 value for key '" + key + "'");
35     }
36 }
```

```

34     }
35     return value;
36 }
37
38 float read_float(std::istream& input, const std::string& key) {
39     expect_key(input, key);
40     float value = 0.0F;
41     if (!(input >> value)) {
42         throw std::runtime_error("invalid float value for key '" + key + "'");
43     }
44     return value;
45 }
46
47 std::vector<std::int8_t> read_i8_vector(std::istream& input,
48                                         const std::string& key,
49                                         std::int32_t count) {
50     expect_key(input, key);
51     if (count < 0) {
52         throw std::runtime_error("negative vector size for key '" + key + "'");
53     }
54     std::vector<std::int8_t> values(static_cast<std::size_t>(count));
55     for (std::int32_t i = 0; i < count; ++i) {
56         std::int32_t raw = 0;
57         if (!(input >> raw)) {
58             throw std::runtime_error("invalid int8 payload for key '" + key + "'");
59         }
60         if (raw < LCQI_INT8_MIN_VALUE || raw > LCQI_INT8_MAX_VALUE) {
61             throw std::runtime_error("int8 payload out of range for key '" +
62                                     key + "'");
63         }
64         values[static_cast<std::size_t>(i)] = static_cast<std::int8_t>(raw);
65     }
66     return values;
67 }
68
69 std::vector<float> read_float_vector(std::istream& input,
70                                      const std::string& key,
71                                      std::int32_t count) {
72     expect_key(input, key);
73     if (count < 0) {
74         throw std::runtime_error("negative vector size for key '" + key + "'");
75     }
76     std::vector<float> values(static_cast<std::size_t>(count));
77     for (std::int32_t i = 0; i < count; ++i) {
78         if (!(input >> values[static_cast<std::size_t>(i)])) {
79             throw std::runtime_error("invalid float payload for key '" + key + "'");
80         }
81     }
82     return values;
83 }

```

```

83
84 void validate_layer(const QuantizedLinearLayer& layer, const std::string& name)↵
    ↵ {
85     if (layer.input_size <= 0 || layer.output_size <= 0) {
86         throw std::runtime_error(name + " layer has invalid shape");
87     }
88     const std::size_t expected_weights =
89         static_cast<std::size_t>(layer.input_size) *
90         static_cast<std::size_t>(layer.output_size);
91     if (layer.weights.size() != expected_weights) {
92         throw std::runtime_error(name + " layer weight size mismatch");
93     }
94     if (layer.bias.size() != static_cast<std::size_t>(layer.output_size)) {
95         throw std::runtime_error(name + " layer bias size mismatch");
96     }
97 }
98
99 std::int32_t checked_element_count(std::int32_t rows,
100                                   std::int32_t columns,
101                                   const std::string& name) {
102     if (rows <= 0 || columns <= 0) {
103         throw std::runtime_error(name + " layer has invalid shape");
104     }
105     const std::int64_t count =
106         static_cast<std::int64_t>(rows) * static_cast<std::int64_t>(columns);
107     if (count > static_cast<std::int64_t>(std::numeric_limits<std::int32_t>::↵
    ↵ max())) {
108         throw std::runtime_error(name + " layer element count is too large");
109     }
110     return static_cast<std::int32_t>(count);
111 }
112
113 } // namespace
114
115 TinyMlpModel load_model(const std::filesystem::path& path) {
116     std::ifstream input(path);
117     if (!input) {
118         throw std::runtime_error("cannot open model file: " + path.string());
119     }
120
121     expect_key(input, "LCQI_MODEL_V1");
122
123     TinyMlpModel model;
124     model.input_size = read_i32(input, "input_size");
125     model.hidden_size = read_i32(input, "hidden_size");
126     model.output_size = read_i32(input, "output_size");
127
128     const std::int32_t hidden_weight_count =
129         checked_element_count(model.input_size, model.hidden_size, "hidden");
130     const std::int32_t output_weight_count =
131         checked_element_count(model.hidden_size, model.output_size, "output");
132

```

```

133     model.hidden.input_size = model.input_size;
134     model.hidden.output_size = model.hidden_size;
135     model.hidden.scale = read_float(input, "hidden_scale");
136     model.hidden.weights = read_i8_vector(
137         input,
138         "hidden_weights",
139         hidden_weight_count);
140     model.hidden.bias = read_float_vector(input, "hidden_bias", model.hidden_size);
141
142     model.output.input_size = model.hidden_size;
143     model.output.output_size = model.output_size;
144     model.output.scale = read_float(input, "output_scale");
145     model.output.weights = read_i8_vector(
146         input,
147         "output_weights",
148         output_weight_count);
149     model.output.bias = read_float_vector(input, "output_bias", model.output_size);
150
151     validate_layer(model.hidden, "hidden");
152     validate_layer(model.output, "output");
153     return model;
154 }
155
156 } // namespace lcqi

```

`inference.cpp` 执行 int8 linear、ReLU、第二层 logits 和 predicted class。它是系统中最短的“模型资产到输出”路径。

Listing 3.2 src/inference.cpp

```

1  #include <lcqi/inference.hpp>
2
3  #include <algorithm>
4  #include <cmath>
5  #include <stdexcept>
6
7  namespace lcqi {
8  namespace {
9
10 void validate_input_size(std::span<const float> input, std::int32_t expected) {
11     if (input.size() != static_cast<std::size_t>(expected)) {
12         throw std::runtime_error("input size does not match model shape");
13     }
14 }
15
16 std::int32_t argmax(std::span<const float> values) {
17     if (values.empty()) {
18         throw std::runtime_error("cannot take argmax of empty logits");
19     }
20     std::int32_t best_index = 0;
21     float best_value = values[0];

```

```

22     for (std::int32_t i = 1; i < static_cast<std::int32_t>(values.size()); ++i) ←
    ↪ {
23         const float candidate = values[static_cast<std::size_t>(i)];
24         if (candidate > best_value) {
25             best_value = candidate;
26             best_index = i;
27         }
28     }
29     return best_index;
30 }
31
32 } // namespace
33
34 void linear_i8(const QuantizedLinearLayer& layer,
35               std::span<const float> input,
36               std::span<float> output) {
37     if (input.size() != static_cast<std::size_t>(layer.input_size)) {
38         throw std::runtime_error("linear_i8 input size mismatch");
39     }
40     if (output.size() != static_cast<std::size_t>(layer.output_size)) {
41         throw std::runtime_error("linear_i8 output size mismatch");
42     }
43
44     for (std::int32_t out = 0; out < layer.output_size; ++out) {
45         const std::size_t row_base =
46             static_cast<std::size_t>(out) * static_cast<std::size_t>(layer. ←
    ↪ input_size);
47         float acc = layer.bias[static_cast<std::size_t>(out)];
48         for (std::int32_t in = 0; in < layer.input_size; ++in) {
49             const std::int8_t weight =
50                 layer.weights[row_base + static_cast<std::size_t>(in)];
51             acc += static_cast<float>(weight) * layer.scale *
52                 input[static_cast<std::size_t>(in)];
53         }
54         output[static_cast<std::size_t>(out)] = acc;
55     }
56 }
57
58 void relu_inplace(std::span<float> values) {
59     for (float& value : values) {
60         value = std::max(0.0F, value);
61     }
62 }
63
64 InferenceResult run_inference(const TinyMlpModel& model,
65                               std::span<const float> input) {
66     validate_input_size(input, model.input_size);
67
68     std::vector<float> hidden(static_cast<std::size_t>(model.hidden_size), 0.0F ←
    ↪ );
69     std::vector<float> logits(static_cast<std::size_t>(model.output_size), 0.0F ←
    ↪ );

```

```

70
71     linear_i8(model.hidden, input, hidden);
72     relu_inplace(hidden);
73     linear_i8(model.output, hidden, logits);
74
75     InferenceResult result;
76     result.predicted_class = argmax(logits);
77     result.logits = std::move(logits);
78     return result;
79 }
80
81 } // namespace lcqi

```

`main.cpp` 把模型路径和输入接到 `run_inference`，并打印结果。CLI 应保持薄层，不复制模型解析和推理逻辑。

Listing 3.3 src/main.cpp

```

1  #include <lcqi/inference.hpp>
2  #include <lcqi/model.hpp>
3
4  #include <chrono>
5  #include <cstdint>
6  #include <cstdlib>
7  #include <filesystem>
8  #include <iostream>
9  #include <span>
10 #include <stdexcept>
11 #include <string>
12 #include <vector>
13
14 namespace {
15
16 constexpr std::int32_t LCQI_DEFAULT_REPEAT = 1000;
17 constexpr int LCQI_EXPECTED_ARGC_MIN = 2;
18 constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
19
20 std::vector<float> parse_input(int argc, char** argv, std::int32_t ↵
    ↵ expected_size) {
21     if (argc < LCQI_EXPECTED_ARGC_MIN + expected_size) {
22         throw std::runtime_error("usage: lcqi_cli <model.txt> <input floats...>↵
    ↵ [repeat]");
23     }
24     std::vector<float> input(static_cast<std::size_t>(expected_size));
25     for (std::int32_t i = 0; i < expected_size; ++i) {
26         input[static_cast<std::size_t>(i)] =
27             std::stof(argv[LCQI_EXPECTED_ARGC_MIN + i]);
28     }
29     return input;
30 }
31
32 std::int32_t parse_repeat(int argc, char** argv, std::int32_t input_size) {
33     const int repeat_index = LCQI_EXPECTED_ARGC_MIN + input_size;

```



```

34     if (argc <= repeat_index) {
35         return LCQI_DEFAULT_REPEAT;
36     }
37     const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[↵
↵ repeat_index]));
38     if (repeat <= 0) {
39         throw std::runtime_error("repeat must be positive");
40     }
41     return repeat;
42 }
43
44 } // namespace
45
46 int main(int argc, char** argv) {
47     try {
48         if (argc < LCQI_EXPECTED_ARGC_MIN) {
49             throw std::runtime_error("usage: lcqi_cli <model.txt> <input floats↵
↵ ...> [repeat]");
50         }
51
52         const std::filesystem::path model_path = argv[1];
53         const lcqi::TinyMlpModel model = lcqi::load_model(model_path);
54         const std::vector<float> input = parse_input(argc, argv, model.↵
↵ input_size);
55         const std::int32_t repeat = parse_repeat(argc, argv, model.input_size);
56
57         const lcqi::InferenceResult result = lcqi::run_inference(model, input);
58         std::cout << "predicted_class " << result.predicted_class << "\n";
59         std::cout << "logits";
60         for (const float value : result.logits) {
61             std::cout << ' ' << value;
62         }
63         std::cout << "\n";
64
65         const auto begin = std::chrono::steady_clock::now();
66         std::int32_t checksum = 0;
67         for (std::int32_t i = 0; i < repeat; ++i) {
68             checksum += lcqi::run_inference(model, input).predicted_class;
69         }
70         const auto end = std::chrono::steady_clock::now();
71         const std::chrono::duration<double> elapsed = end - begin;
72         const double average_us =
73             elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double↵
↵ >(repeat);
74         std::cout << "repeat " << repeat << "\n";
75         std::cout << "average_us " << average_us << "\n";
76         std::cout << "checksum " << checksum << "\n";
77         return EXIT_SUCCESS;
78     } catch (const std::exception& error) {
79         std::cerr << "error: " << error.what() << "\n";
80         return EXIT_FAILURE;
81     }

```

```
82 }
```

3.3 Int8 Kernel 基础实现

`int8_kernels.cpp` 包含 scalar、packed、workspace、deterministic fixture 和误差比较。它是 AVX2 路径的 reference 对照，也是 benchmark 的正确性底座。

Listing 3.4 src/int8_kernels.cpp

```
1 #include <lcqi/int8_kernels.hpp>
2
3 #include <lcqi/inference.hpp>
4
5 #include <algorithm>
6 #include <chrono>
7 #include <cmath>
8 #include <stdexcept>
9 #include <string>
10
11 namespace lcqi {
12 namespace {
13
14 constexpr std::int32_t LCQI_I8_PATTERN_MODULUS = 23;
15 constexpr std::int32_t LCQI_I8_PATTERN_CENTER = 11;
16 constexpr std::int32_t LCQI_INPUT_PATTERN_MODULUS = 17;
17 constexpr std::int32_t LCQI_INPUT_PATTERN_CENTER = 8;
18 constexpr float LCQI_INPUT_PATTERN_SCALE = 0.125F;
19 constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
20 constexpr std::int32_t LCQI_SIMD_OUTPUT_BLOCK = 8;
21 constexpr std::size_t LCQI_BENCHMARK_BACKEND_COUNT = 3;
22 constexpr const char* LCQI_BACKEND_SCALAR = "scalar";
23 constexpr const char* LCQI_BACKEND_PACKED_SCALAR = "packed_scalar";
24 constexpr const char* LCQI_BACKEND_AVX2 = "avx2";
25
26 void require_positive(std::int32_t value, const char* name) {
27     if (value <= 0) {
28         throw std::runtime_error(std::string(name) + " must be positive");
29     }
30 }
31
32 std::size_t checked_size(std::int32_t value, const char* name) {
33     if (value < 0) {
34         throw std::runtime_error(std::string(name) + " cannot be negative");
35     }
36     return static_cast<std::size_t>(value);
37 }
38
39 void validate_quantized_layer(const QuantizedLinearLayer& layer) {
40     require_positive(layer.input_size, "input_size");
41     require_positive(layer.output_size, "output_size");
42     const std::size_t expected_weights =
43         checked_size(layer.input_size, "input_size") *
```

```

44     checked_size(layer.output_size, "output_size");
45     if (layer.weights.size() != expected_weights) {
46         throw std::runtime_error("quantized layer weight size mismatch");
47     }
48     if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
49         throw std::runtime_error("quantized layer bias size mismatch");
50     }
51 }
52
53 std::int32_t round_up_to_block(std::int32_t value, std::int32_t block_size) {
54     require_positive(value, "value");
55     require_positive(block_size, "block_size");
56     return ((value + block_size - 1) / block_size) * block_size;
57 }
58
59 float checksum(std::span<const float> values) {
60     float sum = 0.0F;
61     for (const float value : values) {
62         sum += value;
63     }
64     return sum;
65 }
66
67 void add_unavailable_backend(KernelBenchmarkResult& result, const char* name) {
68     result.backends.push_back(KernelBackendBenchmark{
69         .name = name,
70         .available = false,
71         .average_us = 0.0,
72         .max_abs_diff = 0.0F,
73         .checksum = 0.0F,
74     });
75 }
76
77 template <typename Callable>
78 double measure_average_us(std::int32_t repeat, Callable&& callable) {
79     require_positive(repeat, "repeat");
80     const auto begin = std::chrono::steady_clock::now();
81     for (std::int32_t index = 0; index < repeat; ++index) {
82         callable();
83     }
84     const auto end = std::chrono::steady_clock::now();
85     const std::chrono::duration<double> elapsed = end - begin;
86     return elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>(repeat);
87 }
88
89 } // namespace
90
91 PackedLinearI8 pack_linear_i8(const QuantizedLinearLayer& layer,
92                               std::int32_t output_block_size) {
93     validate_quantized_layer(layer);
94     require_positive(output_block_size, "output_block_size");

```

```

95
96     PackedLinearI8 packed;
97     packed.input_size = layer.input_size;
98     packed.output_size = layer.output_size;
99     packed.output_block_size = output_block_size;
100    packed.scale = layer.scale;
101    packed.bias = layer.bias;
102
103    const std::int32_t padded_outputs =
104        round_up_to_block(layer.output_size, output_block_size);
105    packed.packed_weights.assign(
106        checked_size(padded_outputs, "padded_outputs") *
107        checked_size(layer.input_size, "input_size"),
108        0);
109
110    std::size_t packed_index = 0;
111    for (std::int32_t output_block = 0; output_block < padded_outputs;
112         output_block += output_block_size) {
113        for (std::int32_t input_index = 0; input_index < layer.input_size; ++↵
↵ input_index) {
114            for (std::int32_t offset = 0; offset < output_block_size; ++offset)↵
↵ {
115                const std::int32_t output_index = output_block + offset;
116                if (output_index < layer.output_size) {
117                    const std::size_t source =
118                        checked_size(output_index, "output_index") *
119                        checked_size(layer.input_size, "input_size") +
120                        checked_size(input_index, "input_index");
121                    packed.packed_weights[packed_index] = layer.weights[source]↵
↵ ];
122                }
123                ++packed_index;
124            }
125        }
126    }
127    return packed;
128 }
129
130 void linear_i8_packed(const PackedLinearI8& layer,
131                     std::span<const float> input,
132                     std::span<float> output) {
133     std::vector<float> accumulators(
134         checked_size(layer.output_block_size, "output_block_size"),
135         0.0F);
136     linear_i8_packed_with_workspace(layer, input, output, accumulators);
137 }
138
139 void linear_i8_packed_with_workspace(const PackedLinearI8& layer,
140                                     std::span<const float> input,
141                                     std::span<float> output,
142                                     std::span<float> accumulator_workspace) {
143     require_positive(layer.input_size, "packed input_size");

```

```

144     require_positive(layer.output_size, "packed output_size");
145     require_positive(layer.output_block_size, "packed output_block_size");
146     if (input.size() != checked_size(layer.input_size, "input_size")) {
147         throw std::runtime_error("linear_i8_packed input size mismatch");
148     }
149     if (output.size() != checked_size(layer.output_size, "output_size")) {
150         throw std::runtime_error("linear_i8_packed output size mismatch");
151     }
152     if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
153         throw std::runtime_error("linear_i8_packed bias size mismatch");
154     }
155
156     const std::int32_t padded_outputs =
157         round_up_to_block(layer.output_size, layer.output_block_size);
158     const std::size_t expected_packed_size =
159         checked_size(padded_outputs, "padded_outputs") *
160         checked_size(layer.input_size, "input_size");
161     if (layer.packed_weights.size() != expected_packed_size) {
162         throw std::runtime_error("linear_i8_packed packed weight size mismatch" ←
163     ↪ );
164     }
165     if (accumulator_workspace.size() != checked_size(layer.output_block_size, " ←
166     ↪ output_block_size")) {
167         throw std::runtime_error("linear_i8_packed accumulator workspace size ←
168     ↪ mismatch");
169     }
170
171     std::size_t packed_index = 0;
172     for (std::int32_t output_block = 0; output_block < padded_outputs;
173         output_block += layer.output_block_size) {
174         std::fill(accumulator_workspace.begin(), accumulator_workspace.end(), ←
175     ↪ 0.0F);
176         for (std::int32_t offset = 0; offset < layer.output_block_size; ++ ←
177     ↪ offset) {
178             const std::int32_t output_index = output_block + offset;
179             if (output_index < layer.output_size) {
180                 accumulator_workspace[checked_size(offset, "offset")] =
181                 layer.bias[checked_size(output_index, "output_index")];
182             }
183         }
184
185         for (std::int32_t input_index = 0; input_index < layer.input_size; ++ ←
186     ↪ input_index) {
187             const float input_value = input[checked_size(input_index, " ←
188     ↪ input_index")];
189             for (std::int32_t offset = 0; offset < layer.output_block_size; ++ ←
190     ↪ offset) {
191                 const std::int32_t output_index = output_block + offset;
192                 const std::int8_t weight = layer.packed_weights[packed_index ←
193     ↪ ++];
194                 if (output_index < layer.output_size) {
195                     accumulator_workspace[checked_size(offset, "offset")] +=

```

```

187         static_cast<float>(weight) * layer.scale * input_value;
188     }
189 }
190 }
191
192     for (std::int32_t offset = 0; offset < layer.output_block_size; ++offset) {
193         const std::int32_t output_index = output_block + offset;
194         if (output_index < layer.output_size) {
195             output[checked_size(output_index, "output_index")] =
196                 accumulator_workspace[checked_size(offset, "offset")];
197         }
198     }
199 }
200 }
201
202 QuantizedLinearLayer make_deterministic_i8_layer(std::int32_t input_size,
203                                                  std::int32_t output_size,
204                                                  float scale) {
205     require_positive(input_size, "input_size");
206     require_positive(output_size, "output_size");
207     QuantizedLinearLayer layer;
208     layer.input_size = input_size;
209     layer.output_size = output_size;
210     layer.scale = scale;
211     const std::size_t weight_count =
212         checked_size(input_size, "input_size") * checked_size(output_size, "output_size");
213     layer.weights.resize(weight_count);
214     layer.bias.resize(checked_size(output_size, "output_size"));
215     for (std::int32_t output_index = 0; output_index < output_size; ++output_index) {
216         layer.bias[checked_size(output_index, "output_index")] =
217             static_cast<float>((output_index % LCQI_INPUT_PATTERN_MODULUS) -
218                               LCQI_INPUT_PATTERN_CENTER) *
219             LCQI_INPUT_PATTERN_SCALE;
220         for (std::int32_t input_index = 0; input_index < input_size; ++input_index) {
221             const std::int32_t raw =
222                 ((output_index * input_size + input_index) % LCQI_I8_PATTERN_MODULUS) -
223                 LCQI_I8_PATTERN_CENTER;
224             layer.weights[checked_size(output_index, "output_index") *
225                           checked_size(input_size, "input_size") +
226                           checked_size(input_index, "input_index")] =
227                 static_cast<std::int8_t>(raw);
228         }
229     }
230     return layer;
231 }
232
233 std::vector<float> make_deterministic_input(std::int32_t input_size) {

```

```

234     require_positive(input_size, "input_size");
235     std::vector<float> input(checkered_size(input_size, "input_size"));
236     for (std::int32_t input_index = 0; input_index < input_size; ++input_index) {
237         input[checked_size(input_index, "input_index")] =
238             static_cast<float>((input_index % LCQI_INPUT_PATTERN_MODULUS) -
239                               LCQI_INPUT_PATTERN_CENTER) *
240             LCQI_INPUT_PATTERN_SCALE;
241     }
242     return input;
243 }
244
245 float max_abs_diff(std::span<const float> lhs, std::span<const float> rhs) {
246     if (lhs.size() != rhs.size()) {
247         throw std::runtime_error("max_abs_diff size mismatch");
248     }
249     float diff = 0.0F;
250     for (std::size_t index = 0; index < lhs.size(); ++index) {
251         diff = std::max(diff, std::fabs(lhs[index] - rhs[index]));
252     }
253     return diff;
254 }
255
256 KernelBenchmarkResult benchmark_linear_i8_case(const KernelBenchmarkCase&
257     benchmark_case) {
258     require_positive(benchmark_case.input_size, "benchmark input_size");
259     require_positive(benchmark_case.output_size, "benchmark output_size");
260     require_positive(benchmark_case.output_block_size, "benchmark
261     output_block_size");
262     require_positive(benchmark_case.repeat, "benchmark repeat");
263
264     const QuantizedLinearLayer layer =
265         make_deterministic_i8_layer(benchmark_case.input_size,
266                                     benchmark_case.output_size,
267                                     0.015625F);
268     const PackedLinearI8 packed = pack_linear_i8(layer, benchmark_case.
269     output_block_size);
270     const PackedLinearI8 packed_simd = pack_linear_i8(layer,
271     LCQI_SIMD_OUTPUT_BLOCK);
272     const std::vector<float> input = make_deterministic_input(benchmark_case.
273     input_size);
274     std::vector<float> scalar_output(checked_size(benchmark_case.output_size, "
275     output_size"),
276                                     0.0F);
277     std::vector<float> packed_output(scalar_output.size(), 0.0F);
278     std::vector<float> simd_output(scalar_output.size(), 0.0F);
279     std::vector<float> packed_workspace(
280         checked_size(benchmark_case.output_block_size, "output_block_size"),
281         0.0F);
282
283     linear_i8(layer, input, scalar_output);
284     linear_i8_packed_with_workspace(packed, input, packed_output,

```

```

    ↪ packed_workspace);

279
280 KernelBenchmarkResult result;
281 result.benchmark_case = benchmark_case;
282 result.backends.reserve(LCQI_BENCHMARK_BACKEND_COUNT);
283 result.backends.push_back(KernelBackendBenchmark{
284     .name = LCQI_BACKEND_SCALAR,
285     .available = true,
286     .average_us = measure_average_us(benchmark_case.repeat,
287                                     [&]() { linear_i8(layer, input, ↪
    ↪ scalar_output); })),
288     .max_abs_diff = 0.0F,
289     .checksum = checksum(scalar_output),
290 });
291 result.backends.push_back(KernelBackendBenchmark{
292     .name = LCQI_BACKEND_PACKED_SCALAR,
293     .available = true,
294     .average_us = measure_average_us(
295         benchmark_case.repeat,
296         [&]() { linear_i8_packed_with_workspace(packed,
297                                                  input,
298                                                  packed_output,
299                                                  packed_workspace); })),
300     .max_abs_diff = max_abs_diff(scalar_output, packed_output),
301     .checksum = checksum(packed_output),
302 });
303 if (linear_i8_packed_avx2_available()) {
304     linear_i8_packed_avx2(packed_simd, input, simd_output);
305     result.backends.push_back(KernelBackendBenchmark{
306         .name = LCQI_BACKEND_AVX2,
307         .available = true,
308         .average_us = measure_average_us(
309             benchmark_case.repeat,
310             [&]() { linear_i8_packed_avx2(packed_simd, input, simd_output); ↪
    ↪ }),
311         .max_abs_diff = max_abs_diff(scalar_output, simd_output),
312         .checksum = checksum(simd_output),
313     });
314 } else {
315     add_unavailable_backend(result, LCQI_BACKEND_AVX2);
316 }
317 return result;
318 }
319
320 } // namespace lcqi
321
322 #if !defined(LCQI_ENABLE_AVX2)
323 namespace lcqi {
324
325 bool linear_i8_packed_avx2_available() noexcept {
326     return false;
327 }

```



```

39         std::span<float> output) {
40     require_positive(layer.input_size, "packed input_size");
41     require_positive(layer.output_size, "packed output_size");
42     if (layer.output_block_size != LCQI_AVX2_OUTPUT_BLOCK) {
43         throw std::runtime_error("AVX2 packed kernel requires output_block_size ←
↵ == 8");
44     }
45     if (input.size() != checked_size(layer.input_size, "input_size")) {
46         throw std::runtime_error("AVX2 packed kernel input size mismatch");
47     }
48     if (output.size() != checked_size(layer.output_size, "output_size")) {
49         throw std::runtime_error("AVX2 packed kernel output size mismatch");
50     }
51     if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
52         throw std::runtime_error("AVX2 packed kernel bias size mismatch");
53     }
54     const std::int32_t padded_outputs =
55         round_up_to_block(layer.output_size, layer.output_block_size);
56     const std::size_t expected_packed_size =
57         checked_size(padded_outputs, "padded_outputs") *
58         checked_size(layer.input_size, "input_size");
59     if (layer.packed_weights.size() != expected_packed_size) {
60         throw std::runtime_error("AVX2 packed kernel packed weight size ←
↵ mismatch");
61     }
62 }
63
64 } // namespace
65
66 bool linear_i8_packed_avx2_available() noexcept {
67     #if defined(__GNUC__) || defined(__clang__)
68         __builtin_cpu_init();
69         return __builtin_cpu_supports("avx2") && __builtin_cpu_supports("fma");
70     #else
71         return true;
72     #endif
73 }
74
75 void linear_i8_packed_avx2(const PackedLinearI8& layer,
76     std::span<const float> input,
77     std::span<float> output) {
78     if (!linear_i8_packed_avx2_available()) {
79         throw std::runtime_error("AVX2/FMA is not available on this CPU");
80     }
81     validate_avx2_contract(layer, input, output);
82
83     const std::int32_t padded_outputs =
84         round_up_to_block(layer.output_size, layer.output_block_size);
85     std::size_t packed_index = 0;
86     alignas(32) float block_output[LCQI_AVX2_OUTPUT_BLOCK] = {};
87
88     for (std::int32_t output_block = 0; output_block < padded_outputs;

```

```

89     output_block += LCQI_AVX2_OUTPUT_BLOCK) {
90     __m256 accumulator = _mm256_setzero_ps();
91
92     for (std::int32_t offset = 0; offset < LCQI_AVX2_OUTPUT_BLOCK; ++offset) {
93         const std::int32_t output_index = output_block + offset;
94         block_output[checked_size(offset, "offset")] =
95             output_index < layer.output_size
96             ? layer.bias[checked_size(output_index, "output_index")]
97             : 0.0F;
98     }
99     accumulator = _mm256_load_ps(block_output);
100
101     for (std::int32_t input_index = 0; input_index < layer.input_size; ++input_index) {
102         std::uint64_t packed_bytes = 0;
103         std::memcpy(&packed_bytes,
104             layer.packed_weights.data() + packed_index,
105             sizeof(packed_bytes));
106         const __m128i packed_i8 =
107             _mm_cvtsi64_si128(static_cast<long long>(packed_bytes));
108         packed_index += LCQI_AVX2_OUTPUT_BLOCK;
109         const __m256i weights_i32 = _mm256_cvtepi8_epi32(packed_i8);
110         const __m256 weights_f32 = _mm256_cvtepi32_ps(weights_i32);
111         const __m256 input_scaled =
112             _mm256_set1_ps(input[checked_size(input_index, "input_index")] * layer.scale);
113         accumulator = _mm256_fmadd_ps(weights_f32, input_scaled, accumulator);
114     }
115
116     _mm256_store_ps(block_output, accumulator);
117     for (std::int32_t offset = 0; offset < LCQI_AVX2_OUTPUT_BLOCK; ++offset) {
118         const std::int32_t output_index = output_block + offset;
119         if (output_index < layer.output_size) {
120             output[checked_size(output_index, "output_index")] =
121                 block_output[checked_size(offset, "offset")];
122         }
123     }
124 }
125 }
126
127 } // namespace lcqi
128
129 #endif

```

第 4 章

Reference Decoder、GPT-2 与模型导入

4.1 从 tiny MLP 走向 decoder-only 模型

Reference decoder 和 GPT-2 reference path 是 LCQI 从教学闭环走向真实模型导入的关键层。这里的代码更长，是因为真实模型不只有矩阵乘法：它还需要 tokenizer、位置、归一化、QKV 拆分、attention mask、KV cache、激活函数、lm head、权重命名映射、dtype 转换和坏资产拒绝。

4.2 Tiny Reference Decoder

`reference_decoder.cpp` 实现 embedding、RMSNorm、Q/K/V、RoPE、KV cache、attention、SwiGLU、final norm、logits 和 sampler，并在关键位置生成 trace checkpoint。读者应按 checkpoint 顺序读，而不是从最终 logits 倒推。

Listing 4.1 src/reference_decoder.cpp

```
1 #include <lcqi/reference_decoder.hpp>
2
3 #include <algorithm>
4 #include <array>
5 #include <cmath>
6 #include <limits>
7 #include <stdexcept>
8 #include <string>
9
10 namespace lcqi {
11 namespace {
12
13 constexpr float LCQI_SOFTMAX_NEGATIVE_INFINITY = -std::numeric_limits<float>::infinity();
14 constexpr std::int32_t LCQI_ROPE_PAIR_WIDTH = 2;
15 constexpr const char* LCQI_TRACE_DTYPE_F32 = "f32";
16 constexpr const char* LCQI_TRACE_LAYOUT_CONTIGUOUS = "contiguous";
17 constexpr const char* LCQI_TRACE_LAYOUT_ROW_MAJOR = "row_major";
18 constexpr const char* LCQI_TRACE_LAYOUT_KV_CONTIGUOUS = "kv_contiguous";
19
20 void require_positive(std::int32_t value, const char* name) {
21     if (value <= 0) {
22         throw std::runtime_error(std::string(name) + " must be positive");
23     }
24 }
25
26 void validate_config(const DecoderConfig& config) {
27     require_positive(config.hidden_size, "hidden_size");
28     require_positive(config.query_heads, "query_heads");
29     require_positive(config.kv_heads, "kv_heads");
30     require_positive(config.head_dim, "head_dim");
31     require_positive(config.intermediate_size, "intermediate_size");
32     require_positive(config.vocab_size, "vocab_size");
```

```

33     require_positive(config.max_context, "max_context");
34     if (config.query_heads % config.kv_heads != 0) {
35         throw std::runtime_error("query_heads must be divisible by kv_heads");
36     }
37     if (config.hidden_size != config.query_heads * config.head_dim) {
38         throw std::runtime_error("hidden_size must equal query_heads * head_dim↵
↵ ");
39     }
40     if (config.head_dim % LCQI_ROPE_PAIR_WIDTH != 0) {
41         throw std::runtime_error("head_dim must be even for RoPE");
42     }
43     if (config.rope_base <= 1.0F) {
44         throw std::runtime_error("rope_base must be greater than 1");
45     }
46 }
47
48 std::size_t checked_size(std::int32_t value, const char* name) {
49     if (value < 0) {
50         throw std::runtime_error(std::string(name) + " cannot be negative");
51     }
52     return static_cast<std::size_t>(value);
53 }
54
55 std::int32_t kv_width(const DecoderConfig& config) {
56     return config.kv_heads * config.head_dim;
57 }
58
59 void validate_linear(const LinearWeightsF32& layer, const char* name) {
60     require_positive(layer.input_size, "linear input_size");
61     require_positive(layer.output_size, "linear output_size");
62     const std::size_t expected_weights =
63         checked_size(layer.input_size, name) * checked_size(layer.output_size, ↵
↵ name);
64     if (layer.weights.size() != expected_weights) {
65         throw std::runtime_error(std::string(name) + " weight size mismatch");
66     }
67     if (!layer.bias.empty() &&
68         layer.bias.size() != checked_size(layer.output_size, name)) {
69         throw std::runtime_error(std::string(name) + " bias size mismatch");
70     }
71 }
72
73 void validate_span_size(std::size_t actual, std::int32_t expected, const char* ↵
↵ name) {
74     if (actual != checked_size(expected, name)) {
75         throw std::runtime_error(std::string(name) + " size mismatch");
76     }
77 }
78
79 std::int32_t argmax(std::span<const float> values) {
80     if (values.empty()) {
81         throw std::runtime_error("cannot take argmax of empty values");

```

```

82     }
83     std::int32_t best_index = 0;
84     float best_value = values[0];
85     for (std::int32_t index = 1; index < static_cast<std::int32_t>(values.size()
↵    )); ++index) {
86         const float candidate = values[static_cast<std::size_t>(index)];
87         if (candidate > best_value) {
88             best_value = candidate;
89             best_index = index;
90         }
91     }
92     return best_index;
93 }
94
95 float silu(float value) {
96     return value / (1.0F + std::exp(-value));
97 }
98
99 std::span<const float> row_span(std::span<const float> values,
100                                std::int32_t row,
101                                std::int32_t row_width) {
102     const std::size_t begin = checked_size(row, "row") * checked_size(row_width,
↵    , "row_width");
103     return values.subspan(begin, checked_size(row_width, "row_width"));
104 }
105
106 void add_inplace(std::span<float> target, std::span<const float> source) {
107     if (target.size() != source.size()) {
108         throw std::runtime_error("residual add size mismatch");
109     }
110     for (std::size_t index = 0; index < target.size(); ++index) {
111         target[index] += source[index];
112     }
113 }
114
115 void swiglu_mlp(const DecoderLayerWeightsF32& layer,
116                std::span<const float> input,
117                std::span<float> output) {
118     std::vector<float> gate(checked_size(layer.w_gate.output_size, "gate"), 0.0F
↵    );
119     std::vector<float> up(checked_size(layer.w_up.output_size, "up"), 0.0F);
120     std::vector<float> hidden(gate.size(), 0.0F);
121     linear_f32(layer.w_gate, input, gate);
122     linear_f32(layer.w_up, input, up);
123     if (gate.size() != up.size()) {
124         throw std::runtime_error("SwiGLU gate/up size mismatch");
125     }
126     for (std::size_t index = 0; index < hidden.size(); ++index) {
127         hidden[index] = silu(gate[index]) * up[index];
128     }
129     linear_f32(layer.w_down, hidden, output);
130 }

```

```

131
132 float checksum(std::span<const float> values) {
133     float sum = 0.0F;
134     for (const float value : values) {
135         sum += value;
136     }
137     return sum;
138 }
139
140 float max_abs(std::span<const float> values) {
141     float result = 0.0F;
142     for (const float value : values) {
143         result = std::max(result, std::fabs(value));
144     }
145     return result;
146 }
147
148 std::vector<std::int32_t> contiguous_stride(std::span<const std::int32_t> shape ←
    ↪ ) {
149     std::vector<std::int32_t> stride(shape.size(), 1);
150     std::int32_t running = 1;
151     for (std::int32_t index = static_cast<std::int32_t>(shape.size()) - 1; ←
    ↪ index >= 0; --index) {
152         stride[checked_size(index, "stride index")] = running;
153         running *= shape[checked_size(index, "shape index")];
154     }
155     return stride;
156 }
157
158 void add_checkpoint(std::vector<ReferenceTensorCheckpoint>* checkpoints,
159                    const char* name,
160                    std::int32_t token_position,
161                    std::int32_t layer_id,
162                    std::span<const std::int32_t> shape,
163                    const char* layout,
164                    std::span<const float> values) {
165     if (checkpoints == nullptr) {
166         return;
167     }
168     ReferenceTensorCheckpoint checkpoint;
169     checkpoint.name = name;
170     checkpoint.token_position = token_position;
171     checkpoint.layer_id = layer_id;
172     checkpoint.shape.assign(shape.begin(), shape.end());
173     checkpoint.stride = contiguous_stride(shape);
174     checkpoint.dtype = LCQI_TRACE_DTYPE_F32;
175     checkpoint.layout = layout;
176     checkpoint.checksum = checksum(values);
177     checkpoint.max_abs = max_abs(values);
178     checkpoint.values.assign(values.begin(), values.end());
179     checkpoints->push_back(std::move(checkpoint));
180 }

```

```

181
182 void swiglu_mlp_with_trace(const DecoderLayerWeightsF32& layer,
183                             std::span<const float> input,
184                             std::span<float> output,
185                             std::int32_t token_position,
186                             std::int32_t layer_id,
187                             std::vector<ReferenceTensorCheckpoint>* checkpoints)↵
    ↵ {
188     std::vector<float> gate(check_size(layer.w_gate.output_size, "gate"), 0.0↵
    ↵ F);
189     std::vector<float> up(check_size(layer.w_up.output_size, "up"), 0.0F);
190     std::vector<float> hidden(gate.size(), 0.0F);
191     linear_f32(layer.w_gate, input, gate);
192     linear_f32(layer.w_up, input, up);
193     if (gate.size() != up.size()) {
194         throw std::runtime_error("SwiGLU gate/up size mismatch");
195     }
196     const std::array<std::int32_t, 1> intermediate_shape{layer.w_gate.↵
    ↵ output_size};
197     add_checkpoint(checkpoints,
198                     "mlp_gate",
199                     token_position,
200                     layer_id,
201                     intermediate_shape,
202                     LCQI_TRACE_LAYOUT_CONTIGUOUS,
203                     gate);
204     add_checkpoint(checkpoints,
205                     "mlp_up",
206                     token_position,
207                     layer_id,
208                     intermediate_shape,
209                     LCQI_TRACE_LAYOUT_CONTIGUOUS,
210                     up);
211     for (std::size_t index = 0; index < hidden.size(); ++index) {
212         hidden[index] = silu(gate[index]) * up[index];
213     }
214     add_checkpoint(checkpoints,
215                     "mlp_hidden",
216                     token_position,
217                     layer_id,
218                     intermediate_shape,
219                     LCQI_TRACE_LAYOUT_CONTIGUOUS,
220                     hidden);
221     linear_f32(layer.w_down, hidden, output);
222 }
223
224 void forward_layer(const DecoderConfig& config,
225                   const DecoderLayerWeightsF32& layer,
226                   std::int32_t layer_id,
227                   std::int32_t model_position,
228                   ReferenceKVCache& cache,
229                   std::span<float> hidden,

```



```

230         std::vector<ReferenceTensorCheckpoint>* checkpoints = ↵
↵ nullptr) {
231     validate_span_size(hidden.size(), config.hidden_size, "hidden");
232
233     std::vector<float> normed(checked_size(config.hidden_size, "hidden_size"), ↵
↵ 0.0F);
234     std::vector<float> query(checked_size(config.hidden_size, "hidden_size"), ↵
↵ 0.0F);
235     std::vector<float> key(checked_size(kv_width(config), "kv_width"), 0.0F);
236     std::vector<float> value(checked_size(kv_width(config), "kv_width"), 0.0F);
237     std::vector<float> attention(checked_size(config.hidden_size, "hidden_size"↵
↵ ), 0.0F);
238     std::vector<float> attention_projected(checked_size(config.hidden_size, "↵
↵ hidden_size"), 0.0F);
239     std::vector<float> mlp(checked_size(config.hidden_size, "hidden_size"), 0.0↵
↵ F);
240
241     const std::array<std::int32_t, 1> hidden_shape{config.hidden_size};
242     const std::array<std::int32_t, 2> query_shape{config.query_heads, config.↵
↵ head_dim};
243     const std::array<std::int32_t, 2> kv_shape{config.kv_heads, config.head_dim↵
↵ };
244     const std::array<std::int32_t, 4> kv_slot_shape{
245         1, 1, config.kv_heads, config.head_dim};
246
247     rms_norm(hidden, layer.rms_attention_weight, config.rms_epsilon, normed);
248     add_checkpoint(checkpoints,
249         "attn_norm",
250         model_position,
251         layer_id,
252         hidden_shape,
253         LCQI_TRACE_LAYOUT_CONTIGUOUS,
254         normed);
255     linear_f32(layer.wq, normed, query);
256     linear_f32(layer.wk, normed, key);
257     linear_f32(layer.wv, normed, value);
258     add_checkpoint(checkpoints,
259         "q_before_rope",
260         model_position,
261         layer_id,
262         query_shape,
263         LCQI_TRACE_LAYOUT_CONTIGUOUS,
264         query);
265     add_checkpoint(checkpoints,
266         "k_before_rope",
267         model_position,
268         layer_id,
269         kv_shape,
270         LCQI_TRACE_LAYOUT_CONTIGUOUS,
271         key);
272     add_checkpoint(checkpoints,
273         "v",

```

```

274         model_position,
275         layer_id,
276         kv_shape,
277         LCQI_TRACE_LAYOUT_CONTIGUOUS,
278         value);
279     apply_rope(query, config.query_heads, config.head_dim, model_position, ↵
↵ config.rope_base);
280     apply_rope(key, config.kv_heads, config.head_dim, model_position, config.↵
↵ rope_base);
281     add_checkpoint(checkpoints,
282         "q_after_rope",
283         model_position,
284         layer_id,
285         query_shape,
286         LCQI_TRACE_LAYOUT_CONTIGUOUS,
287         query);
288     add_checkpoint(checkpoints,
289         "k_after_rope",
290         model_position,
291         layer_id,
292         kv_shape,
293         LCQI_TRACE_LAYOUT_CONTIGUOUS,
294         key);
295
296     cache.append(layer_id, model_position, key, value);
297     add_checkpoint(checkpoints,
298         "kv_cache_key_slot",
299         model_position,
300         layer_id,
301         kv_slot_shape,
302         LCQI_TRACE_LAYOUT_KV_CONTIGUOUS,
303         key);
304     attention_decode(config, cache, layer_id, model_position, query, attention)↵
↵ ;
305     add_checkpoint(checkpoints,
306         "attention_out",
307         model_position,
308         layer_id,
309         hidden_shape,
310         LCQI_TRACE_LAYOUT_CONTIGUOUS,
311         attention);
312     linear_f32(layer.wo, attention, attention_projected);
313     add_inplace(hidden, attention_projected);
314     add_checkpoint(checkpoints,
315         "post_attention_residual",
316         model_position,
317         layer_id,
318         hidden_shape,
319         LCQI_TRACE_LAYOUT_CONTIGUOUS,
320         hidden);
321
322     rms_norm(hidden, layer.rms_mlp_weight, config.rms_epsilon, normed);

```

```

323     add_checkpoint(checkpoints,
324                     "mlp_norm",
325                     model_position,
326                     layer_id,
327                     hidden_shape,
328                     LCQI_TRACE_LAYOUT_CONTIGUOUS,
329                     normed);
330     if (checkpoints == nullptr) {
331         swiglu_mlp(layer, normed, mlp);
332     } else {
333         swiglu_mlp_with_trace(layer, normed, mlp, model_position, layer_id, ↵
↵ checkpoints);
334     }
335     add_inplace(hidden, mlp);
336     add_checkpoint(checkpoints,
337                     "layer_out",
338                     model_position,
339                     layer_id,
340                     hidden_shape,
341                     LCQI_TRACE_LAYOUT_CONTIGUOUS,
342                     hidden);
343 }
344
345 LinearWeightsF32 make_linear(std::int32_t input_size,
346                               std::int32_t output_size,
347                               std::vector<float> weights,
348                               std::vector<float> bias = {}) {
349     LinearWeightsF32 layer;
350     layer.input_size = input_size;
351     layer.output_size = output_size;
352     layer.weights = std::move(weights);
353     layer.bias = std::move(bias);
354     validate_linear(layer, "tiny linear");
355     return layer;
356 }
357
358 std::vector<float> zeros(std::int32_t count) {
359     return std::vector<float>(checked_size(count, "zero count"), 0.0F);
360 }
361
362 } // namespace
363
364 ReferenceDecodeTraceResult run_reference_decode_with_trace(
365     const ReferenceDecoderModel& model,
366     std::span<const std::int32_t> token_ids) {
367     validate_config(model.config);
368     if (token_ids.empty()) {
369         throw std::runtime_error("reference decode needs at least one token");
370     }
371     if (token_ids.size() > checked_size(model.config.max_context, "max_context" ↵
↵ )) {
372         throw std::runtime_error("token_ids exceed max_context");

```

```

373     }
374     if (model.token_embedding.size() !=
375         checked_size(model.config.vocab_size, "vocab_size") *
376         checked_size(model.config.hidden_size, "hidden_size")) {
377         throw std::runtime_error("token embedding size mismatch");
378     }
379     if (model.final_norm_weight.size() != checked_size(model.config.hidden_size,
380 ↪ , "hidden_size")) {
381         throw std::runtime_error("final norm weight size mismatch");
382     }
383     validate_linear(model.lm_head, "lm_head");
384
385     ReferenceDecodeTraceResult trace;
386     ReferenceKVCache cache(model.config, static_cast<std::int32_t>(model.layers.
387 ↪ .size()));
388     std::vector<float> hidden(checked_size(model.config.hidden_size, "
389 ↪ hidden_size"), 0.0F);
390     const std::array<std::int32_t, 1> hidden_shape{model.config.hidden_size};
391     for (std::int32_t position = 0; position < static_cast<std::int32_t>(
392 ↪ token_ids.size());
393         ++position) {
394         const std::int32_t token_id = token_ids[checked_size(position, "
395 ↪ position")];
396         if (token_id < 0 || token_id >= model.config.vocab_size) {
397             throw std::runtime_error("token id out of vocabulary");
398         }
399         const std::span<const float> embedding =
400             row_span(model.token_embedding, token_id, model.config.hidden_size)
401 ↪ ;
402         std::copy(embedding.begin(), embedding.end(), hidden.begin());
403         add_checkpoint(&trace.checkpoints,
404             "embedding",
405             position,
406             -1,
407             hidden_shape,
408             LCQI_TRACE_LAYOUT_ROW_MAJOR,
409             hidden);
410         for (std::int32_t layer_id = 0; layer_id < static_cast<std::int32_t>(
411 ↪ model.layers.size());
412             ++layer_id) {
413             forward_layer(model.config,
414                 model.layers[checked_size(layer_id, "layer_id")],
415                 layer_id,
416                 position,
417                 cache,
418                 hidden,
419                 &trace.checkpoints);
420         }
421     }
422
423     std::vector<float> normed(checked_size(model.config.hidden_size, "
424 ↪ hidden_size"), 0.0F);

```

```

417     std::vector<float> logits(checkered_size(model.config.vocab_size, "vocab_size" ←
    ↪ "), 0.0F);
418     const std::array<std::int32_t, 1> logits_shape{model.config.vocab_size};
419     const std::int32_t last_position = static_cast<std::int32_t>(token_ids.size ←
    ↪ ()) - 1;
420     rms_norm(hidden, model.final_norm_weight, model.config.rms_epsilon, normed) ←
    ↪ ;
421     add_checkpoint(&trace.checkpoints,
422                   "final_norm",
423                   last_position,
424                   -1,
425                   hidden_shape,
426                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
427                   normed);
428     linear_f32(model.lm_head, normed, logits);
429     add_checkpoint(&trace.checkpoints,
430                   "logits",
431                   last_position,
432                   -1,
433                   logits_shape,
434                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
435                   logits);
436
437     trace.result.predicted_token = argmax(logits);
438     trace.result.logits = std::move(logits);
439     return trace;
440 }
441
442 ReferenceKVCache::ReferenceKVCache(const DecoderConfig& config, std::int32_t ←
    ↪ layer_count)
443 : config_(config),
444   layer_count_(layer_count) {
445     validate_config(this->config_);
446     require_positive(this->layer_count_, "layer_count");
447     const std::size_t element_count =
448         checked_size(this->layer_count_, "layer_count") *
449         checked_size(this->config_.max_context, "max_context") *
450         checked_size(this->config_.kv_heads, "kv_heads") *
451         checked_size(this->config_.head_dim, "head_dim");
452     this->keys_.assign(element_count, 0.0F);
453     this->values_.assign(element_count, 0.0F);
454     this->written_.assign(
455         checked_size(this->layer_count_, "layer_count") *
456         checked_size(this->config_.max_context, "max_context"),
457         0);
458 }
459
460 void ReferenceKVCache::append(std::int32_t layer_id,
461                               std::int32_t model_position,
462                               std::span<const float> key,
463                               std::span<const float> value) {
464     validate_span_size(key.size(), kv_width(this->config_), "KV key");

```

```

465     validate_span_size(value.size(), kv_width(this->config_), "KV value");
466     this->validate_address(layer_id, model_position, 0);
467     for (std::int32_t kv_head = 0; kv_head < this->config_.kv_heads; ++kv_head) {
468         const std::size_t target = this->base_offset(layer_id, model_position,
469         kv_head);
469         const std::size_t source =
470             checked_size(kv_head, "kv_head") * checked_size(this->config_.
471         head_dim, "head_dim");
471         std::copy_n(key.begin() + static_cast<std::ptrdiff_t>(source),
472             this->config_.head_dim,
473             this->keys_.begin() + static_cast<std::ptrdiff_t>(target));
474         std::copy_n(value.begin() + static_cast<std::ptrdiff_t>(source),
475             this->config_.head_dim,
476             this->values_.begin() + static_cast<std::ptrdiff_t>(target));
477     };
478     const std::size_t written_index =
479         checked_size(layer_id, "layer_id") * checked_size(this->config_.
480         max_context, "max_context") +
481         checked_size(model_position, "model_position");
481     this->written_[written_index] = 1;
482     this->filled_tokens_ = std::max(this->filled_tokens_, model_position + 1);
483 }
484
485 std::span<const float> ReferenceKVCache::key(std::int32_t layer_id,
486                                             std::int32_t model_position,
487                                             std::int32_t kv_head) const {
488     this->validate_address(layer_id, model_position, kv_head);
489     const std::size_t offset = this->base_offset(layer_id, model_position,
490     kv_head);
491     return std::span<const float>(this->keys_.data() + offset,
492     checked_size(this->config_.head_dim, "
493     head_dim"));
494 }
495
496 std::span<const float> ReferenceKVCache::value(std::int32_t layer_id,
497                                             std::int32_t model_position,
498                                             std::int32_t kv_head) const {
499     this->validate_address(layer_id, model_position, kv_head);
500     const std::size_t offset = this->base_offset(layer_id, model_position,
501     kv_head);
502     return std::span<const float>(this->values_.data() + offset,
503     checked_size(this->config_.head_dim, "
504     head_dim"));
505 }
506
507 std::int32_t ReferenceKVCache::layer_count() const noexcept {
508     return this->layer_count_;
509 }
510
511 std::int32_t ReferenceKVCache::filled_tokens() const noexcept {

```

```

508     return this->filled_tokens_;
509 }
510
511 std::size_t ReferenceKVCache::base_offset(std::int32_t layer_id,
512                                           std::int32_t model_position,
513                                           std::int32_t kv_head) const {
514     return (((checked_size(layer_id, "layer_id") *
515                    checked_size(this->config.max_context, "max_context")) +
516            checked_size(model_position, "model_position")) *
517            checked_size(this->config.kv_heads, "kv_heads") +
518            checked_size(kv_head, "kv_head")) *
519            checked_size(this->config.head_dim, "head_dim");
520 }
521
522 void ReferenceKVCache::validate_address(std::int32_t layer_id,
523                                         std::int32_t model_position,
524                                         std::int32_t kv_head) const {
525     if (layer_id < 0 || layer_id >= this->layer_count_) {
526         throw std::runtime_error("KV layer_id out of range");
527     }
528     if (model_position < 0 || model_position >= this->config.max_context) {
529         throw std::runtime_error("KV model_position out of range");
530     }
531     if (kv_head < 0 || kv_head >= this->config.kv_heads) {
532         throw std::runtime_error("KV head out of range");
533     }
534 }
535
536 void rms_norm(std::span<const float> input,
537              std::span<const float> weight,
538              float epsilon,
539              std::span<float> output) {
540     if (input.size() != weight.size() || input.size() != output.size()) {
541         throw std::runtime_error("RMSNorm size mismatch");
542     }
543     if (input.empty()) {
544         throw std::runtime_error("RMSNorm input is empty");
545     }
546     float sum_square = 0.0F;
547     for (const float value : input) {
548         sum_square += value * value;
549     }
550     const float mean_square = sum_square / static_cast<float>(input.size());
551     const float scale = 1.0F / std::sqrt(mean_square + epsilon);
552     for (std::size_t index = 0; index < input.size(); ++index) {
553         output[index] = input[index] * scale * weight[index];
554     }
555 }
556
557 void linear_f32(const LinearWeightsF32& layer,
558                std::span<const float> input,
559                std::span<float> output) {

```

```

560     validate_linear(layer, "linear_f32");
561     validate_span_size(input.size(), layer.input_size, "linear input");
562     validate_span_size(output.size(), layer.output_size, "linear output");
563     for (std::int32_t out = 0; out < layer.output_size; ++out) {
564         const std::size_t row_base =
565             checked_size(out, "out") * checked_size(layer.input_size, "↵
↵ input_size");
566         float accumulator = layer.bias.empty() ? 0.0F : layer.bias[checked_size↵
↵ (out, "out")];
567         for (std::int32_t in = 0; in < layer.input_size; ++in) {
568             accumulator += layer.weights[row_base + checked_size(in, "in")] *
569                 input[checked_size(in, "in")];
570         }
571         output[checked_size(out, "out")] = accumulator;
572     }
573 }
574
575 void apply_rope(std::span<float> heads,
576               std::int32_t head_count,
577               std::int32_t head_dim,
578               std::int32_t model_position,
579               float rope_base) {
580     require_positive(head_count, "head_count");
581     require_positive(head_dim, "head_dim");
582     validate_span_size(heads.size(), head_count * head_dim, "RoPE heads");
583     if (head_dim % LCQI_ROPE_PAIR_WIDTH != 0) {
584         throw std::runtime_error("RoPE head_dim must be even");
585     }
586     if (model_position < 0) {
587         throw std::runtime_error("RoPE model_position cannot be negative");
588     }
589     if (rope_base <= 1.0F) {
590         throw std::runtime_error("RoPE base must be greater than 1");
591     }
592     for (std::int32_t head = 0; head < head_count; ++head) {
593         for (std::int32_t offset = 0; offset < head_dim; offset += ↵
↵ LCQI_ROPE_PAIR_WIDTH) {
594             const float exponent = static_cast<float>(offset) / static_cast<↵
↵ float>(head_dim);
595             const float theta = 1.0F / std::pow(rope_base, exponent);
596             const float angle = static_cast<float>(model_position) * theta;
597             const float cosine = std::cos(angle);
598             const float sine = std::sin(angle);
599             const std::size_t first =
600                 checked_size(head, "head") * checked_size(head_dim, "head_dim")↵
↵ +
601                 checked_size(offset, "offset");
602             const float x0 = heads[first];
603             const float x1 = heads[first + 1];
604             heads[first] = x0 * cosine - x1 * sine;
605             heads[first + 1] = x0 * sine + x1 * cosine;
606         }

```



```

607     }
608 }
609
610 void attention_decode(const DecoderConfig& config,
611                     const ReferenceKVCache& cache,
612                     std::int32_t layer_id,
613                     std::int32_t model_position,
614                     std::span<const float> query,
615                     std::span<float> output) {
616     validate_config(config);
617     validate_span_size(query.size(), config.hidden_size, "attention query");
618     validate_span_size(output.size(), config.hidden_size, "attention output");
619     if (model_position < 0 || model_position >= cache.filled_tokens()) {
620         throw std::runtime_error("attention model_position has not been written
621     ↪ ");
622     }
623     std::fill(output.begin(), output.end(), 0.0F);
624
625     const std::int32_t group_size = config.query_heads / config.kv_heads;
626     const float score_scale = 1.0F / std::sqrt(static_cast<float>(config.kv_
627     ↪ head_dim));
628     std::vector<float> scores(check_size(model_position + 1, "score count"),
629                             LCQI_SOFTMAX_NEGATIVE_INFINITY);
630     std::vector<float> probabilities(scores.size(), 0.0F);
631
632     for (std::int32_t query_head = 0; query_head < config.query_heads; ++
633     ↪ query_head) {
634         const std::int32_t kv_head = query_head / group_size;
635         const std::size_t query_base =
636         ↪ checked_size(query_head, "query_head") * checked_size(config.kv_
637         ↪ head_dim, "head_dim");
638         float max_score = LCQI_SOFTMAX_NEGATIVE_INFINITY;
639         for (std::int32_t position = 0; position <= model_position; ++position)
640         ↪ {
641             const std::span<const float> key = cache.key(layer_id, position,
642             ↪ kv_head);
643             float dot = 0.0F;
644             for (std::int32_t dim = 0; dim < config.head_dim; ++dim) {
645                 dot += query[query_base + checked_size(dim, "dim")] *
646                 ↪ key[checked_size(dim, "dim")];
647             }
648             const float score = dot * score_scale;
649             scores[checked_size(position, "position")] = score;
650             max_score = std::max(max_score, score);
651         }
652
653         float probability_sum = 0.0F;
654         for (std::int32_t position = 0; position <= model_position; ++position)
655         ↪ {
656             const std::size_t index = checked_size(position, "position");
657             const float unnormalized = std::exp(scores[index] - max_score);
658             probabilities[index] = unnormalized;

```

```

652         probability_sum += unnormalized;
653     }
654     if (probability_sum <= 0.0F) {
655         throw std::runtime_error("attention probability sum is invalid");
656     }
657     for (std::int32_t position = 0; position <= model_position; ++position) {
658         const float probability =
659             probabilities[checked_size(position, "position")] /
660             probability_sum;
661         const std::span<const float> value = cache.value(layer_id, position,
662             kv_head);
663         for (std::int32_t dim = 0; dim < config.head_dim; ++dim) {
664             output[query_base + checked_size(dim, "dim")] +=
665                 probability * value[checked_size(dim, "dim")];
666         }
667     }
668 }
669 ReferenceDecodeResult run_reference_decode(const ReferenceDecoderModel& model,
670     std::span<const std::int32_t> token_ids) {
671     validate_config(model.config);
672     if (token_ids.empty()) {
673         throw std::runtime_error("reference decode needs at least one token");
674     }
675     if (token_ids.size() > checked_size(model.config.max_context, "max_context")) {
676         throw std::runtime_error("token_ids exceed max_context");
677     }
678     if (model.token_embedding.size() !=
679         checked_size(model.config.vocab_size, "vocab_size") *
680         checked_size(model.config.hidden_size, "hidden_size")) {
681         throw std::runtime_error("token embedding size mismatch");
682     }
683     if (model.final_norm_weight.size() != checked_size(model.config.hidden_size,
684         "hidden_size")) {
685         throw std::runtime_error("final norm weight size mismatch");
686     }
687     validate_linear(model.lm_head, "lm_head");
688     ReferenceKVCache cache(model.config, static_cast<std::int32_t>(model.layers.
689         size()));
690     std::vector<float> hidden(checked_size(model.config.hidden_size, "
691         hidden_size"), 0.0F);
692     for (std::int32_t position = 0; position < static_cast<std::int32_t>(
693         token_ids.size()));
694         ++position) {
695         const std::int32_t token_id = token_ids[checked_size(position, "
696         position")];
697         if (token_id < 0 || token_id >= model.config.vocab_size) {

```

```

694         throw std::runtime_error("token id out of vocabulary");
695     }
696     const std::span<const float> embedding =
697         row_span(model.token_embedding, token_id, model.config.hidden_size)↵
↵ ;
698     std::copy(embedding.begin(), embedding.end(), hidden.begin());
699     for (std::int32_t layer_id = 0; layer_id < static_cast<std::int32_t>(↵
↵ model.layers.size());
700         ++layer_id) {
701         forward_layer(model.config,
702                       model.layers[checked_size(layer_id, "layer_id")],
703                       layer_id,
704                       position,
705                       cache,
706                       hidden);
707     }
708 }
709
710 std::vector<float> normed(checked_size(model.config.hidden_size, "↵
↵ hidden_size"), 0.0F);
711 std::vector<float> logits(checked_size(model.config.vocab_size, "vocab_size↵
↵ "), 0.0F);
712 rms_norm(hidden, model.final_norm_weight, model.config.rms_epsilon, normed)↵
↵ ;
713 linear_f32(model.lm_head, normed, logits);
714
715 ReferenceDecodeResult result;
716 result.predicted_token = argmax(logits);
717 result.logits = std::move(logits);
718 return result;
719 }
720
721 ReferenceDecoderModel make_tiny_reference_decoder_model() {
722     ReferenceDecoderModel model;
723     model.config.hidden_size = 4;
724     model.config.query_heads = 2;
725     model.config.kv_heads = 1;
726     model.config.head_dim = 2;
727     model.config.intermediate_size = 6;
728     model.config.vocab_size = 5;
729     model.config.max_context = 8;
730     model.config.rms_epsilon = 1.0e-5F;
731     model.config.rope_base = 10000.0F;
732
733     model.token_embedding = {
734         0.20F, -0.10F, 0.05F, 0.30F,
735         -0.40F, 0.10F, 0.25F, -0.20F,
736         0.15F, 0.35F, -0.30F, 0.05F,
737         0.50F, -0.25F, 0.10F, -0.15F,
738         -0.05F, 0.20F, 0.40F, -0.35F,
739     };
740

```

```

741 DecoderLayerWeightsF32 layer;
742 layer.rms_attention_weight = {1.0F, 0.9F, 1.1F, 1.0F};
743 layer.wq = make_linear(4,
744                        4,
745                        {
746                            0.20F, -0.10F, 0.05F, 0.30F,
747                            -0.15F, 0.25F, 0.10F, -0.05F,
748                            0.05F, 0.10F, -0.20F, 0.15F,
749                            0.30F, -0.05F, 0.20F, 0.10F,
750                        });
751 layer.wk = make_linear(4,
752                        2,
753                        {
754                            0.10F, 0.20F, -0.10F, 0.05F,
755                            -0.05F, 0.15F, 0.25F, -0.20F,
756                        });
757 layer.wv = make_linear(4,
758                        2,
759                        {
760                            0.25F, -0.05F, 0.10F, 0.15F,
761                            -0.10F, 0.30F, 0.05F, 0.20F,
762                        });
763 layer.wo = make_linear(4,
764                        4,
765                        {
766                            0.20F, 0.10F, -0.05F, 0.15F,
767                            -0.10F, 0.25F, 0.20F, -0.05F,
768                            0.05F, -0.20F, 0.30F, 0.10F,
769                            0.15F, 0.05F, -0.10F, 0.25F,
770                        });
771 layer.rms_mlp_weight = {0.95F, 1.05F, 1.0F, 0.9F};
772 layer.w_gate = make_linear(4,
773                            6,
774                            {
775                                0.20F, -0.10F, 0.05F, 0.10F,
776                                -0.05F, 0.15F, 0.20F, -0.10F,
777                                0.10F, 0.05F, -0.15F, 0.25F,
778                                -0.20F, 0.10F, 0.15F, 0.05F,
779                                0.05F, -0.25F, 0.10F, 0.20F,
780                                0.15F, 0.05F, 0.05F, -0.15F,
781                            });
782 layer.w_up = make_linear(4,
783                            6,
784                            {
785                                0.10F, 0.20F, -0.05F, 0.05F,
786                                -0.15F, 0.05F, 0.25F, 0.10F,
787                                0.20F, -0.10F, 0.05F, 0.15F,
788                                0.05F, 0.10F, -0.20F, 0.25F,
789                                -0.05F, 0.15F, 0.10F, -0.10F,
790                                0.25F, -0.05F, 0.05F, 0.10F,
791                            });
792 layer.w_down = make_linear(6,

```

```

793         4,
794         {
795             0.10F, -0.05F, 0.15F, 0.20F, -0.10F, 0.05F,
796             -0.20F, 0.10F, 0.05F, -0.05F, 0.15F, 0.20F,
797             0.05F, 0.15F, -0.10F, 0.10F, 0.20F, -0.05F,
798             0.20F, 0.05F, 0.10F, -0.15F, -0.05F, 0.10F,
799         });
800     model.layers.push_back(std::move(layer));
801     model.final_norm_weight = {1.0F, 0.95F, 1.05F, 1.0F};
802     model.lm_head = make_linear(4,
803                                 5,
804                                 {
805                 0.20F, -0.10F, 0.05F, 0.15F,
806                 -0.05F, 0.25F, 0.10F, -0.20F,
807                 0.15F, 0.05F, -0.25F, 0.10F,
808                 -0.10F, 0.20F, 0.15F, 0.05F,
809                 0.05F, -0.15F, 0.20F, 0.25F,
810             },
811             zeros(5));
812     return model;
813 }
814
815 } // namespace lcqi

```

4.3 Tokenizer 与 Safetensors 实现

`tokenizer.cpp` 把 prompt 变成 token id, 并固定 unknown token 行为。读者应检查 BOS/EOS 是否稳定插入, UNK 是否按合同处理。

Listing 4.2 src/tokenizer.cpp

```

1  #include <lcqi/tokenizer.hpp>
2
3  #include <cstdint>
4  #include <fstream>
5  #include <stdexcept>
6  #include <string>
7  #include <string_view>
8
9  namespace lcqi {
10 namespace {
11
12 constexpr std::uint64_t LCQI_FNV_OFFSET_BASIS = 1469598103934665603ULL;
13 constexpr std::uint64_t LCQI_FNV_PRIME = 1099511628211ULL;
14 constexpr std::int32_t LCQI_INVALID_TOKEN_ID = -1;
15
16 std::string read_key(std::istream& input) {
17     std::string key;
18     if (!(input >> key)) {
19         throw std::runtime_error("unexpected end of tokenizer file");
20     }
21     return key;

```

```

22 }
23
24 void expect_key(std::istream& input, const std::string& expected) {
25     const std::string key = read_key(input);
26     if (key != expected) {
27         throw std::runtime_error("expected tokenizer key '" + expected + "', ←
↪ got '" + key + "'");
28     }
29 }
30
31 std::int32_t read_i32(std::istream& input, const std::string& key) {
32     expect_key(input, key);
33     std::int32_t value = 0;
34     if (!(input >> value)) {
35         throw std::runtime_error("invalid tokenizer int32 value for key '" + ←
↪ key + "'");
36     }
37     return value;
38 }
39
40 std::string read_string(std::istream& input, const std::string& key) {
41     expect_key(input, key);
42     std::string value;
43     if (!(input >> value)) {
44         throw std::runtime_error("invalid tokenizer string value for key '" + ←
↪ key + "'");
45     }
46     return value;
47 }
48
49 std::int32_t lookup_token_id(const TokenizerModel& tokenizer, std::string_view ←
↪ text) {
50     for (const TokenEntry& entry : tokenizer.vocab) {
51         if (entry.text == text) {
52             return entry.id;
53         }
54     }
55     return LCQI_INVALID_TOKEN_ID;
56 }
57
58 void validate_tokenizer(const TokenizerModel& tokenizer) {
59     if (tokenizer.vocab.empty()) {
60         throw std::runtime_error("tokenizer vocab is empty");
61     }
62     for (std::size_t i = 0; i < tokenizer.vocab.size(); ++i) {
63         const TokenEntry& left = tokenizer.vocab[i];
64         if (left.id < 0) {
65             throw std::runtime_error("tokenizer token id must be non-negative")↪
↪ ;
66         }
67         for (std::size_t j = i + 1; j < tokenizer.vocab.size(); ++j) {
68             const TokenEntry& right = tokenizer.vocab[j];

```

```

69         if (left.id == right.id) {
70             throw std::runtime_error("duplicate tokenizer token id");
71         }
72         if (left.text == right.text) {
73             throw std::runtime_error("duplicate tokenizer token text");
74         }
75     }
76 }
77 if (tokenizer.bos_id != LCQI_INVALID_TOKEN_ID &&
78     lookup_token_id(tokenizer, "<bos>") != tokenizer.bos_id) {
79     throw std::runtime_error("tokenizer BOS id does not match vocab");
80 }
81 if (tokenizer.eos_id != LCQI_INVALID_TOKEN_ID &&
82     lookup_token_id(tokenizer, "<eos>") != tokenizer.eos_id) {
83     throw std::runtime_error("tokenizer EOS id does not match vocab");
84 }
85 }
86
87 std::uint64_t hash_byte(std::uint64_t hash, unsigned char value) {
88     hash ^= static_cast<std::uint64_t>(value);
89     hash *= LCQI_FNV_PRIME;
90     return hash;
91 }
92
93 std::uint64_t hash_string(std::uint64_t hash, std::string_view text) {
94     for (const unsigned char value : text) {
95         hash = hash_byte(hash, value);
96     }
97     return hash;
98 }
99
100 std::uint64_t hash_i32(std::uint64_t hash, std::int32_t value) {
101     const std::uint32_t bits = static_cast<std::uint32_t>(value);
102     for (std::int32_t shift = 0; shift < 32; shift += 8) {
103         hash = hash_byte(hash, static_cast<unsigned char>((bits >> shift) & 0x
104         ↪ xFFU));
105     }
106     return hash;
107 }
108 } // namespace
109
110 TokenizerModel load_tokenizer(const std::filesystem::path& path) {
111     std::ifstream input(path);
112     if (!input) {
113         throw std::runtime_error("cannot open tokenizer file: " + path.string()
114         ↪ );
115     }
116
117     expect_key(input, "LCQI_TOKENIZER_V1");
118
119     TokenizerModel tokenizer;

```

```

119     tokenizer.bos_id = read_i32(input, "bos_id");
120     tokenizer.eos_id = read_i32(input, "eos_id");
121     tokenizer.unk_id = read_i32(input, "unk_id");
122     tokenizer.chat_template_hash = read_string(input, "chat_template_hash");
123     const std::int32_t vocab_size = read_i32(input, "vocab_size");
124     if (vocab_size <= 0) {
125         throw std::runtime_error("tokenizer vocab_size must be positive");
126     }
127
128     tokenizer.vocab.reserve(static_cast<std::size_t>(vocab_size));
129     expect_key(input, "vocab");
130     for (std::int32_t i = 0; i < vocab_size; ++i) {
131         TokenEntry entry;
132         if (!(input >> entry.id >> entry.text)) {
133             throw std::runtime_error("invalid tokenizer vocab entry");
134         }
135         tokenizer.vocab.push_back(entry);
136     }
137
138     validate_tokenizer(tokenizer);
139     return tokenizer;
140 }
141
142 std::vector<std::int32_t> encode_prompt(const TokenizerModel& tokenizer,
143                                         std::string_view prompt) {
144     std::vector<std::int32_t> ids;
145     if (tokenizer.bos_id != LCQI_INVALID_TOKEN_ID) {
146         ids.push_back(tokenizer.bos_id);
147     }
148
149     std::size_t begin = 0;
150     while (begin < prompt.size()) {
151         while (begin < prompt.size() && prompt[begin] == ' ') {
152             ++begin;
153         }
154         if (begin >= prompt.size()) {
155             break;
156         }
157         std::size_t end = begin;
158         while (end < prompt.size() && prompt[end] != ' ') {
159             ++end;
160         }
161         const std::string_view token = prompt.substr(begin, end - begin);
162         const std::int32_t token_id = lookup_token_id(tokenizer, token);
163         if (token_id == LCQI_INVALID_TOKEN_ID) {
164             if (tokenizer.unk_id == LCQI_INVALID_TOKEN_ID) {
165                 throw std::runtime_error("tokenizer encountered unknown token");
166             }
167             ids.push_back(tokenizer.unk_id);
168         } else {
169             ids.push_back(token_id);

```



```

170     }
171     begin = end;
172 }
173
174 if (tokenizer.eos_id != LCQI_INVALID_TOKEN_ID) {
175     ids.push_back(tokenizer.eos_id);
176 }
177 return ids;
178 }
179
180 std::uint64_t tokenizer_contract_hash(const TokenizerModel& tokenizer) {
181     std::uint64_t hash = LCQI_FNV_OFFSET_BASIS;
182     hash = hash_string(hash, tokenizer.chat_template_hash);
183     hash = hash_i32(hash, tokenizer.bos_id);
184     hash = hash_i32(hash, tokenizer.eos_id);
185     hash = hash_i32(hash, tokenizer.unk_id);
186     for (const TokenEntry& entry : tokenizer.vocab) {
187         hash = hash_string(hash, entry.text);
188         hash = hash_i32(hash, entry.id);
189     }
190     return hash;
191 }
192
193 } // namespace lcqi

```

safetensors.cpp 解析 header、metadata、dtype、shape 和 data offsets。坏 dtype、坏 shape、坏 offset 和 payload 太短都应该被拒绝。

Listing 4.3 src/safetensors.cpp

```

1 #include <lcqi/safetensors.hpp>
2
3 #include <algorithm>
4 #include <cctype>
5 #include <cstdint>
6 #include <cmath>
7 #include <cstring>
8 #include <fstream>
9 #include <limits>
10 #include <stdexcept>
11 #include <string>
12 #include <string_view>
13 #include <vector>
14
15 namespace lcqi {
16 namespace {
17
18 constexpr std::size_t LCQI_SAFETENSORS_PREFIX_BYTES = 8;
19 constexpr std::uint64_t LCQI_SAFETENSORS_MAX_HEADER_BYTES = 16U * 1024U * 1024U↵
    ↵ ;
20 constexpr std::size_t LCQI_SAFETENSORS_OFFSET_COUNT = 2;
21 constexpr std::uint64_t LCQI_BYTE_BITS = 8;
22 constexpr std::uint64_t LCQI_DECIMAL_BASE = 10;

```

```

23 constexpr std::uint64_t LCQI_BYTE_MASK = 0xFFU;
24 constexpr unsigned char LCQI_JSON_CONTROL_CHAR_LIMIT = 0x20U;
25 constexpr std::uint64_t LCQI_DTYPE_BYTES_F64 = 8;
26 constexpr std::uint64_t LCQI_DTYPE_BYTES_F32 = 4;
27 constexpr std::uint64_t LCQI_DTYPE_BYTES_F16 = 2;
28 constexpr std::uint64_t LCQI_DTYPE_BYTES_I64 = 8;
29 constexpr std::uint64_t LCQI_DTYPE_BYTES_I32 = 4;
30 constexpr std::uint64_t LCQI_DTYPE_BYTES_I16 = 2;
31 constexpr std::uint64_t LCQI_DTYPE_BYTES_I8 = 1;
32 constexpr std::uint64_t LCQI_DTYPE_BYTES_U8 = 1;
33 constexpr std::uint32_t LCQI_F32_EXPONENT_MASK = 0x7F800000U;
34 constexpr std::uint32_t LCQI_F16_SIGN_MASK = 0x8000U;
35 constexpr std::uint32_t LCQI_F16_EXPONENT_MASK = 0x7C00U;
36 constexpr std::uint32_t LCQI_F16_MANTISSA_MASK = 0x03FFU;
37 constexpr std::uint32_t LCQI_F16_EXPONENT_BIAS = 15U;
38 constexpr std::uint32_t LCQI_F32_EXPONENT_BIAS = 127U;
39 constexpr std::uint32_t LCQI_F32_MANTISSA_BITS = 23U;
40 constexpr std::uint32_t LCQI_F16_MANTISSA_BITS = 10U;
41 constexpr std::uint32_t LCQI_F16_TO_F32_SIGN_SHIFT = 16U;
42
43 class HeaderCursor {
44 public:
45     explicit HeaderCursor(std::string_view text) : text_(text) {}
46
47     void expect(char expected) {
48         this->skip_ws();
49         if (this->eof() || this->text_[this->position_] != expected) {
50             throw std::runtime_error("invalid safetensors header syntax");
51         }
52         ++this->position_;
53     }
54
55     [[nodiscard]] bool consume(char expected) {
56         this->skip_ws();
57         if (!this->eof() && this->text_[this->position_] == expected) {
58             ++this->position_;
59             return true;
60         }
61         return false;
62     }
63
64     [[nodiscard]] std::string parse_string() {
65         this->skip_ws();
66         this->expect('\"');
67         std::string result;
68         while (!this->eof()) {
69             const char ch = this->text_[this->position_++];
70             if (ch == '\"') {
71                 return result;
72             }
73             if (static_cast<unsigned char>(ch) < LCQI_JSON_CONTROL_CHAR_LIMIT) ←
↪ {

```

```

74         throw std::runtime_error("control character in safetensors ↵
↵ string");
75     }
76     if (ch == '\\') {
77         result.push_back(this->parse_escape());
78     } else {
79         result.push_back(ch);
80     }
81 }
82 throw std::runtime_error("unterminated safetensors string");
83 }
84
85 [[nodiscard]] std::uint64_t parse_u64() {
86     this->skip_ws();
87     if (this->eof() ||
88         !std::isdigit(static_cast<unsigned char>(this->text_[this->↵
↵ position_]))) {
89         throw std::runtime_error("expected unsigned integer in safetensors ↵
↵ header");
90     }
91     std::uint64_t value = 0;
92     while (!this->eof() &&
93         std::isdigit(static_cast<unsigned char>(this->text_[this->↵
↵ position_]))) {
94         const std::uint64_t digit =
95             static_cast<std::uint64_t>(this->text_[this->position_] - '0');
96         if (value >
97             (std::numeric_limits<std::uint64_t>::max() - digit) / ↵
↵ LCQI_DECIMAL_BASE) {
98             throw std::runtime_error("safetensors integer overflow");
99         }
100         value = value * LCQI_DECIMAL_BASE + digit;
101         ++this->position_;
102     }
103     return value;
104 }
105
106 [[nodiscard]] std::vector<std::int64_t> parse_i64_array() {
107     std::vector<std::int64_t> values;
108     this->expect('[');
109     if (this->consume(' ')) {
110         return values;
111     }
112     while (true) {
113         const std::uint64_t raw = this->parse_u64();
114         if (raw > static_cast<std::uint64_t>(std::numeric_limits<std::↵
↵ int64_t>::max())) {
115             throw std::runtime_error("safetensors shape value is too large"↵
↵ );
116         }
117         values.push_back(static_cast<std::int64_t>(raw));
118         if (this->consume(' ')) {

```

```

119         return values;
120     }
121     this->expect(',');
122 }
123 }
124
125 [[nodiscard]] std::vector<std::uint64_t> parse_u64_array() {
126     std::vector<std::uint64_t> values;
127     this->expect('[');
128     if (this->consume(' ')) {
129         return values;
130     }
131     while (true) {
132         values.push_back(this->parse_u64());
133         if (this->consume(' ')) {
134             return values;
135         }
136         this->expect(',');
137     }
138 }
139
140 [[nodiscard]] bool eof_after_ws() {
141     this->skip_ws();
142     return this->eof();
143 }
144
145 private:
146     std::string_view text_;
147     std::size_t position_ = 0;
148
149     [[nodiscard]] bool eof() const {
150         return this->position_ >= this->text_.size();
151     }
152
153     void skip_ws() {
154         while (!this->eof() &&
155             std::isspace(static_cast<unsigned char>(this->text_[this->position_])) {
156             ++this->position_;
157         }
158     }
159
160     [[nodiscard]] char parse_escape() {
161         if (this->eof()) {
162             throw std::runtime_error("unterminated safetensors escape");
163         }
164         const char escaped = this->text_[this->position_++];
165         switch (escaped) {
166             case '"':
167             case '\\':
168             case '/':
169                 return escaped;

```

```

170         case 'b':
171             return '\\b';
172         case 'f':
173             return '\\f';
174         case 'n':
175             return '\\n';
176         case 'r':
177             return '\\r';
178         case 't':
179             return '\\t';
180         default:
181             throw std::runtime_error("unsupported safetensors string escape↵
↵ ");
182     }
183 }
184 };
185
186 std::uint64_t read_le_u64(std::span<const std::uint8_t> bytes) {
187     if (bytes.size() < LCQI_SAFETENSORS_PREFIX_BYTES) {
188         throw std::runtime_error("safetensors file is too small");
189     }
190     std::uint64_t value = 0;
191     for (std::size_t i = 0; i < LCQI_SAFETENSORS_PREFIX_BYTES; ++i) {
192         value |= static_cast<std::uint64_t>(bytes[i]) << (LCQI_BYTE_BITS * i);
193     }
194     return value;
195 }
196
197 std::string read_file_bytes(const std::filesystem::path& path) {
198     std::ifstream input(path, std::ios::binary);
199     if (!input) {
200         throw std::runtime_error("cannot open safetensors file: " + path.string↵
↵ ());
201     }
202     input.seekg(0, std::ios::end);
203     const std::streamoff size = input.tellg();
204     if (size < 0) {
205         throw std::runtime_error("cannot determine safetensors file size");
206     }
207     input.seekg(0, std::ios::beg);
208     std::string bytes(static_cast<std::size_t>(size), '\\0');
209     if (!bytes.empty() && !input.read(bytes.data(), static_cast<std::streamsize↵
↵ >(bytes.size())) {
210         throw std::runtime_error("cannot read safetensors file");
211     }
212     return bytes;
213 }
214
215 std::vector<std::uint8_t> read_file_u8(const std::filesystem::path& path) {
216     std::ifstream input(path, std::ios::binary);
217     if (!input) {
218         throw std::runtime_error("cannot open safetensors file: " + path.string↵

```

```

    ↪ ();
219 }
220 input.seekg(0, std::ios::end);
221 const std::streamoff size = input.tellg();
222 if (size < 0) {
223     throw std::runtime_error("cannot determine safetensors file size");
224 }
225 input.seekg(0, std::ios::beg);
226 std::vector<std::uint8_t> bytes(static_cast<std::size_t>(size), 0);
227 if (!bytes.empty() &&
228     !input.read(reinterpret_cast<char*>(bytes.data()),
229                 static_cast<std::streamsize>(bytes.size()))) {
230     throw std::runtime_error("cannot read safetensors file");
231 }
232 return bytes;
233 }
234
235 void parse_metadata_value(HeaderCursor& cursor,
236                           SafeTensorManifest& manifest,
237                           const std::string& key) {
238     if (key == "__metadata__") {
239         cursor.expect('{');
240         if (cursor.consume('}')) {
241             return;
242         }
243         while (true) {
244             const std::string metadata_key = cursor.parse_string();
245             cursor.expect(':');
246             const std::string metadata_value = cursor.parse_string();
247             manifest.metadata.emplace_back(metadata_key, metadata_value);
248             if (cursor.consume('}')) {
249                 return;
250             }
251             cursor.expect(',');
252         }
253     }
254     throw std::runtime_error("unexpected safetensors metadata value");
255 }
256
257 SafeTensorEntry parse_tensor_entry(HeaderCursor& cursor, const std::string& ↪
    ↪ name) {
258     SafeTensorEntry entry;
259     entry.name = name;
260     bool saw_dtype = false;
261     bool saw_shape = false;
262     bool saw_offsets = false;
263
264     cursor.expect('{');
265     if (cursor.consume('}')) {
266         throw std::runtime_error("empty safetensors tensor entry");
267     }
268     while (true) {

```

```

269     const std::string field = cursor.parse_string();
270     cursor.expect(':');
271     if (field == "dtype") {
272         entry.dtype = cursor.parse_string();
273         saw_dtype = true;
274     } else if (field == "shape") {
275         entry.shape = cursor.parse_i64_array();
276         saw_shape = true;
277     } else if (field == "data_offsets") {
278         const std::vector<std::uint64_t> offsets = cursor.parse_u64_array();
279         ↵ ;
280         if (offsets.size() != LCQI_SAFETENSORS_OFFSET_COUNT) {
281             throw std::runtime_error("safetensors data_offsets must contain ↵
282             ↵ two values");
283         }
284         entry.data_begin = offsets[0];
285         entry.data_end = offsets[1];
286         saw_offsets = true;
287     } else {
288         throw std::runtime_error("unsupported safetensors tensor field: " + ↵
289         ↵ field);
290     }
291     if (cursor.consume('}')) {
292         break;
293     }
294     cursor.expect(',');
295 }
296
297 if (!saw_dtype || !saw_shape || !saw_offsets) {
298     throw std::runtime_error("safetensors tensor entry is missing required ↵
299     ↵ fields");
300 }
301 if (entry.data_end < entry.data_begin) {
302     throw std::runtime_error("safetensors tensor offset range is invalid");
303 }
304 return entry;
305 }
306
307 void parse_header(std::string_view header, SafeTensorManifest& manifest) {
308     HeaderCursor cursor(header);
309     cursor.expect('{');
310     if (cursor.consume('}')) {
311         throw std::runtime_error("safetensors header is empty");
312     }
313
314     while (true) {
315         const std::string key = cursor.parse_string();
316         cursor.expect(':');
317         if (key == "__metadata__") {
318             parse_metadata_value(cursor, manifest, key);
319         } else {
320             manifest.tensors.push_back(parse_tensor_entry(cursor, key));

```

```

317     }
318     if (cursor.consume('}')) {
319         break;
320     }
321     cursor.expect(',');
322 }
323
324 if (!cursor.eof_after_ws()) {
325     throw std::runtime_error("trailing data after safetensors header");
326 }
327 }
328
329 std::uint64_t expected_tensor_bytes(const SafeTensorEntry& entry);
330
331 void validate_manifest(const SafeTensorManifest& manifest,
332                       std::uint64_t file_size) {
333     const std::uint64_t payload_size = file_size - manifest.data_start_offset;
334     for (std::size_t i = 0; i < manifest.tensors.size(); ++i) {
335         const SafeTensorEntry& left = manifest.tensors[i];
336         if (left.name.empty() || left.dtype.empty()) {
337             throw std::runtime_error("safetensors tensor has empty name or ↵
↵ dtype");
338         }
339         if (left.data_end > payload_size) {
340             throw std::runtime_error("safetensors tensor data_offsets exceed ↵
↵ payload size");
341         }
342         const std::uint64_t expected_bytes = expected_tensor_bytes(left);
343         if (left.byte_size() != expected_bytes) {
344             throw std::runtime_error("safetensors tensor byte size does not ↵
↵ match dtype and shape");
345         }
346         for (const std::int64_t dim : left.shape) {
347             if (dim < 0) {
348                 throw std::runtime_error("safetensors shape dimension is ↵
↵ negative");
349             }
350         }
351         for (std::size_t j = i + 1; j < manifest.tensors.size(); ++j) {
352             const SafeTensorEntry& right = manifest.tensors[j];
353             if (left.name == right.name) {
354                 throw std::runtime_error("duplicate safetensors tensor name");
355             }
356             const bool overlaps =
357                 left.data_begin < right.data_end && right.data_begin < left.↵
↵ data_end;
358             if (overlaps) {
359                 throw std::runtime_error("overlapping safetensors tensor data ↵
↵ range");
360             }
361         }
362     }

```



```

363 }
364
365 std::uint64_t dtype_byte_size(std::string_view dtype) {
366     if (dtype == "F64") {
367         return LCQI_DTYPE_BYTES_F64;
368     }
369     if (dtype == "F32") {
370         return LCQI_DTYPE_BYTES_F32;
371     }
372     if (dtype == "F16" || dtype == "BF16") {
373         return LCQI_DTYPE_BYTES_F16;
374     }
375     if (dtype == "I64" || dtype == "U64") {
376         return LCQI_DTYPE_BYTES_I64;
377     }
378     if (dtype == "I32" || dtype == "U32") {
379         return LCQI_DTYPE_BYTES_I32;
380     }
381     if (dtype == "I16" || dtype == "U16") {
382         return LCQI_DTYPE_BYTES_I16;
383     }
384     if (dtype == "I8") {
385         return LCQI_DTYPE_BYTES_I8;
386     }
387     if (dtype == "U8" || dtype == "B00L") {
388         return LCQI_DTYPE_BYTES_U8;
389     }
390     throw std::runtime_error("unsupported safetensors dtype: " + std::string(
391         dtype));
392 }
393
394 std::uint64_t expected_tensor_bytes(const SafeTensorEntry& entry) {
395     std::uint64_t element_count = 1;
396     for (const std::int64_t dim : entry.shape) {
397         if (dim < 0) {
398             throw std::runtime_error("safetensors shape dimension is negative");
399         }
400         const std::uint64_t extent = static_cast<std::uint64_t>(dim);
401         if (extent != 0 &&
402             element_count > std::numeric_limits<std::uint64_t>::max() / extent) {
403             throw std::runtime_error("safetensors shape element count overflow");
404         }
405         element_count *= extent;
406     }
407     const std::uint64_t dtype_bytes = dtype_byte_size(entry.dtype);
408     if (dtype_bytes != 0 &&
409         element_count > std::numeric_limits<std::uint64_t>::max() / dtype_bytes) {
410         throw std::runtime_error("safetensors tensor byte size overflow");
411     }
412 }

```

```

410     }
411     return element_count * dtype_bytes;
412 }
413
414 SafeTensorManifest parse_manifest_from_bytes(std::span<const std::uint8_t> ↵
    ↵ bytes) {
415     const std::uint64_t header_size = read_le_u64(bytes);
416     if (header_size == 0 || header_size > LCQI_SAFETENSORS_MAX_HEADER_BYTES) {
417         throw std::runtime_error("safetensors header size is invalid");
418     }
419     const std::uint64_t data_start_offset =
420         static_cast<std::uint64_t>(LCQI_SAFETENSORS_PREFIX_BYTES) + header_size↵
    ↵ ;
421     if (data_start_offset > bytes.size()) {
422         throw std::runtime_error("safetensors header extends beyond file size")↵
    ↵ ;
423     }
424
425     SafeTensorManifest manifest;
426     manifest.header_size = header_size;
427     manifest.data_start_offset = data_start_offset;
428     const char* header_begin =
429         reinterpret_cast<const char*>(bytes.data() + ↵
    ↵ LCQI_SAFETENSORS_PREFIX_BYTES);
430     const std::string_view header(header_begin, static_cast<std::size_t>(↵
    ↵ header_size));
431     parse_header(header, manifest);
432     validate_manifest(manifest, static_cast<std::uint64_t>(bytes.size()));
433     return manifest;
434 }
435
436 std::uint16_t read_le_u16(std::span<const std::uint8_t> bytes, std::size_t ↵
    ↵ offset) {
437     return static_cast<std::uint16_t>(
438         static_cast<std::uint16_t>(bytes[offset]) |
439         (static_cast<std::uint16_t>(bytes[offset + 1]) << LCQI_BYTE_BITS));
440 }
441
442 std::uint32_t read_le_u32(std::span<const std::uint8_t> bytes, std::size_t ↵
    ↵ offset) {
443     std::uint32_t value = 0;
444     for (std::size_t i = 0; i < LCQI_DTYPE_BYTES_F32; ++i) {
445         value |= static_cast<std::uint32_t>(bytes[offset + i]) << (↵
    ↵ LCQI_BYTE_BITS * i);
446     }
447     return value;
448 }
449
450 float bits_to_float(std::uint32_t bits) {
451     float value = 0.0F;
452     std::memcpy(&value, &bits, sizeof(value));
453     return value;

```

```

454 }
455
456 float f16_to_f32(std::uint16_t bits) {
457     const std::uint32_t sign = (static_cast<std::uint32_t>(bits) & ↵
↵ LCQI_F16_SIGN_MASK)
458         << LCQI_F16_TO_F32_SIGN_SHIFT;
459     const std::uint32_t exponent = static_cast<std::uint32_t>(bits) & ↵
↵ LCQI_F16_EXPONENT_MASK;
460     const std::uint32_t mantissa = static_cast<std::uint32_t>(bits) & ↵
↵ LCQI_F16_MANTISSA_MASK;
461
462     if (exponent == 0U) {
463         if (mantissa == 0U) {
464             return bits_to_float(sign);
465         }
466         float value = static_cast<float>(mantissa) /
467             static_cast<float>(1U << LCQI_F16_MANTISSA_BITS);
468         value = std::ldexp(value, 1 - static_cast<int>(LCQI_F16_EXPONENT_BIAS));↵
↵ ;
469         return (sign == 0U) ? value : -value;
470     }
471
472     if (exponent == LCQI_F16_EXPONENT_MASK) {
473         const std::uint32_t f32_mantissa =
474             mantissa << (LCQI_F32_MANTISSA_BITS - LCQI_F16_MANTISSA_BITS);
475         return bits_to_float(sign | LCQI_F32_EXPONENT_MASK | f32_mantissa);
476     }
477
478     const std::uint32_t f32_exponent =
479         ((exponent >> LCQI_F16_MANTISSA_BITS) - LCQI_F16_EXPONENT_BIAS +
480         LCQI_F32_EXPONENT_BIAS)
481         << LCQI_F32_MANTISSA_BITS;
482     const std::uint32_t f32_mantissa =
483         mantissa << (LCQI_F32_MANTISSA_BITS - LCQI_F16_MANTISSA_BITS);
484     return bits_to_float(sign | f32_exponent | f32_mantissa);
485 }
486
487 float bf16_to_f32(std::uint16_t bits) {
488     return bits_to_float(static_cast<std::uint32_t>(bits) << 16U);
489 }
490
491 std::span<const std::uint8_t> tensor_payload_span(const SafeTensorFile& file,
492     const SafeTensorEntry& entry)↵
493     {
494     const std::uint64_t begin = file.manifest.data_start_offset + entry.↵
↵ data_begin;
495     const std::uint64_t end = file.manifest.data_start_offset + entry.data_end;
496     if (begin > end || end > file.bytes.size()) {
497         throw std::runtime_error("safetensors tensor byte range is invalid");
498     }
499     return std::span<const std::uint8_t>(
        file.bytes.data() + static_cast<std::size_t>(begin),

```

```

500     static_cast<std::size_t>(end - begin));
501 }
502
503 } // namespace
504
505 std::uint64_t SafeTensorEntry::byte_size() const {
506     return this->data_end - this->data_begin;
507 }
508
509 const SafeTensorEntry* SafeTensorManifest::find_tensor(std::string_view name) ←
    ↪ const {
510     const auto found = std::find_if(
511         this->tensors.begin(),
512         this->tensors.end(),
513         [name](const SafeTensorEntry& entry) {
514             return entry.name == name;
515         });
516     if (found == this->tensors.end()) {
517         return nullptr;
518     }
519     return &(*found);
520 }
521
522 SafeTensorManifest load_safetensors_manifest(const std::filesystem::path& path) ←
    ↪ {
523     const std::string bytes = read_file_bytes(path);
524     return parse_manifest_from_bytes(
525         std::span<const std::uint8_t>(
526             reinterpret_cast<const std::uint8_t*>(bytes.data()),
527             bytes.size()));
528 }
529
530 SafeTensorFile load_safetensors_file(const std::filesystem::path& path) {
531     SafeTensorFile file;
532     file.path = path;
533     file.bytes = read_file_u8(path);
534     file.manifest = parse_manifest_from_bytes(file.bytes);
535     return file;
536 }
537
538 std::span<const std::uint8_t> read_safetensor_tensor_bytes(
539     const SafeTensorFile& file,
540     std::string_view name) {
541     const SafeTensorEntry* entry = file.manifest.find_tensor(name);
542     if (entry == nullptr) {
543         throw std::runtime_error("missing safetensors tensor: " + std::string(←
    ↪ name));
544     }
545     return tensor_payload_span(file, *entry);
546 }
547
548 std::vector<float> read_safetensor_f32_tensor(const SafeTensorFile& file,

```

```

549         std::string_view name) {
550     const SafeTensorEntry* entry = file.manifest.find_tensor(name);
551     if (entry == nullptr) {
552         throw std::runtime_error("missing safetensors tensor: " + std::string(↵
↵ name));
553     }
554     const std::span<const std::uint8_t> bytes = tensor_payload_span(file, *↵
↵ entry);
555     std::vector<float> values;
556     if (entry->dtype == "F32") {
557         values.resize(bytes.size() / LCQI_DTYPE_BYTES_F32);
558         for (std::size_t index = 0; index < values.size(); ++index) {
559             values[index] = bits_to_float(
560                 read_le_u32(bytes, index * LCQI_DTYPE_BYTES_F32));
561         }
562         return values;
563     }
564     if (entry->dtype == "F16") {
565         values.resize(bytes.size() / LCQI_DTYPE_BYTES_F16);
566         for (std::size_t index = 0; index < values.size(); ++index) {
567             values[index] = f16_to_f32(
568                 read_le_u16(bytes, index * LCQI_DTYPE_BYTES_F16));
569         }
570         return values;
571     }
572     if (entry->dtype == "BF16") {
573         values.resize(bytes.size() / LCQI_DTYPE_BYTES_F16);
574         for (std::size_t index = 0; index < values.size(); ++index) {
575             values[index] = bf16_to_f32(
576                 read_le_u16(bytes, index * LCQI_DTYPE_BYTES_F16));
577         }
578         return values;
579     }
580     throw std::runtime_error("safetensors tensor is not float-compatible: " + ↵
↵ entry->name);
581 }
582
583 } // namespace lcqi

```

4.4 GPT-2 Reference Path

`gpt2_reference.cpp` 包括 JSON 配置读取、byte-level BPE、safetensors F32/F16/BF16 张量读取、HuggingFace 权重名前缀解析、LayerNorm、packed QKV、causal attention、GELU、tied lm head、full-prefix greedy generation 和 cached-KV generation。它是正确性优先的 reference path，暂不追求吞吐。

Listing 4.4 `src/gpt2_reference.cpp`

```

1 #include <lcqi/gpt2_reference.hpp>
2
3 #include <lcqi/safetensors.hpp>
4

```

```

5  #include <algorithm>
6  #include <array>
7  #include <cmath>
8  #include <cstdint>
9  #include <fstream>
10 #include <limits>
11 #include <optional>
12 #include <sstream>
13 #include <stdexcept>
14 #include <string>
15 #include <string_view>
16 #include <unordered_set>
17 #include <utility>
18
19 namespace lcqi {
20 namespace {
21
22 constexpr std::int32_t LCQI_GPT2_DEFAULT_BOS_EOS = 50256;
23 constexpr std::int32_t LCQI_GPT2_ATTENTION_PROJECTIONS = 3;
24 constexpr std::int32_t LCQI_GPT2_PAIR_TOKEN_COUNT = 2;
25 constexpr std::int32_t LCQI_GPT2_BYTE_COUNT = 256;
26 constexpr std::int32_t LCQI_GPT2_UNICODE_PRIVATE_BASE = 256;
27 constexpr std::int32_t LCQI_GPT2_ASCII_EXCLAMATION = 33;
28 constexpr std::int32_t LCQI_GPT2_ASCII_TILDE = 126;
29 constexpr std::int32_t LCQI_GPT2_LATIN_INVERTED_EXCLAMATION = 161;
30 constexpr std::int32_t LCQI_GPT2_LATIN_NOT_SIGN = 172;
31 constexpr std::int32_t LCQI_GPT2_LATIN_REGISTERED = 174;
32 constexpr std::int32_t LCQI_GPT2_LATIN_Y_DIAERESIS = 255;
33 constexpr std::int32_t LCQI_GPT2_UTF8_CONTINUATION_MASK = 0x3F;
34 constexpr std::int32_t LCQI_GPT2_UTF8_CONTINUATION_BITS = 0x80;
35 constexpr std::int32_t LCQI_GPT2_UTF8_TWO_BYTE_HEAD = 0xC0;
36 constexpr std::int32_t LCQI_GPT2_UTF8_THREE_BYTE_HEAD = 0xE0;
37 constexpr std::int32_t LCQI_GPT2_UTF8_FOUR_BYTE_HEAD = 0xF0;
38 constexpr std::int32_t LCQI_GPT2_UTF8_ONE_BYTE_LIMIT = 0x80;
39 constexpr std::int32_t LCQI_GPT2_UTF8_TWO_BYTE_LIMIT = 0x800;
40 constexpr std::int32_t LCQI_GPT2_UTF8_THREE_BYTE_LIMIT = 0x10000;
41 constexpr unsigned char LCQI_GPT2_ASCII_APOSTROPHE = 0x27U;
42 constexpr float LCQI_GPT2_GELU_SQRT_2_OVER_PI = 0.7978845608028654F;
43 constexpr float LCQI_GPT2_GELU_CUBIC_COEFF = 0.044715F;
44 constexpr float LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY =
45     -std::numeric_limits<float>::infinity();
46
47 class JsonCursor {
48 public:
49     explicit JsonCursor(std::string_view text) : text_(text) {}
50
51     void expect(char expected) {
52         this->skip_ws();
53         if (this->eof() || this->text_[this->position_] != expected) {
54             throw std::runtime_error("invalid JSON syntax");
55         }
56         ++this->position_;

```

```

57     }
58
59     [[nodiscard]] bool consume(char expected) {
60         this->skip_ws();
61         if (!this->eof() && this->text_[this->position_] == expected) {
62             ++this->position_;
63             return true;
64         }
65         return false;
66     }
67
68     [[nodiscard]] std::string parse_string() {
69         this->skip_ws();
70         this->expect('"');
71         std::string result;
72         while (!this->eof()) {
73             const char ch = this->text_[this->position_++];
74             if (ch == '"') {
75                 return result;
76             }
77             if (ch == '\\') {
78                 result += this->parse_escape();
79             } else {
80                 result.push_back(ch);
81             }
82         }
83         throw std::runtime_error("unterminated JSON string");
84     }
85
86     [[nodiscard]] std::int64_t parse_i64() {
87         this->skip_ws();
88         bool negative = false;
89         if (this->consume('-')) {
90             negative = true;
91         }
92         if (this->eof() || this->text_[this->position_] < '0' ||
93             this->text_[this->position_] > '9') {
94             throw std::runtime_error("expected JSON integer");
95         }
96         std::int64_t value = 0;
97         while (!this->eof() && this->text_[this->position_] >= '0' &&
98             this->text_[this->position_] <= '9') {
99             const std::int64_t digit = this->text_[this->position_] - '0';
100             if (value >
101                 (std::numeric_limits<std::int64_t>::max() - digit) / 10) {
102                 throw std::runtime_error("JSON integer overflow");
103             }
104             value = value * 10 + digit;
105             ++this->position_;
106         }
107         return negative ? -value : value;
108     }

```

```

109
110 [[nodiscard]] double parse_number() {
111     this->skip_ws();
112     const std::size_t begin = this->position_;
113     if (!this->eof() && this->text_[this->position_] == '-') {
114         ++this->position_;
115     }
116     while (!this->eof() && this->text_[this->position_] >= '0' &&
117           this->text_[this->position_] <= '9') {
118         ++this->position_;
119     }
120     if (!this->eof() && this->text_[this->position_] == '.') {
121         ++this->position_;
122         while (!this->eof() && this->text_[this->position_] >= '0' &&
123               this->text_[this->position_] <= '9') {
124             ++this->position_;
125         }
126     }
127     if (!this->eof() &&
128         (this->text_[this->position_] == 'e' || this->text_[this->position_ ←
↵ ] == 'E')) {
129         ++this->position_;
130         if (!this->eof() &&
131             (this->text_[this->position_] == '+' || this->text_[this-> ←
↵ position_] == '-')) {
132             ++this->position_;
133         }
134         while (!this->eof() && this->text_[this->position_] >= '0' &&
135               this->text_[this->position_] <= '9') {
136             ++this->position_;
137         }
138     }
139     if (begin == this->position_) {
140         throw std::runtime_error("expected JSON number");
141     }
142     return std::stod(std::string(this->text_.substr(begin, this->position_ ←
↵ - begin)));
143 }
144
145 void skip_value() {
146     this->skip_ws();
147     if (this->eof()) {
148         throw std::runtime_error("unexpected end of JSON");
149     }
150     const char ch = this->text_[this->position_];
151     if (ch == '"') {
152         static_cast<void>(this->parse_string());
153         return;
154     }
155     if (ch == '{') {
156         this->expect('{');
157         if (this->consume('}')) {

```



```

158         return;
159     }
160     while (true) {
161         static_cast<void>(this->parse_string());
162         this->expect(':');
163         this->skip_value();
164         if (this->consume('}')) {
165             return;
166         }
167         this->expect(',');
168     }
169 }
170 if (ch == '[') {
171     this->expect('[');
172     if (this->consume(']')) {
173         return;
174     }
175     while (true) {
176         this->skip_value();
177         if (this->consume(']')) {
178             return;
179         }
180         this->expect(',');
181     }
182 }
183 if (ch == 't') {
184     this->expect_literal("true");
185     return;
186 }
187 if (ch == 'f') {
188     this->expect_literal("false");
189     return;
190 }
191 if (ch == 'n') {
192     this->expect_literal("null");
193     return;
194 }
195 static_cast<void>(this->parse_number());
196 }
197
198 [[nodiscard]] bool eof_after_ws() {
199     this->skip_ws();
200     return this->eof();
201 }
202
203 private:
204     std::string_view text_;
205     std::size_t position_ = 0;
206
207     [[nodiscard]] bool eof() const {
208         return this->position_ >= this->text_.size();
209     }

```

```

210
211 void skip_ws() {
212     while (!this->eof() &&
213            (this->text_[this->position_] == ' ' ||
214             this->text_[this->position_] == '\n' ||
215             this->text_[this->position_] == '\r' ||
216             this->text_[this->position_] == '\t')) {
217         ++this->position_;
218     }
219 }
220
221 void expect_literal(std::string_view literal) {
222     for (const char expected : literal) {
223         if (this->eof() || this->text_[this->position_] != expected) {
224             throw std::runtime_error("invalid JSON literal");
225         }
226         ++this->position_;
227     }
228 }
229
230 [[nodiscard]] std::string parse_escape() {
231     if (this->eof()) {
232         throw std::runtime_error("unterminated JSON escape");
233     }
234     const char escaped = this->text_[this->position_++];
235     switch (escaped) {
236         case '"':
237         case '\\':
238         case '/':
239             return std::string(1, escaped);
240         case 'b':
241             return "\b";
242         case 'f':
243             return "\f";
244         case 'n':
245             return "\n";
246         case 'r':
247             return "\r";
248         case 't':
249             return "\t";
250         case 'u':
251             return this->parse_unicode_escape();
252         default:
253             throw std::runtime_error("unsupported JSON escape");
254     }
255 }
256
257 [[nodiscard]] std::string parse_unicode_escape() {
258     std::uint32_t value = 0;
259     for (std::int32_t i = 0; i < 4; ++i) {
260         if (this->eof()) {
261             throw std::runtime_error("unterminated JSON unicode escape");

```

```

262     }
263     const char ch = this->text_[this->position_++];
264     value <= 4U;
265     if (ch >= '0' && ch <= '9') {
266         value += static_cast<std::uint32_t>(ch - '0');
267     } else if (ch >= 'a' && ch <= 'f') {
268         value += static_cast<std::uint32_t>(10 + ch - 'a');
269     } else if (ch >= 'A' && ch <= 'F') {
270         value += static_cast<std::uint32_t>(10 + ch - 'A');
271     } else {
272         throw std::runtime_error("invalid JSON unicode escape");
273     }
274 }
275 std::string encoded;
276 if (value < LCQI_GPT2_UTF8_ONE_BYTE_LIMIT) {
277     encoded.push_back(static_cast<char>(value));
278 } else if (value < LCQI_GPT2_UTF8_TWO_BYTE_LIMIT) {
279     encoded.push_back(static_cast<char>(
280         LCQI_GPT2_UTF8_TWO_BYTE_HEAD | (value >> 6U)));
281     encoded.push_back(static_cast<char>(
282         LCQI_GPT2_UTF8_CONTINUATION_BITS |
283         (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
284 } else {
285     encoded.push_back(static_cast<char>(
286         LCQI_GPT2_UTF8_THREE_BYTE_HEAD | (value >> 12U)));
287     encoded.push_back(static_cast<char>(
288         LCQI_GPT2_UTF8_CONTINUATION_BITS |
289         ((value >> 6U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
290     encoded.push_back(static_cast<char>(
291         LCQI_GPT2_UTF8_CONTINUATION_BITS |
292         (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
293 }
294 return encoded;
295 }
296 };
297
298 std::string read_text_file(const std::filesystem::path& path) {
299     std::ifstream input(path);
300     if (!input) {
301         throw std::runtime_error("cannot open text file: " + path.string());
302     }
303     std::ostringstream buffer;
304     buffer << input.rdbuf();
305     return buffer.str();
306 }
307
308 std::size_t checked_size(std::int64_t value, const char* name) {
309     if (value < 0) {
310         throw std::runtime_error(std::string(name) + " cannot be negative");
311     }
312     return static_cast<std::size_t>(value);
313 }

```

```

314
315 std::string encode_codepoint(std::uint32_t value) {
316     std::string encoded;
317     if (value < LCQI_GPT2_UTF8_ONE_BYTE_LIMIT) {
318         encoded.push_back(static_cast<char>(value));
319     } else if (value < LCQI_GPT2_UTF8_TWO_BYTE_LIMIT) {
320         encoded.push_back(static_cast<char>(
321             LCQI_GPT2_UTF8_TWO_BYTE_HEAD | (value >> 6U)));
322         encoded.push_back(static_cast<char>(
323             LCQI_GPT2_UTF8_CONTINUATION_BITS |
324             (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
325     } else if (value < LCQI_GPT2_UTF8_THREE_BYTE_LIMIT) {
326         encoded.push_back(static_cast<char>(
327             LCQI_GPT2_UTF8_THREE_BYTE_HEAD | (value >> 12U)));
328         encoded.push_back(static_cast<char>(
329             LCQI_GPT2_UTF8_CONTINUATION_BITS |
330             ((value >> 6U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
331         encoded.push_back(static_cast<char>(
332             LCQI_GPT2_UTF8_CONTINUATION_BITS |
333             (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
334     } else {
335         encoded.push_back(static_cast<char>(
336             LCQI_GPT2_UTF8_FOUR_BYTE_HEAD | (value >> 18U)));
337         encoded.push_back(static_cast<char>(
338             LCQI_GPT2_UTF8_CONTINUATION_BITS |
339             ((value >> 12U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
340         encoded.push_back(static_cast<char>(
341             LCQI_GPT2_UTF8_CONTINUATION_BITS |
342             ((value >> 6U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
343         encoded.push_back(static_cast<char>(
344             LCQI_GPT2_UTF8_CONTINUATION_BITS |
345             (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
346     }
347     return encoded;
348 }
349
350 void require_positive(std::int32_t value, const char* name) {
351     if (value <= 0) {
352         throw std::runtime_error(std::string(name) + " must be positive");
353     }
354 }
355
356 void validate_config(const Gpt2Config& config) {
357     require_positive(config.hidden_size, "hidden_size");
358     require_positive(config.head_count, "head_count");
359     require_positive(config.layer_count, "layer_count");
360     require_positive(config.max_positions, "max_positions");
361     require_positive(config.vocab_size, "vocab_size");
362     require_positive(config.intermediate_size, "intermediate_size");
363     if (config.hidden_size % config.head_count != 0) {
364         throw std::runtime_error("hidden_size must be divisible by head_count")↵
        ↵ ;

```

```

365     }
366     if (config.layer_norm_epsilon <= 0.0F) {
367         throw std::runtime_error("layer_norm_epsilon must be positive");
368     }
369 }
370
371 std::int32_t head_dim(const Gpt2Config& config) {
372     return config.hidden_size / config.head_count;
373 }
374
375 std::size_t matrix_size(std::int32_t rows, std::int32_t columns) {
376     return checked_size(rows, "rows") * checked_size(columns, "columns");
377 }
378
379 void validate_vector_size(std::size_t actual, std::int32_t expected, const char↵
↵ * name) {
380     if (actual != checked_size(expected, name)) {
381         throw std::runtime_error(std::string(name) + " size mismatch");
382     }
383 }
384
385 void validate_linear(const Gpt2LinearF32& linear, const char* name) {
386     require_positive(linear.input_size, "linear input_size");
387     require_positive(linear.output_size, "linear output_size");
388     if (linear.weights.size() != matrix_size(linear.output_size, linear.↵
↵ input_size)) {
389         throw std::runtime_error(std::string(name) + " weight size mismatch");
390     }
391     if (!linear.bias.empty() &&
392         linear.bias.size() != checked_size(linear.output_size, "linear ↵
↵ output_size")) {
393         throw std::runtime_error(std::string(name) + " bias size mismatch");
394     }
395 }
396
397 void validate_model(const Gpt2ReferenceModel& model) {
398     validate_config(model.config);
399     if (model.layers.size() != checked_size(model.config.layer_count, "↵
↵ layer_count")) {
400         throw std::runtime_error("GPT-2 layer count mismatch");
401     }
402     if (model.token_embedding.size() !=
403         matrix_size(model.config.vocab_size, model.config.hidden_size)) {
404         throw std::runtime_error("GPT-2 token embedding size mismatch");
405     }
406     if (model.position_embedding.size() !=
407         matrix_size(model.config.max_positions, model.config.hidden_size)) {
408         throw std::runtime_error("GPT-2 position embedding size mismatch");
409     }
410     validate_vector_size(model.final_ln_weight.size(), model.config.hidden_size↵
↵ , "ln_f weight");
411     validate_vector_size(model.final_ln_bias.size(), model.config.hidden_size, ↵

```

```

↪ "ln_f bias");
412 if (model.tie_lm_head_to_embedding) {
413     if (!model.lm_head_weight.empty()) {
414         throw std::runtime_error("tied GPT-2 lm_head should not carry ↪
↪ separate weights");
415     }
416 } else if (model.lm_head_weight.size() !=
417     matrix_size(model.config.vocab_size, model.config.hidden_size)) ↪
↪ {
418     throw std::runtime_error("GPT-2 lm_head size mismatch");
419 }
420 for (const Gpt2LayerWeightsF32& layer : model.layers) {
421     validate_vector_size(layer.ln_1_weight.size(), model.config.hidden_size ↪
↪ , "ln_1 weight");
422     validate_vector_size(layer.ln_1_bias.size(), model.config.hidden_size, ↪
↪ "ln_1 bias");
423     validate_linear(layer.c_attn, "c_attn");
424     validate_linear(layer.c_proj, "c_proj");
425     validate_vector_size(layer.ln_2_weight.size(), model.config.hidden_size ↪
↪ , "ln_2 weight");
426     validate_vector_size(layer.ln_2_bias.size(), model.config.hidden_size, ↪
↪ "ln_2 bias");
427     validate_linear(layer.c_fc, "c_fc");
428     validate_linear(layer.mlp_c_proj, "mlp_c_proj");
429 }
430 }
431
432 std::span<const float> row_span(std::span<const float> values,
433     std::int32_t row,
434     std::int32_t row_width) {
435     return values.subspan(check_size(row, "row") * checked_size(row_width, "↪
↪ row_width"),
436         checked_size(row_width, "row_width"));
437 }
438
439 void add_inplace(std::span<float> target, std::span<const float> source) {
440     if (target.size() != source.size()) {
441         throw std::runtime_error("GPT-2 residual add size mismatch");
442     }
443     for (std::size_t index = 0; index < target.size(); ++index) {
444         target[index] += source[index];
445     }
446 }
447
448 void layer_norm(std::span<const float> input,
449     std::span<const float> weight,
450     std::span<const float> bias,
451     float epsilon,
452     std::span<float> output) {
453     if (input.empty() || input.size() != weight.size() || input.size() != bias.↪
↪ size() ||
454         input.size() != output.size()) {

```

```

455     throw std::runtime_error("GPT-2 LayerNorm size mismatch");
456 }
457 float mean = 0.0F;
458 for (const float value : input) {
459     mean += value;
460 }
461 mean /= static_cast<float>(input.size());
462
463 float variance = 0.0F;
464 for (const float value : input) {
465     const float centered = value - mean;
466     variance += centered * centered;
467 }
468 variance /= static_cast<float>(input.size());
469 const float scale = 1.0F / std::sqrt(variance + epsilon);
470
471 for (std::size_t index = 0; index < input.size(); ++index) {
472     output[index] = (input[index] - mean) * scale * weight[index] + bias[↵
↵ index];
473 }
474 }
475
476 void linear_f32(const Gpt2LinearF32& linear,
477                std::span<const float> input,
478                std::span<float> output) {
479     validate_linear(linear, "GPT-2 linear");
480     if (input.size() != checked_size(linear.input_size, "linear input") ||
481         output.size() != checked_size(linear.output_size, "linear output")) {
482         throw std::runtime_error("GPT-2 linear input/output size mismatch");
483     }
484     for (std::int32_t out = 0; out < linear.output_size; ++out) {
485         float sum = linear.bias.empty() ? 0.0F : linear.bias[checked_size(out, ↵
↵ "out")];
486         const std::size_t row_base = checked_size(out, "out") *
487                                     checked_size(linear.input_size, "linear ↵
↵ input");
488         for (std::int32_t in = 0; in < linear.input_size; ++in) {
489             sum += linear.weights[row_base + checked_size(in, "in")] *
490                   input[checked_size(in, "in")];
491         }
492         output[checked_size(out, "out")] = sum;
493     }
494 }
495
496 float gelu_new(float value) {
497     const float cubic = value * value * value;
498     return 0.5F * value *
499           (1.0F + std::tanh(LCQI_GPT2_GELU_SQRT_2_OVER_PI *
500                             (value + LCQI_GPT2_GELU_CUBIC_COEFF * cubic)));
501 }
502
503 std::int32_t argmax(std::span<const float> values) {

```

```

504     if (values.empty()) {
505         throw std::runtime_error("cannot take argmax of empty logits");
506     }
507     std::int32_t best_index = 0;
508     float best_value = values.front();
509     for (std::int32_t index = 1; index < static_cast<std::int32_t>(values.size()
↵ )); ++index) {
510         const float value = values[checked_size(index, "argmax index")];
511         if (value > best_value) {
512             best_value = value;
513             best_index = index;
514         }
515     }
516     return best_index;
517 }
518
519 void project_qkv(const Gpt2Config& config,
520                 const Gpt2LayerWeightsF32& layer,
521                 std::span<const float> hidden,
522                 std::span<float> q,
523                 std::span<float> k,
524                 std::span<float> v) {
525     std::vector<float> packed(matrix_size(LCQI_GPT2_ATTENTION_PROJECTIONS,
526                                           config.hidden_size),
527                               0.0F);
528     linear_f32(layer.c_attn, hidden, packed);
529     const std::size_t width = checked_size(config.hidden_size, "hidden_size");
530     std::copy_n(packed.begin(), config.hidden_size, q.begin());
531     std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width),
532               config.hidden_size,
533               k.begin());
534     std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width * 2),
535               config.hidden_size,
536               v.begin());
537 }
538
539 void attention_cached_position(const Gpt2Config& config,
540                              const Gpt2KvCache& cache,
541                              std::int32_t layer_id,
542                              std::int32_t model_position,
543                              std::span<const float> query,
544                              std::span<float> output) {
545     const std::int32_t local_head_dim = head_dim(config);
546     const float score_scale = 1.0F / std::sqrt(static_cast<float>(
↵ local_head_dim));
547     std::fill(output.begin(), output.end(), 0.0F);
548     std::vector<float> scores(checked_size(model_position + 1, "model_position"
↵ ),
549                               LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY);
550
551     for (std::int32_t head = 0; head < config.head_count; ++head) {
552         float max_score = LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY;

```



```

553     for (std::int32_t past = 0; past <= model_position; ++past) {
554         const std::span<const float> key = cache.key(layer_id, past, head);
555         float dot = 0.0F;
556         for (std::int32_t dim = 0; dim < local_head_dim; ++dim) {
557             const std::size_t query_index =
558                 matrix_size(head, local_head_dim) + checked_size(dim, "head"
↵ dim");
559             dot += query[query_index] * key[checked_size(dim, "head dim")];
560         }
561         const float score = dot * score_scale;
562         scores[checked_size(past, "past")] = score;
563         max_score = std::max(max_score, score);
564     }
565
566     float denominator = 0.0F;
567     for (std::int32_t past = 0; past <= model_position; ++past) {
568         const std::size_t index = checked_size(past, "past");
569         scores[index] = std::exp(scores[index] - max_score);
570         denominator += scores[index];
571     }
572     if (denominator <= 0.0F) {
573         throw std::runtime_error("GPT-2 cached attention denominator is
↵ invalid");
574     }
575
576     for (std::int32_t past = 0; past <= model_position; ++past) {
577         const float probability = scores[checked_size(past, "past")] /
↵ denominator;
578         const std::span<const float> value = cache.value(layer_id, past,
↵ head);
579         for (std::int32_t dim = 0; dim < local_head_dim; ++dim) {
580             const std::size_t output_index =
581                 matrix_size(head, local_head_dim) + checked_size(dim, "head"
↵ dim");
582             output[output_index] += probability * value[checked_size(dim, "
↵ head dim")];
583         }
584     }
585 }
586 }
587
588 void attention_full_sequence(const Gpt2Config& config,
589                             std::span<const float> q_by_pos,
590                             std::span<const float> k_by_pos,
591                             std::span<const float> v_by_pos,
592                             std::int32_t sequence_length,
593                             std::span<float> output_by_pos) {
594     const std::int32_t local_head_dim = head_dim(config);
595     const float score_scale = 1.0F / std::sqrt(static_cast<float>(<
↵ local_head_dim));
596     std::fill(output_by_pos.begin(), output_by_pos.end(), 0.0F);
597     std::vector<float> scores(checked_size(sequence_length, "sequence_length"),

```

```

598         LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY);
599
600     for (std::int32_t position = 0; position < sequence_length; ++position) {
601         for (std::int32_t head = 0; head < config.head_count; ++head) {
602             float max_score = LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY;
603             for (std::int32_t past = 0; past <= position; ++past) {
604                 float dot = 0.0F;
605                 for (std::int32_t dim = 0; dim < local_head_dim; ++dim) {
606                     const std::size_t q_index =
607                         matrix_size(position, config.hidden_size) +
608                         matrix_size(head, local_head_dim) + checked_size(dim, "↵
↵ head dim");
609                     const std::size_t k_index =
610                         matrix_size(past, config.hidden_size) +
611                         matrix_size(head, local_head_dim) + checked_size(dim, "↵
↵ head dim");
612                     dot += q_by_pos[q_index] * k_by_pos[k_index];
613                 }
614                 const float score = dot * score_scale;
615                 scores[checked_size(past, "past")] = score;
616                 max_score = std::max(max_score, score);
617             }
618
619             float denominator = 0.0F;
620             for (std::int32_t past = 0; past <= position; ++past) {
621                 const std::size_t index = checked_size(past, "past");
622                 scores[index] = std::exp(scores[index] - max_score);
623                 denominator += scores[index];
624             }
625             if (denominator <= 0.0F) {
626                 throw std::runtime_error("GPT-2 attention denominator is ↵
↵ invalid");
627             }
628             for (std::int32_t past = 0; past <= position; ++past) {
629                 const float probability = scores[checked_size(past, "past")] / ↵
↵ denominator;
630                 for (std::int32_t dim = 0; dim < local_head_dim; ++dim) {
631                     const std::size_t output_index =
632                         matrix_size(position, config.hidden_size) +
633                         matrix_size(head, local_head_dim) + checked_size(dim, "↵
↵ head dim");
634                     const std::size_t value_index =
635                         matrix_size(past, config.hidden_size) +
636                         matrix_size(head, local_head_dim) + checked_size(dim, "↵
↵ head dim");
637                     output_by_pos[output_index] += probability * v_by_pos[↵
↵ value_index];
638                 }
639             }
640         }
641     }
642 }

```

```

643
644 void forward_gpt2_layer(const Gpt2Config& config,
645                         const Gpt2LayerWeightsF32& layer,
646                         std::int32_t sequence_length,
647                         std::vector<float>& hidden_by_pos) {
648     std::vector<float> normed(checkered_size(config.hidden_size, "hidden_size"), ←
        ↪ 0.0F);
649     std::vector<float> q_by_pos(matrix_size(sequence_length, config.hidden_size, ←
        ↪ ), 0.0F);
650     std::vector<float> k_by_pos(q_by_pos.size(), 0.0F);
651     std::vector<float> v_by_pos(q_by_pos.size(), 0.0F);
652     std::vector<float> attention_by_pos(q_by_pos.size(), 0.0F);
653     std::vector<float> projected(checkered_size(config.hidden_size, "hidden_size" ←
        ↪ ), 0.0F);
654     std::vector<float> mlp_fc(checkered_size(config.intermediate_size, " ←
        ↪ intermediate_size"), 0.0F);
655     std::vector<float> mlp_out(checkered_size(config.hidden_size, "hidden_size"), ←
        ↪ 0.0F);
656
657     for (std::int32_t position = 0; position < sequence_length; ++position) {
658         const std::span<float> hidden = std::span<float>(
659             hidden_by_pos.data() + matrix_size(position, config.hidden_size),
660             checked_size(config.hidden_size, "hidden_size"));
661         layer_norm(hidden,
662                 layer.ln_1_weight,
663                 layer.ln_1_bias,
664                 config.layer_norm_epsilon,
665                 normed);
666         std::span<float> q = std::span<float>(
667             q_by_pos.data() + matrix_size(position, config.hidden_size),
668             checked_size(config.hidden_size, "hidden_size"));
669         std::span<float> k = std::span<float>(
670             k_by_pos.data() + matrix_size(position, config.hidden_size),
671             checked_size(config.hidden_size, "hidden_size"));
672         std::span<float> v = std::span<float>(
673             v_by_pos.data() + matrix_size(position, config.hidden_size),
674             checked_size(config.hidden_size, "hidden_size"));
675         project_qkv(config, layer, normed, q, k, v);
676     }
677
678     attention_full_sequence(config, q_by_pos, k_by_pos, v_by_pos, ←
        ↪ sequence_length, attention_by_pos);
679
680     for (std::int32_t position = 0; position < sequence_length; ++position) {
681         const std::span<float> hidden = std::span<float>(
682             hidden_by_pos.data() + matrix_size(position, config.hidden_size),
683             checked_size(config.hidden_size, "hidden_size"));
684         const std::span<const float> attention = std::span<const float>(
685             attention_by_pos.data() + matrix_size(position, config.hidden_size) ←
        ↪ ,
686             checked_size(config.hidden_size, "hidden_size"));
687         linear_f32(layer.c_proj, attention, projected);

```

```

688     add_inplace(hidden, projected);
689
690     layer_norm(hidden,
691                 layer.ln_2_weight,
692                 layer.ln_2_bias,
693                 config.layer_norm_epsilon,
694                 normed);
695     linear_f32(layer.c_fc, normed, mlp_fc);
696     for (float& value : mlp_fc) {
697         value = gelu_new(value);
698     }
699     linear_f32(layer.mlp_c_proj, mlp_fc, mlp_out);
700     add_inplace(hidden, mlp_out);
701 }
702 }
703
704 void forward_gpt2_layer_cached(const Gpt2Config& config,
705                                const Gpt2LayerWeightsF32& layer,
706                                std::int32_t layer_id,
707                                std::int32_t model_position,
708                                Gpt2KvCache& cache,
709                                std::vector<float>& hidden) {
710     std::vector<float> normed(check_size(config.hidden_size, "hidden_size"), ←
711                               ↪ 0.0F);
712     std::vector<float> query(check_size(config.hidden_size, "hidden_size"), ←
713                              ↪ 0.0F);
714     std::vector<float> key(check_size(config.hidden_size, "hidden_size"), 0.0F);
715     std::vector<float> value(check_size(config.hidden_size, "hidden_size"), ←
716                              ↪ 0.0F);
717     std::vector<float> attention(check_size(config.hidden_size, "hidden_size"), ←
718                                  ↪ 0.0F);
719     std::vector<float> projected(check_size(config.hidden_size, "hidden_size"), ←
720                                  ↪ 0.0F);
721     std::vector<float> mlp_fc(check_size(config.intermediate_size, "intermediate_size"), ←
722                               ↪ 0.0F);
723     std::vector<float> mlp_out(check_size(config.hidden_size, "hidden_size"), ←
724                                ↪ 0.0F);
725
726     layer_norm(hidden,
727                 layer.ln_1_weight,
728                 layer.ln_1_bias,
729                 config.layer_norm_epsilon,
730                 normed);
731     project_qkv(config, layer, normed, query, key, value);
732     cache.append(layer_id, model_position, key, value);
733     attention_cached_position(config, cache, layer_id, model_position, query, ←
734                               ↪ attention);
735
736     linear_f32(layer.c_proj, attention, projected);
737     add_inplace(hidden, projected);
738 }

```

```

731     layer_norm(hidden,
732                 layer.ln_2_weight,
733                 layer.ln_2_bias,
734                 config.layer_norm_epsilon,
735                 normed);
736     linear_f32(layer.c_fc, normed, mlp_fc);
737     for (float& value_ref : mlp_fc) {
738         value_ref = gelu_new(value_ref);
739     }
740     linear_f32(layer.mlp_c_proj, mlp_fc, mlp_out);
741     add_inplace(hidden, mlp_out);
742 }
743
744 void compute_logits(const Gpt2ReferenceModel& model,
745                    std::span<const float> hidden,
746                    std::span<float> logits) {
747     if (logits.size() != checked_size(model.config.vocab_size, "vocab_size")) {
748         throw std::runtime_error("GPT-2 logits size mismatch");
749     }
750     const std::span<const float> weight =
751         model.tie_lm_head_to_embedding ? std::span<const float>(model.↵
↵ token_embedding)
752                                         : std::span<const float>(model.↵
↵ lm_head_weight);
753     for (std::int32_t token = 0; token < model.config.vocab_size; ++token) {
754         float sum = 0.0F;
755         const std::span<const float> row = row_span(weight, token, model.config.↵
↵ .hidden_size);
756         for (std::int32_t dim = 0; dim < model.config.hidden_size; ++dim) {
757             sum += row[checked_size(dim, "dim")] * hidden[checked_size(dim, "↵
↵ dim")];
758         }
759         logits[checked_size(token, "token")] = sum;
760     }
761 }
762
763 std::vector<std::string> gpt2_bytes_to_unicode() {
764     std::vector<std::int32_t> bytes;
765     for (std::int32_t value = LCQI_GPT2_ASCII_EXCLAMATION;
766          value <= LCQI_GPT2_ASCII_TILDE;
767          ++value) {
768         bytes.push_back(value);
769     }
770     for (std::int32_t value = LCQI_GPT2_LATIN_INVERTED_EXCLAMATION;
771          value <= LCQI_GPT2_LATIN_NOT_SIGN;
772          ++value) {
773         bytes.push_back(value);
774     }
775     for (std::int32_t value = LCQI_GPT2_LATIN_REGISTERED;
776          value <= LCQI_GPT2_LATIN_Y_DIAERESIS;
777          ++value) {
778         bytes.push_back(value);

```

```

779     }
780
781     std::unordered_set<std::int32_t> byte_set(bytes.begin(), bytes.end());
782     std::vector<std::int32_t> chars = bytes;
783     std::int32_t private_offset = 0;
784     for (std::int32_t byte = 0; byte < LCQI_GPT2_BYTE_COUNT; ++byte) {
785         if (!byte_set.contains(byte)) {
786             bytes.push_back(byte);
787             chars.push_back(LCQI_GPT2_UNICODE_PRIVATE_BASE + private_offset);
788             ++private_offset;
789         }
790     }
791
792     std::vector<std::string> mapping(LCQI_GPT2_BYTE_COUNT);
793     for (std::size_t index = 0; index < bytes.size(); ++index) {
794         mapping[checked_size(bytes[index], "byte unicode index")] =
795             encode_codepoint(static_cast<std::uint32_t>(chars[index]));
796     }
797     return mapping;
798 }
799
800 std::unordered_map<std::string, unsigned char> gpt2_unicode_to_bytes(
801     const std::vector<std::string>& byte_encoder) {
802     std::unordered_map<std::string, unsigned char> result;
803     for (std::size_t byte = 0; byte < byte_encoder.size(); ++byte) {
804         result.emplace(byte_encoder[byte], static_cast<unsigned char>(byte));
805     }
806     return result;
807 }
808
809 std::string bytes_to_bpe_alphabet(std::string_view text) {
810     static const std::vector<std::string> BYTE_ENCODER = gpt2_bytes_to_unicode↵
↵ ();
811     std::string result;
812     for (const unsigned char byte : text) {
813         result += BYTE_ENCODER[byte];
814     }
815     return result;
816 }
817
818 std::vector<std::string> utf8_symbols(std::string_view text) {
819     std::vector<std::string> symbols;
820     std::size_t offset = 0;
821     while (offset < text.size()) {
822         const unsigned char lead = static_cast<unsigned char>(text[offset]);
823         std::size_t width = 1;
824         if ((lead & 0xE0U) == LCQI_GPT2_UTF8_TWO_BYTE_HEAD) {
825             width = 2;
826         } else if ((lead & 0xF0U) == LCQI_GPT2_UTF8_THREE_BYTE_HEAD) {
827             width = 3;
828         } else if ((lead & 0xF8U) == LCQI_GPT2_UTF8_FOUR_BYTE_HEAD) {
829             width = 4;

```

```

830     }
831     if (offset + width > text.size()) {
832         throw std::runtime_error("invalid UTF-8/BPE symbol boundary");
833     }
834     symbols.emplace_back(text.substr(offset, width));
835     offset += width;
836 }
837 return symbols;
838 }
839
840 std::string pair_key(std::string_view left, std::string_view right) {
841     std::string key(left);
842     key.push_back('\n');
843     key += right;
844     return key;
845 }
846
847 std::vector<std::string> bpe_apply(const Gpt2Tokenizer& tokenizer, std::string_view token) {
848     std::vector<std::string> word = utf8_symbols(token);
849     if (word.size() <= 1) {
850         return word;
851     }
852
853     while (true) {
854         std::optional<std::int32_t> best_rank;
855         std::string best_left;
856         std::string best_right;
857         for (std::size_t index = 0; index + 1 < word.size(); ++index) {
858             const auto rank = tokenizer.bpe_ranks.find(pair_key(word[index],
859                 word[index + 1]));
860             if (rank != tokenizer.bpe_ranks.end() &&
861                 (!best_rank.has_value() || rank->second < *best_rank)) {
862                 best_rank = rank->second;
863                 best_left = word[index];
864                 best_right = word[index + 1];
865             }
866         }
867         if (!best_rank.has_value()) {
868             return word;
869         }
870
871         std::vector<std::string> merged;
872         std::size_t index = 0;
873         while (index < word.size()) {
874             if (index + 1 < word.size() && word[index] == best_left &&
875                 word[index + 1] == best_right) {
876                 merged.push_back(best_left + best_right);
877                 index += LCQI_GPT2_PAIR_TOKEN_COUNT;
878             } else {
879                 merged.push_back(word[index]);
880                 ++index;
881             }
882         }
883     }
884 }

```

```

880     }
881 }
882 word = std::move(merged);
883 if (word.size() == 1) {
884     return word;
885 }
886 }
887 }
888
889 bool is_ascii_letter(unsigned char value) {
890     return (value >= 'A' && value <= 'Z') || (value >= 'a' && value <= 'z');
891 }
892
893 bool is_ascii_digit(unsigned char value) {
894     return value >= '0' && value <= '9';
895 }
896
897 bool is_ascii_space(unsigned char value) {
898     return value == ' ' || value == '\t' || value == '\n' ||
899         value == '\r' || value == '\f' || value == '\v';
900 }
901
902 bool is_gpt2_punctuation(unsigned char value) {
903     return !is_ascii_letter(value) && !is_ascii_digit(value) &&
904         !is_ascii_space(value);
905 }
906
907 std::size_t consume_utf8_codepoint(std::string_view text, std::size_t offset) {
908     const unsigned char lead = static_cast<unsigned char>(text[offset]);
909     std::size_t width = 1;
910     if ((lead & 0xE0U) == LCQI_GPT2_UTF8_TWO_BYTE_HEAD) {
911         width = 2;
912     } else if ((lead & 0xF0U) == LCQI_GPT2_UTF8_THREE_BYTE_HEAD) {
913         width = 3;
914     } else if ((lead & 0xF8U) == LCQI_GPT2_UTF8_FOUR_BYTE_HEAD) {
915         width = 4;
916     }
917     if (offset + width > text.size()) {
918         throw std::runtime_error("invalid UTF-8 text for GPT-2 tokenizer");
919     }
920     return offset + width;
921 }
922
923 bool starts_with_contraction(std::string_view text,
924                             std::size_t offset,
925                             std::string_view contraction) {
926     if (offset + contraction.size() > text.size()) {
927         return false;
928     }
929     for (std::size_t index = 0; index < contraction.size(); ++index) {
930         char left = text[offset + index];
931         const char right = contraction[index];

```



```

932     if (left >= 'A' && left <= 'Z') {
933         left = static_cast<char>(left - 'A' + 'a');
934     }
935     if (left != right) {
936         return false;
937     }
938 }
939 return true;
940 }
941
942 std::vector<std::string> gpt2_pretokenize(std::string_view text) {
943     std::vector<std::string> pieces;
944     std::size_t offset = 0;
945     const std::array<std::string_view, 7> contractions{
946         "s", "t", "re", "ve", "m", "ll", "d",
947     };
948     while (offset < text.size()) {
949         bool emitted_contraction = false;
950         for (const std::string_view contraction : contractions) {
951             if (starts_with_contraction(text, offset, contraction)) {
952                 pieces.emplace_back(text.substr(offset, contraction.size()));
953                 offset += contraction.size();
954                 emitted_contraction = true;
955                 break;
956             }
957         }
958         if (emitted_contraction) {
959             continue;
960         }
961
962         const std::size_t begin = offset;
963         bool has_prefix_space = false;
964         if (text[offset] == ' ' && offset + 1 < text.size() &&
965             !is_ascii_space(static_cast<unsigned char>(text[offset + 1]))) {
966             has_prefix_space = true;
967             ++offset;
968         }
969
970         if (offset >= text.size()) {
971             pieces.emplace_back(text.substr(begin, offset - begin));
972             continue;
973         }
974
975         const unsigned char ch = static_cast<unsigned char>(text[offset]);
976         if (is_ascii_letter(ch)) {
977             while (offset < text.size() &&
978                 is_ascii_letter(static_cast<unsigned char>(text[offset]))) {
979                 ++offset;
980             }
981         } else if (is_ascii_digit(ch)) {
982             while (offset < text.size() &&
983                 is_ascii_digit(static_cast<unsigned char>(text[offset]))) {

```

```

984         ++offset;
985     }
986     } else if (is_gpt2_punctuation(ch) && ch != LCQI_GPT2_ASCII_APOSTROPHE) {
987         while (offset < text.size() &&
988             is_gpt2_punctuation(static_cast<unsigned char>(text[offset]))
989             && static_cast<unsigned char>(text[offset]) !=
990             LCQI_GPT2_ASCII_APOSTROPHE) {
991             offset = consume_utf8_codepoint(text, offset);
992         }
993     } else if (is_ascii_space(ch)) {
994         while (offset < text.size() &&
995             is_ascii_space(static_cast<unsigned char>(text[offset]))) {
996             ++offset;
997         }
998     } else {
999         offset = consume_utf8_codepoint(text, offset);
1000     }
1001     if (has_prefix_space || offset > begin) {
1002         pieces.emplace_back(text.substr(begin, offset - begin));
1003     } else {
1004         ++offset;
1005     }
1006 }
1007 return pieces;
1008 }
1009
1010 std::int32_t parse_i32_field(JsonCursor& cursor) {
1011     const std::int64_t value = cursor.parse_i64();
1012     if (value < std::numeric_limits<std::int32_t>::min() ||
1013         value > std::numeric_limits<std::int32_t>::max()) {
1014         throw std::runtime_error("JSON int32 field out of range");
1015     }
1016     return static_cast<std::int32_t>(value);
1017 }
1018
1019 std::unordered_map<std::string, std::int32_t> parse_vocab_json(std::string_view
1020     text) {
1021     JsonCursor cursor(text);
1022     std::unordered_map<std::string, std::int32_t> vocab;
1023     cursor.expect('{');
1024     if (cursor.consume('}')) {
1025         return vocab;
1026     }
1027     while (true) {
1028         const std::string key = cursor.parse_string();
1029         cursor.expect(':');
1030         const std::int32_t id = parse_i32_field(cursor);
1031         vocab.emplace(key, id);
1032         if (cursor.consume('}')) {

```

```

1032         break;
1033     }
1034     cursor.expect(',');
1035 }
1036 if (!cursor.eof_after_ws()) {
1037     throw std::runtime_error("trailing data after vocab JSON");
1038 }
1039 return vocab;
1040 }
1041
1042 std::vector<std::string> build_id_to_token(
1043     const std::unordered_map<std::string, std::int32_t>& vocab) {
1044     std::int32_t max_id = -1;
1045     for (const auto& [token, id] : vocab) {
1046         if (id < 0) {
1047             throw std::runtime_error("GPT-2 vocab id cannot be negative");
1048         }
1049         max_id = std::max(max_id, id);
1050     }
1051     std::vector<std::string> result(check_size(max_id + 1, "max vocab id"));
1052     for (const auto& [token, id] : vocab) {
1053         std::string& slot = result[checked_size(id, "vocab id")];
1054         if (!slot.empty()) {
1055             throw std::runtime_error("duplicate GPT-2 vocab id");
1056         }
1057         slot = token;
1058     }
1059     return result;
1060 }
1061
1062 std::unordered_map<std::string, std::int32_t> parse_merges(std::string_view ↵
    ↵ text) {
1063     std::unordered_map<std::string, std::int32_t> ranks;
1064     std::istringstream input{std::string(text)};
1065     std::string line;
1066     std::int32_t rank = 0;
1067     while (std::getline(input, line)) {
1068         if (line.empty() || line[0] == '#') {
1069             continue;
1070         }
1071         std::istringstream line_stream(line);
1072         std::string left;
1073         std::string right;
1074         if (!(line_stream >> left >> right)) {
1075             throw std::runtime_error("invalid GPT-2 merges line");
1076         }
1077         ranks.emplace(pair_key(left, right), rank);
1078         ++rank;
1079     }
1080     return ranks;
1081 }
1082

```

```

1083 void transpose_hf_conv1d(std::vector<float>& values,
1084                          std::int32_t input_size,
1085                          std::int32_t output_size) {
1086     if (values.size() != matrix_size(input_size, output_size)) {
1087         throw std::runtime_error("GPT-2 Conv1D tensor size mismatch");
1088     }
1089     std::vector<float> transposed(matrix_size(output_size, input_size), 0.0F);
1090     for (std::int32_t input = 0; input < input_size; ++input) {
1091         for (std::int32_t output = 0; output < output_size; ++output) {
1092             transposed[matrix_size(output, input_size) + checked_size(input, "↵
↵ input")] =
1093                 values[matrix_size(input, output_size) + checked_size(output, "↵
↵ output")];
1094         }
1095     }
1096     values = std::move(transposed);
1097 }
1098
1099 std::vector<float> require_tensor(const SafeTensorFile& file,
1100                                  std::string_view name,
1101                                  std::span<const std::int64_t> expected_shape)↵
↵ {
1102     const SafeTensorEntry* entry = file.manifest.find_tensor(name);
1103     if (entry == nullptr) {
1104         throw std::runtime_error("missing GPT-2 tensor: " + std::string(name));
1105     }
1106     if (entry->shape.size() != expected_shape.size()) {
1107         throw std::runtime_error("GPT-2 tensor rank mismatch: " + std::string(↵
↵ name));
1108     }
1109     for (std::size_t index = 0; index < expected_shape.size(); ++index) {
1110         if (entry->shape[index] != expected_shape[index]) {
1111             throw std::runtime_error("GPT-2 tensor shape mismatch: " + std::↵
↵ string(name));
1112         }
1113     }
1114     return read_safetensor_f32_tensor(file, name);
1115 }
1116
1117 std::vector<float> require_first_tensor(const SafeTensorFile& file,
1118                                         std::span<const std::string_view> names↵
↵ ,
1119                                         std::span<const std::int64_t> ↵
↵ expected_shape) {
1120     for (const std::string_view name : names) {
1121         if (file.manifest.find_tensor(name) != nullptr) {
1122             return require_tensor(file, name, expected_shape);
1123         }
1124     }
1125     throw std::runtime_error("missing GPT-2 tensor: " + std::string(names.front↵
↵ ()));
1126 }

```

```

1127
1128 Gpt2LinearF32 make_linear(std::int32_t input_size,
1129                             std::int32_t output_size,
1130                             std::vector<float> weights,
1131                             std::vector<float> bias = {}) {
1132     Gpt2LinearF32 linear;
1133     linear.input_size = input_size;
1134     linear.output_size = output_size;
1135     linear.weights = std::move(weights);
1136     linear.bias = std::move(bias);
1137     validate_linear(linear, "make GPT-2 linear");
1138     return linear;
1139 }
1140
1141 std::vector<float> zeros(std::int32_t count) {
1142     return std::vector<float>(checked_size(count, "zero count"), 0.0F);
1143 }
1144
1145 std::vector<float> ones(std::int32_t count) {
1146     return std::vector<float>(checked_size(count, "one count"), 1.0F);
1147 }
1148
1149 std::string tensor_name(std::int32_t layer, std::string_view suffix) {
1150     return "transformer.h." + std::to_string(layer) + "." + std::string(suffix) ←
    ↪ ;
1151 }
1152
1153 std::string bare_tensor_name(std::int32_t layer, std::string_view suffix) {
1154     return "h." + std::to_string(layer) + "." + std::string(suffix);
1155 }
1156
1157 std::array<std::string_view, 2> embedding_names(std::string_view bare) {
1158     if (bare == "wte.weight") {
1159         return {"transformer.wte.weight", "wte.weight"};
1160     }
1161     if (bare == "wpe.weight") {
1162         return {"transformer.wpe.weight", "wpe.weight"};
1163     }
1164     if (bare == "ln_f.weight") {
1165         return {"transformer.ln_f.weight", "ln_f.weight"};
1166     }
1167     if (bare == "ln_f.bias") {
1168         return {"transformer.ln_f.bias", "ln_f.bias"};
1169     }
1170     throw std::runtime_error("unsupported GPT-2 embedding tensor alias");
1171 }
1172
1173 std::array<std::string, 2> layer_names(std::int32_t layer, std::string_view ←
    ↪ suffix) {
1174     return {tensor_name(layer, suffix), bare_tensor_name(layer, suffix)};
1175 }
1176

```

```

1177 std::vector<float> require_layer_tensor(const SafeTensorFile& file,
1178                                         std::int32_t layer,
1179                                         std::string_view suffix,
1180                                         std::span<const std::int64_t> ↵
1181                                         ↵ expected_shape) {
1182     const std::array<std::string, 2> names = layer_names(layer, suffix);
1183     const std::array<std::string_view, 2> views{names[0], names[1]};
1184     return require_first_tensor(file, views, expected_shape);
1185 }
1186
1187 std::filesystem::path find_gpt2_weight_file(const std::filesystem::path& ↵
1188 ↵ model_directory) {
1189     const std::array<const char*, 2> candidates{
1190         "model.safetensors",
1191         "pytorch_model.safetensors",
1192     };
1193     for (const char* candidate : candidates) {
1194         const std::filesystem::path path = model_directory / candidate;
1195         if (std::filesystem::exists(path)) {
1196             return path;
1197         }
1198     }
1199     throw std::runtime_error("cannot find model.safetensors in GPT-2 directory" ↵
1200 ↵ );
1201 }
1202 // namespace
1203
1204 Gpt2Tokenizer load_gpt2_tokenizer(const std::filesystem::path& vocab_json,
1205                                   const std::filesystem::path& merges_txt,
1206                                   std::int32_t bos_token_id,
1207                                   std::int32_t eos_token_id) {
1208     Gpt2Tokenizer tokenizer;
1209     tokenizer.bos_token_id = bos_token_id;
1210     tokenizer.eos_token_id = eos_token_id;
1211     tokenizer.vocab = parse_vocab_json(read_text_file(vocab_json));
1212     tokenizer.id_to_token = build_id_to_token(tokenizer.vocab);
1213     tokenizer.bpe_ranks = parse_merges(read_text_file(merges_txt));
1214     if (tokenizer.vocab.empty() || tokenizer.bpe_ranks.empty()) {
1215         throw std::runtime_error("GPT-2 tokenizer assets are empty");
1216     }
1217     return tokenizer;
1218 }
1219
1220 std::vector<std::int32_t> gpt2_encode(const Gpt2Tokenizer& tokenizer,
1221                                       std::string_view text) {
1222     std::vector<std::int32_t> ids;
1223     for (const std::string& piece : gpt2_pretokenize(text)) {
1224         const std::string byte_piece = bytes_to_bpe_alphabet(piece);
1225         const std::vector<std::string> bpe_tokens = bpe_apply(tokenizer, ↵
1226 ↵ byte_piece);
1227         for (const std::string& token : bpe_tokens) {

```

```

1225         const auto found = tokenizer.vocab.find(token);
1226         if (found == tokenizer.vocab.end()) {
1227             throw std::runtime_error("GPT-2 BPE token is missing from vocab");
1228         }
1229         ids.push_back(found->second);
1230     }
1231 }
1232 return ids;
1233 }
1234
1235 std::string gpt2_decode(const Gpt2Tokenizer& tokenizer,
1236                        std::span<const std::int32_t> token_ids) {
1237     static const std::vector<std::string> BYTE_ENCODER = gpt2_bytes_to_unicode();
1238     static const std::unordered_map<std::string, unsigned char> BYTE_DECODER =
1239         gpt2_unicode_to_bytes(BYTE_ENCODER);
1240
1241     std::string token_text;
1242     for (const std::int32_t token_id : token_ids) {
1243         if (token_id < 0 || token_id >= static_cast<std::int32_t>(tokenizer.id_to_token.size())) {
1244             throw std::runtime_error("GPT-2 token id out of range");
1245         }
1246         token_text += tokenizer.id_to_token[checked_size(token_id, "token id")];
1247     }
1248
1249     std::string result;
1250     const std::vector<std::string> symbols = utf8_symbols(token_text);
1251     for (const std::string& symbol : symbols) {
1252         const auto found = BYTE_DECODER.find(symbol);
1253         if (found == BYTE_DECODER.end()) {
1254             throw std::runtime_error("GPT-2 token cannot be decoded to byte");
1255         }
1256         result.push_back(static_cast<char>(found->second));
1257     }
1258     return result;
1259 }
1260
1261 Gpt2Config load_gpt2_config(const std::filesystem::path& config_json) {
1262     const std::string config_text = read_text_file(config_json);
1263     JsonCursor cursor(config_text);
1264     Gpt2Config config;
1265     config.bos_token_id = LCQI_GPT2_DEFAULT_BOS_EOS;
1266     config.eos_token_id = LCQI_GPT2_DEFAULT_BOS_EOS;
1267
1268     cursor.expect('{');
1269     if (cursor.consume('}') {
1270         throw std::runtime_error("GPT-2 config is empty");
1271     }
1272     while (true) {

```

```

1273     const std::string key = cursor.parse_string();
1274     cursor.expect(':');
1275     if (key == "n_embd") {
1276         config.hidden_size = parse_i32_field(cursor);
1277     } else if (key == "n_head") {
1278         config.head_count = parse_i32_field(cursor);
1279     } else if (key == "n_layer") {
1280         config.layer_count = parse_i32_field(cursor);
1281     } else if (key == "n_positions" || key == "n_ctx") {
1282         const std::int32_t value = parse_i32_field(cursor);
1283         config.max_positions = std::max(config.max_positions, value);
1284     } else if (key == "vocab_size") {
1285         config.vocab_size = parse_i32_field(cursor);
1286     } else if (key == "n_inner") {
1287         config.intermediate_size = parse_i32_field(cursor);
1288     } else if (key == "layer_norm_epsilon") {
1289         config.layer_norm_epsilon = static_cast<float>(cursor.parse_number←
↪ ());
1290     } else if (key == "activation_function") {
1291         config.activation_function = cursor.parse_string();
1292     } else if (key == "bos_token_id") {
1293         config.bos_token_id = parse_i32_field(cursor);
1294     } else if (key == "eos_token_id") {
1295         config.eos_token_id = parse_i32_field(cursor);
1296     } else {
1297         cursor.skip_value();
1298     }
1299     if (cursor.consume('}')) {
1300         break;
1301     }
1302     cursor.expect(',');
1303 }
1304 if (!cursor.eof_after_ws()) {
1305     throw std::runtime_error("trailing data after GPT-2 config JSON");
1306 }
1307 if (config.intermediate_size == 0 && config.hidden_size > 0) {
1308     config.intermediate_size = config.hidden_size * 4;
1309 }
1310 validate_config(config);
1311 return config;
1312 }
1313
1314 Gpt2ReferenceModel load_gpt2_reference_model(const std::filesystem::path& ←
↪ config_json,
1315                                             const std::filesystem::path& ←
↪ safetensors_path) {
1316     Gpt2ReferenceModel model;
1317     model.config = load_gpt2_config(config_json);
1318     const SafeTensorFile file = load_safetensors_file(safetensors_path);
1319     const Gpt2Config& config = model.config;
1320     const std::array<std::int64_t, 2> wte_shape{config.vocab_size, config.←
↪ hidden_size};

```



```

1321     const std::array<std::int64_t, 2> wpe_shape{config.max_positions, config.↵
↵ hidden_size};
1322     const std::array<std::int64_t, 1> hidden_shape{config.hidden_size};
1323
1324     const std::array<std::string_view, 2> wte_names = embedding_names("wte.↵
↵ weight");
1325     const std::array<std::string_view, 2> wpe_names = embedding_names("wpe.↵
↵ weight");
1326     model.token_embedding = require_first_tensor(file, wte_names, wte_shape);
1327     model.position_embedding = require_first_tensor(file, wpe_names, wpe_shape)↵
↵ ;
1328     model.layers.resize(check_size(config.layer_count, "layer_count"));
1329
1330     for (std::int32_t layer_id = 0; layer_id < config.layer_count; ++layer_id) ↵
↵ {
1331         Gpt2LayerWeightsF32& layer = model.layers[check_size(layer_id, "↵
↵ layer_id")];
1332         layer.ln_1_weight = require_layer_tensor(file, layer_id, "ln_1.weight",↵
↵ hidden_shape);
1333         layer.ln_1_bias = require_layer_tensor(file, layer_id, "ln_1.bias", ↵
↵ hidden_shape);
1334         layer.ln_2_weight = require_layer_tensor(file, layer_id, "ln_2.weight",↵
↵ hidden_shape);
1335         layer.ln_2_bias = require_layer_tensor(file, layer_id, "ln_2.bias", ↵
↵ hidden_shape);
1336
1337         const std::array<std::int64_t, 2> c_attn_shape{
1338             config.hidden_size,
1339             config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS};
1340         std::vector<float> c_attn_weight =
1341             require_layer_tensor(file, layer_id, "attn.c_attn.weight", ↵
↵ c_attn_shape);
1342         transpose_hf_conv1d(c_attn_weight,
1343                             config.hidden_size,
1344                             config.hidden_size * ↵
↵ LCQI_GPT2_ATTENTION_PROJECTIONS);
1345         const std::array<std::int64_t, 1> c_attn_bias_shape{
1346             config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS};
1347         layer.c_attn = make_linear(config.hidden_size,
1348                                   config.hidden_size * ↵
↵ LCQI_GPT2_ATTENTION_PROJECTIONS,
1349                                   std::move(c_attn_weight),
1350                                   require_layer_tensor(file,
1351                                                         layer_id,
1352                                                         "attn.c_attn.bias",
1353                                                         c_attn_bias_shape));
1354
1355         const std::array<std::int64_t, 2> c_proj_shape{config.hidden_size, ↵
↵ config.hidden_size};
1356         std::vector<float> c_proj_weight =
1357             require_layer_tensor(file, layer_id, "attn.c_proj.weight", ↵
↵ c_proj_shape);

```

```

1358     transpose_hf_conv1d(c_proj_weight, config.hidden_size, config.↵
↵ hidden_size);
1359     layer.c_proj = make_linear(config.hidden_size,
1360                               config.hidden_size,
1361                               std::move(c_proj_weight),
1362                               require_layer_tensor(file,
1363                                                    layer_id,
1364                                                    "attn.c_proj.bias",
1365                                                    hidden_shape));
1366
1367     const std::array<std::int64_t, 2> c_fc_shape{
1368         config.hidden_size,
1369         config.intermediate_size};
1370     std::vector<float> c_fc_weight =
1371         require_layer_tensor(file, layer_id, "mlp.c_fc.weight", c_fc_shape)↵
↵ ;
1372     transpose_hf_conv1d(c_fc_weight, config.hidden_size, config.↵
↵ intermediate_size);
1373     const std::array<std::int64_t, 1> intermediate_shape{config.↵
↵ intermediate_size};
1374     layer.c_fc = make_linear(config.hidden_size,
1375                              config.intermediate_size,
1376                              std::move(c_fc_weight),
1377                              require_layer_tensor(file,
1378                                                    layer_id,
1379                                                    "mlp.c_fc.bias",
1380                                                    intermediate_shape));
1381
1382     const std::array<std::int64_t, 2> mlp_proj_shape{
1383         config.intermediate_size,
1384         config.hidden_size};
1385     std::vector<float> mlp_proj_weight =
1386         require_layer_tensor(file, layer_id, "mlp.c_proj.weight", ↵
↵ mlp_proj_shape);
1387     transpose_hf_conv1d(mlp_proj_weight, config.intermediate_size, config.↵
↵ hidden_size);
1388     layer.mlp_c_proj = make_linear(config.intermediate_size,
1389                                    config.hidden_size,
1390                                    std::move(mlp_proj_weight),
1391                                    require_layer_tensor(file,
1392                                                         layer_id,
1393                                                         "mlp.c_proj.bias",
1394                                                         hidden_shape));
1395 }
1396
1397 const std::array<std::string_view, 2> ln_weight_names = embedding_names("↵
↵ ln_f.weight");
1398 const std::array<std::string_view, 2> ln_bias_names = embedding_names("ln_f↵
↵ .bias");
1399 model.final_ln_weight = require_first_tensor(file, ln_weight_names, ↵
↵ hidden_shape);
1400 model.final_ln_bias = require_first_tensor(file, ln_bias_names, ↵

```

```

    ↪ hidden_shape);
1401 if (file.manifest.find_tensor("lm_head.weight") != nullptr) {
1402     model.tie_lm_head_to_embedding = false;
1403     model.lm_head_weight = require_tensor(file, "lm_head.weight", wte_shape ↪
    ↪ );
1404 } else {
1405     model.tie_lm_head_to_embedding = true;
1406 }
1407 validate_model(model);
1408 return model;
1409 }
1410
1411 Gpt2ReferenceModel load_gpt2_from_directory(const std::filesystem::path& ↪
    ↪ model_directory) {
1412     return load_gpt2_reference_model(model_directory / "config.json",
1413                                     find_gpt2_weight_file(model_directory));
1414 }
1415
1416 Gpt2KvCache::Gpt2KvCache(const Gpt2Config& config) : config_(config) {
1417     validate_config(this->config_);
1418     const std::size_t element_count =
1419         checked_size(this->config_.layer_count, "layer_count") *
1420         checked_size(this->config_.max_positions, "max_positions") *
1421         checked_size(this->config_.head_count, "head_count") *
1422         checked_size(head_dim(this->config_), "head_dim");
1423     this->keys_.assign(element_count, 0.0F);
1424     this->values_.assign(element_count, 0.0F);
1425     this->written_.assign(
1426         checked_size(this->config_.layer_count, "layer_count") *
1427         checked_size(this->config_.max_positions, "max_positions"),
1428         0);
1429 }
1430
1431 void Gpt2KvCache::append(std::int32_t layer_id,
1432                          std::int32_t model_position,
1433                          std::span<const float> key,
1434                          std::span<const float> value) {
1435     const std::size_t hidden_size = checked_size(this->config_.hidden_size, " ↪
    ↪ hidden_size");
1436     if (key.size() != hidden_size || value.size() != hidden_size) {
1437         throw std::runtime_error("GPT-2 KV key/value size mismatch");
1438     }
1439     this->validate_address(layer_id, model_position, 0);
1440     const std::int32_t local_head_dim = head_dim(this->config_);
1441     for (std::int32_t head = 0; head < this->config_.head_count; ++head) {
1442         const std::size_t source =
1443             checked_size(head, "head") * checked_size(local_head_dim, "head_dim ↪
    ↪ ");
1444         const std::size_t target = this->base_offset(layer_id, model_position, ↪
    ↪ head);
1445         std::copy_n(key.begin() + static_cast<std::ptrdiff_t>(source),
1446                    local_head_dim,

```

```

1447         this->keys_.begin() + static_cast<std::ptrdiff_t>(target));
1448     std::copy_n(value.begin() + static_cast<std::ptrdiff_t>(source),
1449               local_head_dim,
1450               this->values_.begin() + static_cast<std::ptrdiff_t>(target)↵
↵ );
1451 }
1452 const std::size_t written_index =
1453     checked_size(layer_id, "layer_id") *
1454     checked_size(this->config_.max_positions, "max_positions") +
1455     checked_size(model_position, "model_position");
1456 this->written_[written_index] = 1;
1457 this->filled_tokens_ = std::max(this->filled_tokens_, model_position + 1);
1458 }
1459
1460 std::span<const float> Gpt2KvCache::key(std::int32_t layer_id,
1461                                       std::int32_t model_position,
1462                                       std::int32_t head) const {
1463     this->validate_address(layer_id, model_position, head);
1464     this->validate_written(layer_id, model_position);
1465     const std::size_t offset = this->base_offset(layer_id, model_position, head)↵
↵ );
1466     return std::span<const float>(this->keys_.data() + offset,
1467                                  checked_size(head_dim(this->config_), "↵
↵ head_dim"));
1468 }
1469
1470 std::span<const float> Gpt2KvCache::value(std::int32_t layer_id,
1471                                       std::int32_t model_position,
1472                                       std::int32_t head) const {
1473     this->validate_address(layer_id, model_position, head);
1474     this->validate_written(layer_id, model_position);
1475     const std::size_t offset = this->base_offset(layer_id, model_position, head)↵
↵ );
1476     return std::span<const float>(this->values_.data() + offset,
1477                                  checked_size(head_dim(this->config_), "↵
↵ head_dim"));
1478 }
1479
1480 std::int32_t Gpt2KvCache::filled_tokens() const noexcept {
1481     return this->filled_tokens_;
1482 }
1483
1484 std::size_t Gpt2KvCache::byte_size() const noexcept {
1485     return (this->keys_.size() + this->values_.size()) * sizeof(float) +
1486            this->written_.size() * sizeof(std::uint8_t);
1487 }
1488
1489 std::size_t Gpt2KvCache::base_offset(std::int32_t layer_id,
1490                                       std::int32_t model_position,
1491                                       std::int32_t head) const {
1492     return (((checked_size(layer_id, "layer_id") *
1493              checked_size(this->config_.max_positions, "max_positions")) +

```

```

1494         checked_size(model_position, "model_position")) *
1495         checked_size(this->config_.head_count, "head_count") +
1496         checked_size(head, "head")) *
1497         checked_size(head_dim(this->config_), "head_dim");
1498     }
1499
1500     void Gpt2KvCache::validate_address(std::int32_t layer_id,
1501                                       std::int32_t model_position,
1502                                       std::int32_t head) const {
1503         if (layer_id < 0 || layer_id >= this->config_.layer_count) {
1504             throw std::runtime_error("GPT-2 KV layer_id out of range");
1505         }
1506         if (model_position < 0 || model_position >= this->config_.max_positions) {
1507             throw std::runtime_error("GPT-2 KV model_position out of range");
1508         }
1509         if (head < 0 || head >= this->config_.head_count) {
1510             throw std::runtime_error("GPT-2 KV head out of range");
1511         }
1512     }
1513
1514     void Gpt2KvCache::validate_written(std::int32_t layer_id,
1515                                       std::int32_t model_position) const {
1516         const std::size_t written_index =
1517             checked_size(layer_id, "layer_id") *
1518             checked_size(this->config_.max_positions, "max_positions") +
1519             checked_size(model_position, "model_position");
1520         if (written_index >= this->written_.size() || this->written_[written_index] ←
1521             == 0) {
1522             throw std::runtime_error("GPT-2 KV slot has not been written");
1523         }
1524     }
1525
1526     Gpt2ForwardResult run_gpt2_forward(const Gpt2ReferenceModel& model,
1527                                       std::span<const std::int32_t> token_ids) {
1528         validate_model(model);
1529         if (token_ids.empty()) {
1530             throw std::runtime_error("GPT-2 forward needs at least one token");
1531         }
1532         if (token_ids.size() > checked_size(model.config.max_positions, " ←
1533             max_positions")) {
1534             throw std::runtime_error("GPT-2 token_ids exceed max_positions");
1535         }
1536
1537         const std::int32_t sequence_length = static_cast<std::int32_t>(token_ids. ←
1538             size());
1539         std::vector<float> hidden_by_pos(matrix_size(sequence_length, model.config. ←
1540             hidden_size), 0.0F);
1541         for (std::int32_t position = 0; position < sequence_length; ++position) {
1542             const std::int32_t token_id = token_ids[checked_size(position, " ←
1543                 position")];
1544             if (token_id < 0 || token_id >= model.config.vocab_size) {
1545                 throw std::runtime_error("GPT-2 token id out of range");

```

```

1541     }
1542     const std::span<const float> token =
1543         row_span(model.token_embedding, token_id, model.config.hidden_size)↵
1544     ↵ ;
1545     const std::span<const float> position_embedding =
1546         row_span(model.position_embedding, position, model.config.↵
1547     ↵ hidden_size);
1548     std::span<float> hidden = std::span<float>(
1549         hidden_by_pos.data() + matrix_size(position, model.config.↵
1550     ↵ hidden_size),
1551         checked_size(model.config.hidden_size, "hidden_size"));
1552     for (std::int32_t dim = 0; dim < model.config.hidden_size; ++dim) {
1553         hidden[checked_size(dim, "dim")] =
1554             token[checked_size(dim, "dim")] + position_embedding[↵
1555     ↵ checked_size(dim, "dim")];
1556     }
1557 }
1558
1559 for (const Gpt2LayerWeightsF32& layer : model.layers) {
1560     forward_gpt2_layer(model.config, layer, sequence_length, hidden_by_pos)↵
1561     ↵ ;
1562 }
1563
1564 std::vector<float> normed(checked_size(model.config.hidden_size, "↵
1565     ↵ hidden_size"), 0.0F);
1566 const std::span<const float> final_hidden = std::span<const float>(
1567     hidden_by_pos.data() + matrix_size(sequence_length - 1, model.config.↵
1568     ↵ hidden_size),
1569     checked_size(model.config.hidden_size, "hidden_size"));
1570 layer_norm(final_hidden,
1571     model.final_ln_weight,
1572     model.final_ln_bias,
1573     model.config.layer_norm_epsilon,
1574     normed);
1575
1576 Gpt2ForwardResult result;
1577 result.logits.assign(checked_size(model.config.vocab_size, "vocab_size"), ↵
1578     ↵ 0.0F);
1579 compute_logits(model, normed, result.logits);
1580 result.predicted_token = argmax(result.logits);
1581 return result;
1582 }
1583
1584 Gpt2ForwardResult run_gpt2_forward_cached(const Gpt2ReferenceModel& model,
1585     Gpt2KvCache& cache,
1586     std::int32_t token_id) {
1587     validate_model(model);
1588     if (token_id < 0 || token_id >= model.config.vocab_size) {
1589         throw std::runtime_error("GPT-2 token id out of range");
1590     }
1591     const std::int32_t model_position = cache.filled_tokens();
1592     if (model_position >= model.config.max_positions) {

```

```

1585     throw std::runtime_error("GPT-2 cached forward would exceed ↵
↵ max_positions");
1586 }
1587
1588 std::vector<float> hidden(check_size(model.config.hidden_size, "↵
↵ hidden_size"), 0.0F);
1589 const std::span<const float> token =
1590     row_span(model.token_embedding, token_id, model.config.hidden_size);
1591 const std::span<const float> position_embedding =
1592     row_span(model.position_embedding, model_position, model.config.↵
↵ hidden_size);
1593 for (std::int32_t dim = 0; dim < model.config.hidden_size; ++dim) {
1594     hidden[checked_size(dim, "dim")] =
1595         token[checked_size(dim, "dim")] + position_embedding[checked_size(↵
↵ dim, "dim")];
1596 }
1597
1598 for (std::int32_t layer_id = 0;
1599     layer_id < static_cast<std::int32_t>(model.layers.size()));
1600     ++layer_id) {
1601     forward_gpt2_layer_cached(model.config,
1602                               model.layers[checked_size(layer_id, "layer_id↵
↵ ")]),
1603                               layer_id,
1604                               model_position,
1605                               cache,
1606                               hidden);
1607 }
1608
1609 std::vector<float> normed(check_size(model.config.hidden_size, "↵
↵ hidden_size"), 0.0F);
1610 layer_norm(hidden,
1611            model.final_ln_weight,
1612            model.final_ln_bias,
1613            model.config.layer_norm_epsilon,
1614            normed);
1615
1616 Gpt2ForwardResult result;
1617 result.logits.assign(check_size(model.config.vocab_size, "vocab_size"), ↵
↵ 0.0F);
1618 compute_logits(model, normed, result.logits);
1619 result.predicted_token = argmax(result.logits);
1620 return result;
1621 }
1622
1623 std::vector<std::int32_t> gpt2_generate_greedy(const Gpt2ReferenceModel& model,
1624                                               std::span<const std::int32_t> ↵
↵ prompt_token_ids,
1625                                               std::int32_t max_new_tokens) {
1626     if (max_new_tokens < 0) {
1627         throw std::runtime_error("max_new_tokens cannot be negative");
1628     }

```



```

1629     std::vector<std::int32_t> tokens(prompt_token_ids.begin(), prompt_token_ids↵
↵ .end());
1630     for (std::int32_t step = 0; step < max_new_tokens; ++step) {
1631         if (tokens.size() >= checked_size(model.config.max_positions, "↵
↵ max_positions")) {
1632             throw std::runtime_error("GPT-2 generation would exceed ↵
↵ max_positions");
1633         }
1634         const Gpt2ForwardResult result = run_gpt2_forward(model, tokens);
1635         tokens.push_back(result.predicted_token);
1636         if (model.config.eos_token_id >= 0 && result.predicted_token == model.↵
↵ config.eos_token_id) {
1637             break;
1638         }
1639     }
1640     return tokens;
1641 }
1642
1643 std::vector<std::int32_t> gpt2_generate_greedy_cached(
1644     const Gpt2ReferenceModel& model,
1645     std::span<const std::int32_t> prompt_token_ids,
1646     std::int32_t max_new_tokens) {
1647     if (prompt_token_ids.empty()) {
1648         throw std::runtime_error("GPT-2 generation needs at least one prompt ↵
↵ token");
1649     }
1650     if (max_new_tokens < 0) {
1651         throw std::runtime_error("max_new_tokens cannot be negative");
1652     }
1653     if (prompt_token_ids.size() > checked_size(model.config.max_positions, "↵
↵ max_positions")) {
1654         throw std::runtime_error("GPT-2 prompt exceeds max_positions");
1655     }
1656
1657     Gpt2KvCache cache(model.config);
1658     std::vector<std::int32_t> tokens(prompt_token_ids.begin(), prompt_token_ids↵
↵ .end());
1659     if (max_new_tokens == 0) {
1660         return tokens;
1661     }
1662     Gpt2ForwardResult result;
1663     for (const std::int32_t token_id : prompt_token_ids) {
1664         result = run_gpt2_forward_cached(model, cache, token_id);
1665     }
1666     for (std::int32_t step = 0; step < max_new_tokens; ++step) {
1667         if (tokens.size() >= checked_size(model.config.max_positions, "↵
↵ max_positions")) {
1668             throw std::runtime_error("GPT-2 generation would exceed ↵
↵ max_positions");
1669         }
1670         tokens.push_back(result.predicted_token);
1671         if (model.config.eos_token_id >= 0 && result.predicted_token == model.↵

```



```

    ↪ config.eos_token_id) {
1672         break;
1673     }
1674     if (step + 1 < max_new_tokens) {
1675         result = run_gpt2_forward_cached(model, cache, result.↪
    ↪ predicted_token);
1676     }
1677 }
1678 return tokens;
1679 }
1680
1681 Gpt2ReferenceModel make_tiny_gpt2_reference_model() {
1682     Gpt2ReferenceModel model;
1683     model.config.hidden_size = 4;
1684     model.config.head_count = 2;
1685     model.config.layer_count = 1;
1686     model.config.max_positions = 8;
1687     model.config.vocab_size = 6;
1688     model.config.intermediate_size = 8;
1689     model.config.bos_token_id = 0;
1690     model.config.eos_token_id = 5;
1691     model.config.layer_norm_epsilon = 1.0e-5F;
1692     model.config.activation_function = "gelu_new";
1693
1694     model.token_embedding = {
1695         0.20F, -0.10F, 0.00F, 0.10F,
1696         -0.10F, 0.30F, 0.20F, -0.20F,
1697         0.05F, 0.10F, -0.30F, 0.25F,
1698         0.40F, -0.20F, 0.15F, 0.05F,
1699         -0.30F, 0.05F, 0.35F, -0.15F,
1700         0.10F, 0.20F, -0.10F, 0.30F,
1701     };
1702     model.position_embedding = {
1703         0.01F, 0.02F, 0.03F, 0.04F,
1704         -0.02F, 0.01F, 0.00F, 0.03F,
1705         0.03F, -0.01F, 0.02F, 0.00F,
1706         0.00F, 0.02F, -0.02F, 0.01F,
1707         0.02F, 0.00F, 0.01F, -0.01F,
1708         -0.01F, 0.03F, 0.00F, 0.02F,
1709         0.01F, -0.02F, 0.03F, 0.00F,
1710         0.00F, 0.01F, -0.01F, 0.02F,
1711     };
1712
1713     Gpt2LayerWeightsF32 layer;
1714     layer.ln_1_weight = ones(4);
1715     layer.ln_1_bias = zeros(4);
1716     layer.ln_2_weight = {0.9F, 1.1F, 1.0F, 0.95F};
1717     layer.ln_2_bias = {0.01F, -0.02F, 0.00F, 0.03F};
1718     layer.c_attn = make_linear(
1719         4,
1720         12,
1721         {

```

```

1722         0.10F, -0.20F, 0.05F, 0.15F,
1723         -0.05F, 0.12F, 0.20F, -0.10F,
1724         0.18F, 0.02F, -0.08F, 0.11F,
1725         -0.12F, 0.06F, 0.14F, 0.04F,
1726         0.07F, 0.16F, -0.09F, 0.03F,
1727         -0.15F, 0.05F, 0.13F, 0.10F,
1728         0.04F, -0.11F, 0.19F, -0.02F,
1729         0.09F, 0.08F, -0.06F, 0.17F,
1730         0.20F, -0.04F, 0.01F, 0.06F,
1731         -0.08F, 0.15F, 0.07F, -0.03F,
1732         0.05F, 0.10F, -0.14F, 0.12F,
1733         0.11F, -0.07F, 0.16F, 0.02F,
1734     },
1735     {
1736         0.01F, -0.02F, 0.00F, 0.03F,
1737         -0.01F, 0.02F, 0.01F, -0.02F,
1738         0.00F, 0.01F, -0.01F, 0.02F,
1739     });
1740     layer.c_proj = make_linear(
1741         4,
1742         4,
1743         {
1744             0.10F, 0.05F, -0.08F, 0.12F,
1745             -0.04F, 0.11F, 0.09F, -0.02F,
1746             0.07F, -0.10F, 0.13F, 0.05F,
1747             0.02F, 0.14F, -0.06F, 0.08F,
1748         },
1749         {0.01F, 0.00F, -0.01F, 0.02F});
1750     layer.c_fc = make_linear(
1751         4,
1752         8,
1753         {
1754             0.20F, -0.10F, 0.05F, 0.03F,
1755             -0.05F, 0.14F, 0.12F, -0.08F,
1756             0.09F, 0.07F, -0.11F, 0.16F,
1757             -0.12F, 0.05F, 0.18F, 0.02F,
1758             0.04F, -0.09F, 0.13F, 0.10F,
1759             0.11F, 0.03F, -0.07F, 0.15F,
1760             -0.02F, 0.17F, 0.06F, -0.04F,
1761             0.08F, -0.06F, 0.10F, 0.12F,
1762         },
1763         {0.01F, -0.02F, 0.00F, 0.03F, -0.01F, 0.02F, 0.01F, 0.00F});
1764     layer.mlp_c_proj = make_linear(
1765         8,
1766         4,
1767         {
1768             0.10F, -0.08F, 0.06F, 0.12F, -0.04F, 0.07F, 0.03F, 0.11F,
1769             -0.05F, 0.13F, 0.02F, -0.09F, 0.15F, 0.04F, -0.03F, 0.08F,
1770             0.07F, 0.01F, -0.12F, 0.10F, 0.06F, -0.05F, 0.14F, 0.02F,
1771             0.03F, 0.09F, 0.11F, -0.06F, 0.05F, 0.12F, -0.08F, 0.04F,
1772         },
1773         {0.00F, 0.01F, -0.01F, 0.02F});

```

```

1774     model.layers.push_back(std::move(layer));
1775     model.final_ln_weight = {1.0F, 0.95F, 1.05F, 1.0F};
1776     model.final_ln_bias = {0.0F, 0.01F, -0.01F, 0.02F};
1777     model.tie_lm_head_to_embedding = true;
1778     validate_model(model);
1779     return model;
1780 }
1781
1782 } // namespace lcqi

```

`gpt2_cli.cpp` 负责把模型目录、prompt、`max_new_tokens` 和 benchmark 选项接到 GPT-2 reference path。无参数时运行 tiny GPT-2 fixture；传入 HuggingFace 模型目录时加载真实资产。

Listing 4.5 src/gpt2_cli.cpp

```

1  #include <lcqi/gpt2_reference.hpp>
2
3  #include <chrono>
4  #include <cstdlib>
5  #include <exception>
6  #include <filesystem>
7  #include <iostream>
8  #include <span>
9  #include <stdexcept>
10 #include <string>
11 #include <string_view>
12 #include <vector>
13
14 namespace {
15
16 constexpr int LCQI_GPT2_TINY_ARGC = 1;
17 constexpr int LCQI_GPT2_MIN_REAL_ARGC = 3;
18 constexpr std::int32_t LCQI_GPT2_DEFAULT_MAX_NEW_TOKENS = 8;
19 constexpr std::int32_t LCQI_GPT2_TINY_PROMPT_0 = 1;
20 constexpr std::int32_t LCQI_GPT2_TINY_PROMPT_1 = 2;
21 constexpr std::int32_t LCQI_GPT2_TINY_PROMPT_2 = 3;
22
23 enum class Gpt2EngineMode {
24     full_prefix,
25     cached_kv,
26 };
27
28 struct Gpt2CliOptions {
29     Gpt2EngineMode engine = Gpt2EngineMode::cached_kv;
30     bool benchmark = false;
31     std::filesystem::path model_dir;
32     std::string prompt;
33     std::int32_t max_new_tokens = LCQI_GPT2_DEFAULT_MAX_NEW_TOKENS;
34 };
35
36 struct Gpt2BenchmarkTimings {
37     double load_ms = 0.0;
38     double tokenize_ms = 0.0;

```

```

39     double prefill_ms = 0.0;
40     double decode_ms = 0.0;
41     double generate_ms = 0.0;
42     double total_ms = 0.0;
43     std::size_t prefill_steps = 0;
44     std::size_t decode_steps = 0;
45     std::size_t kv_cache_bytes = 0;
46 };
47
48 using Clock = std::chrono::steady_clock;
49
50 void print_ids(std::span<const std::int32_t> ids) {
51     for (std::size_t index = 0; index < ids.size(); ++index) {
52         if (index != 0) {
53             std::cout << ' ';
54         }
55         std::cout << ids[index];
56     }
57 }
58
59 [[nodiscard]] double elapsed_ms(Clock::time_point start, Clock::time_point end)↵
    ↵ {
60     return std::chrono::duration<double, std::milli>(end - start).count();
61 }
62
63 [[nodiscard]] std::size_t checked_size(std::int64_t value, const char* name) {
64     if (value < 0) {
65         throw std::runtime_error(std::string(name) + " cannot be negative");
66     }
67     return static_cast<std::size_t>(value);
68 }
69
70 [[nodiscard]] const char* engine_name(Gpt2EngineMode engine) {
71     switch (engine) {
72         case Gpt2EngineMode::full_prefix:
73             return "full_prefix";
74         case Gpt2EngineMode::cached_kv:
75             return "cached_kv";
76     }
77     return "unknown";
78 }
79
80 [[nodiscard]] Gpt2EngineMode parse_engine(std::string_view value) {
81     if (value == "full" || value == "full_prefix") {
82         return Gpt2EngineMode::full_prefix;
83     }
84     if (value == "cached" || value == "cached_kv") {
85         return Gpt2EngineMode::cached_kv;
86     }
87     throw std::runtime_error("unknown --engine value, expected cached or full")↵
    ↵ ;
88 }

```

```

89
90 [[nodiscard]] std::int32_t parse_non_negative_i32(const std::string& text,
91                                                    const char* name) {
92     const std::int32_t value = static_cast<std::int32_t>(std::stoi(text));
93     if (value < 0) {
94         throw std::runtime_error(std::string(name) + " cannot be negative");
95     }
96     return value;
97 }
98
99 [[nodiscard]] Gpt2CliOptions parse_options(int argc, char** argv) {
100     Gpt2CliOptions options;
101     std::vector<std::string> positional;
102     for (int index = 1; index < argc; ++index) {
103         const std::string arg = argv[index];
104         if (arg == "--benchmark") {
105             options.benchmark = true;
106         } else if (arg == "--engine") {
107             if (index + 1 >= argc) {
108                 throw std::runtime_error("--engine requires a value");
109             }
110             ++index;
111             options.engine = parse_engine(argv[index]);
112         } else if (arg.rfind("--engine=", 0) == 0) {
113             options.engine = parse_engine(std::string_view(arg).substr(std::string("--engine=").size()));
114         } else if (arg == "--full") {
115             options.engine = Gpt2EngineMode::full_prefix;
116         } else if (arg == "--cached") {
117             options.engine = Gpt2EngineMode::cached_kv;
118         } else if (!arg.empty() && arg.front() == '-') {
119             throw std::runtime_error("unknown option: " + arg);
120         } else {
121             positional.push_back(arg);
122         }
123     }
124
125     if (positional.size() < 2 || positional.size() > 3) {
126         throw std::runtime_error(
127             "usage: lcqi_gpt2 [--engine cached|full] [--benchmark] "
128             "model_dir prompt [max_new_tokens]");
129     }
130     options.model_dir = positional[0];
131     options.prompt = positional[1];
132     if (positional.size() == 3) {
133         options.max_new_tokens = parse_non_negative_i32(positional[2], "↵
↵ max_new_tokens");
134     }
135     return options;
136 }
137
138 [[nodiscard]] std::vector<std::int32_t> generate_full_prefix(

```

```

139     const lcqi::Gpt2ReferenceModel& model,
140     std::span<const std::int32_t> prompt_ids,
141     std::int32_t max_new_tokens,
142     Gpt2BenchmarkTimings& timings) {
143     const Clock::time_point generate_start = Clock::now();
144     std::vector<std::int32_t> tokens(prompt_ids.begin(), prompt_ids.end());
145     for (std::int32_t step = 0; step < max_new_tokens; ++step) {
146         if (tokens.size() >= checked_size(model.config.max_positions, "↵
↵ max_positions")) {
147             throw std::runtime_error("GPT-2 generation would exceed ↵
↵ max_positions");
148         }
149         const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(model, ↵
↵ tokens);
150         ++timings.decode_steps;
151         tokens.push_back(result.predicted_token);
152         if (model.config.eos_token_id >= 0 && result.predicted_token == model.↵
↵ config.eos_token_id) {
153             break;
154         }
155     }
156     const Clock::time_point generate_end = Clock::now();
157     timings.generate_ms = elapsed_ms(generate_start, generate_end);
158     timings.decode_ms = timings.generate_ms;
159     return tokens;
160 }

161
162 [[nodiscard]] std::vector<std::int32_t> generate_cached(
163     const lcqi::Gpt2ReferenceModel& model,
164     std::span<const std::int32_t> prompt_ids,
165     std::int32_t max_new_tokens,
166     Gpt2BenchmarkTimings& timings) {
167     if (prompt_ids.empty()) {
168         throw std::runtime_error("GPT-2 generation needs at least one prompt ↵
↵ token");
169     }
170     lcqi::Gpt2KvCache cache(model.config);
171     timings.kv_cache_bytes = cache.byte_size();
172     std::vector<std::int32_t> tokens(prompt_ids.begin(), prompt_ids.end());
173     if (max_new_tokens == 0) {
174         return tokens;
175     }
176
177     const Clock::time_point generate_start = Clock::now();
178     lcqi::Gpt2ForwardResult result;
179     const Clock::time_point prefill_start = Clock::now();
180     for (const std::int32_t token_id : prompt_ids) {
181         result = lcqi::run_gpt2_forward_cached(model, cache, token_id);
182         ++timings.prefill_steps;
183     }
184     const Clock::time_point prefill_end = Clock::now();
185     timings.prefill_ms = elapsed_ms(prefill_start, prefill_end);

```

```

186
187     const Clock::time_point decode_start = Clock::now();
188     for (std::int32_t step = 0; step < max_new_tokens; ++step) {
189         if (tokens.size() >= checked_size(model.config.max_positions, "↵
↵ max_positions")) {
190             throw std::runtime_error("GPT-2 generation would exceed ↵
↵ max_positions");
191         }
192         tokens.push_back(result.predicted_token);
193         if (model.config.eos_token_id >= 0 && result.predicted_token == model.↵
↵ config.eos_token_id) {
194             break;
195         }
196         if (step + 1 < max_new_tokens) {
197             result = lcqi::run_gpt2_forward_cached(model, cache, result.↵
↵ predicted_token);
198             ++timings.decode_steps;
199         }
200     }
201     const Clock::time_point decode_end = Clock::now();
202     const Clock::time_point generate_end = Clock::now();
203     timings.decode_ms = elapsed_ms(decode_start, decode_end);
204     timings.generate_ms = elapsed_ms(generate_start, generate_end);
205     return tokens;
206 }
207
208 void print_benchmark(const Gpt2BenchmarkTimings& timings,
209                     std::size_t prompt_tokens,
210                     std::size_t generated_tokens) {
211     const double new_tokens = static_cast<double>(generated_tokens);
212     const double generated_tokens_per_second =
213         timings.generate_ms > 0.0 ? new_tokens * 1000.0 / timings.generate_ms :↵
↵ 0.0;
214     const double decode_tokens_per_second =
215         timings.decode_ms > 0.0
216         ? static_cast<double>(timings.decode_steps) * 1000.0 / timings.↵
↵ decode_ms
217         : 0.0;
218
219     std::cout << "benchmark_load_ms " << timings.load_ms << "\n";
220     std::cout << "benchmark_tokenize_ms " << timings.tokenize_ms << "\n";
221     std::cout << "benchmark_prefill_ms " << timings.prefill_ms << "\n";
222     std::cout << "benchmark_decode_ms " << timings.decode_ms << "\n";
223     std::cout << "benchmark_generate_ms " << timings.generate_ms << "\n";
224     std::cout << "benchmark_total_ms " << timings.total_ms << "\n";
225     std::cout << "benchmark_prompt_tokens " << prompt_tokens << "\n";
226     std::cout << "benchmark_generated_tokens " << generated_tokens << "\n";
227     std::cout << "benchmark_prefill_steps " << timings.prefill_steps << "\n";
228     std::cout << "benchmark_decode_steps " << timings.decode_steps << "\n";
229     std::cout << "benchmark_generate_tokens_per_second "
230         << generated_tokens_per_second << "\n";
231     std::cout << "benchmark_decode_tokens_per_second "

```

```

232         << decode_tokens_per_second << "\n";
233     std::cout << "benchmark_kv_cache_bytes " << timings.kv_cache_bytes << "\n";
234 }
235
236 int run_tiny_mode() {
237     const lcqi::Gpt2ReferenceModel model = lcqi::make_tiny_gpt2_reference_model(
238         ↵ );
239     const std::vector<std::int32_t> prompt{
240         LCQI_GPT2_TINY_PROMPT_0,
241         LCQI_GPT2_TINY_PROMPT_1,
242         LCQI_GPT2_TINY_PROMPT_2,
243     };
244     const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(model, prompt(
245         ↵ );
246     const std::vector<std::int32_t> generated =
247         lcqi::gpt2_generate_greedy(model, prompt, 2);
248     std::cout << "mode tiny\n";
249     std::cout << "prompt_ids ";
250     print_ids(prompt);
251     std::cout << "\n";
252     std::cout << "predicted_token " << result.predicted_token << "\n";
253     std::cout << "logits";
254     for (const float value : result.logits) {
255         std::cout << ' ' << value;
256     }
257     std::cout << "\n";
258     std::cout << "generated_ids ";
259     print_ids(generated);
260     std::cout << "\n";
261     return EXIT_SUCCESS;
262 }
263
264 int run_real_model_mode(int argc, char** argv) {
265     if (argc < LCQI_GPT2_MIN_REAL_ARGC) {
266         throw std::runtime_error(
267             "usage: lcqi_gpt2 [--engine cached|full] [--benchmark] "
268             "model_dir prompt [max_new_tokens]");
269     }
270     const Clock::time_point total_start = Clock::now();
271     const Gpt2CliOptions options = parse_options(argc, argv);
272
273     Gpt2BenchmarkTimings timings;
274     const Clock::time_point load_start = Clock::now();
275     const lcqi::Gpt2ReferenceModel model = lcqi::load_gpt2_from_directory(
276         ↵ options.model_dir);
277     const lcqi::Gpt2Tokenizer tokenizer = lcqi::load_gpt2_tokenizer(
278         options.model_dir / "vocab.json",
279         options.model_dir / "merges.txt",
280         model.config.bos_token_id,
281         model.config.eos_token_id);
282     timings.load_ms = elapsed_ms(load_start, Clock::now());

```



```

281  const Clock::time_point tokenize_start = Clock::now();
282  const std::vector<std::int32_t> prompt_ids = lcqi::gpt2_encode(tokenizer, ↵
↵ options.prompt);
283  timings.tokenize_ms = elapsed_ms(tokenize_start, Clock::now());
284
285  std::vector<std::int32_t> generated;
286  if (options.engine == Gpt2EngineMode::full_prefix) {
287      generated = generate_full_prefix(model, prompt_ids, options.↵
↵ max_new_tokens, timings);
288  } else {
289      generated = generate_cached(model, prompt_ids, options.max_new_tokens, ↵
↵ timings);
290  }
291  timings.total_ms = elapsed_ms(total_start, Clock::now());
292
293  std::cout << "mode gpt2_directory\n";
294  std::cout << "engine " << engine_name(options.engine) << "\n";
295  std::cout << "model_dir " << options.model_dir.string() << "\n";
296  std::cout << "prompt_ids ";
297  print_ids(prompt_ids);
298  std::cout << "\n";
299  std::cout << "generated_ids ";
300  print_ids(generated);
301  std::cout << "\n";
302  std::cout << "generated_text " << lcqi::gpt2_decode(tokenizer, generated) ↵
↵ << "\n";
303  if (options.benchmark) {
304      const std::size_t generated_tokens = generated.size() - prompt_ids.size()↵
↵ ();
305      print_benchmark(timings, prompt_ids.size(), generated_tokens);
306  }
307  return EXIT_SUCCESS;
308 }
309
310 } // namespace
311
312 int main(int argc, char** argv) {
313     try {
314         if (argc == LCQI_GPT2_TINY_ARGC) {
315             return run_tiny_mode();
316         }
317         return run_real_model_mode(argc, argv);
318     } catch (const std::exception& error) {
319         std::cerr << "error: " << error.what() << "\n";
320         return EXIT_FAILURE;
321     }
322 }

```

第 5 章

工具、报告、Benchmark 与回归测试

5.1 工具不是附属品

LCQI 的工程证据不只在 C++ 源码里。smoke 脚本决定真实模型怎样下载、校验和运行；benchmark 程序决定性能口径；trace、ledger 和 serving probe 决定报告字段；CTest 决定哪些行为能长期回归。读者必须把这些文件当作工程合同的一部分。

5.2 Benchmark、Trace、Ledger 与 Serving Probe

bench.cpp 输出 int8 kernel benchmark。它服务性能观察，但必须和 max diff 对照一起看。

Listing 5.1 src/bench.cpp

```
1 #include <lcqi/int8_kernels.hpp>
2
3 #include <algorithm>
4 #include <cstdlib>
5 #include <iostream>
6 #include <stdexcept>
7 #include <string>
8 #include <vector>
9
10 namespace {
11
12 constexpr const char* LCQI_TARGET_ARCH_X86_64 = "x86_64";
13 constexpr const char* LCQI_TARGET_ARCH_I386 = "i386";
14 constexpr const char* LCQI_TARGET_ARCH_UNKNOWN = "unknown";
15 constexpr std::int32_t LCQI_DEFAULT_REPEAT = 200;
16 constexpr int LCQI_REPEAT_ARG_INDEX = 1;
17 constexpr std::int32_t LCQI_LARGE_CASE_REPEAT_DIVISOR = 4;
18 constexpr std::int32_t LCQI_MIN_LARGE_CASE_REPEAT = 1;
19 constexpr std::int32_t LCQI_DECODE_SHAPE_SIZE = 4096;
20
21 const char* target_arch() noexcept {
22     #if defined(__x86_64__) || defined(_M_X64)
23         return LCQI_TARGET_ARCH_X86_64;
24     #elif defined(__i386__) || defined(_M_IX86)
25         return LCQI_TARGET_ARCH_I386;
26     #else
27         return LCQI_TARGET_ARCH_UNKNOWN;
28     #endif
29 }
30
31 std::int32_t parse_repeat(int argc, char** argv) {
32     if (argc <= LCQI_REPEAT_ARG_INDEX) {
33         return LCQI_DEFAULT_REPEAT;
34     }
35     const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[↵
```

```

    ↪ LCQI_REPEAT_ARG_INDEX]));
36  if (repeat <= 0) {
37      throw std::runtime_error("repeat must be positive");
38  }
39  return repeat;
40 }
41
42 std::vector<lcqi::KernelBenchmarkCase> make_cases(std::int32_t repeat) {
43     return {
44         {128, 128, 16, repeat},
45         {256, 256, 16, repeat},
46         {512, 512, 16, repeat},
47         {513, 257, 16, repeat},
48         {1024, 4096, 16, std::max(LCQI_MIN_LARGE_CASE_REPEAT,
49                                     repeat / LCQI_LARGE_CASE_REPEAT_DIVISOR)},
50         {LCQI_DECODE_SHAPE_SIZE, LCQI_DECODE_SHAPE_SIZE, 16,
51          std::max(LCQI_MIN_LARGE_CASE_REPEAT, repeat / ↪
52 ↪ LCQI_LARGE_CASE_REPEAT_DIVISOR)}},
53     };
54 }
55 } // namespace
56
57 int main(int argc, char** argv) {
58     try {
59         const std::int32_t repeat = parse_repeat(argc, argv);
60         std::cout
61             << "target_arch,input_size,output_size,output_block,repeat,backend,↪
62 ↪ available,"
63             << "average_us,max_abs_diff,checksum\n";
64         for (const lcqi::KernelBenchmarkCase& benchmark_case : make_cases(↪
65 ↪ repeat)) {
66             const lcqi::KernelBenchmarkResult result =
67                 lcqi::benchmark_linear_i8_case(benchmark_case);
68             for (const lcqi::KernelBackendBenchmark& backend : result.backends)↪
69 ↪ {
70                 std::cout << target_arch() << ','
71                     << result.benchmark_case.input_size << ','
72                     << result.benchmark_case.output_size << ','
73                     << result.benchmark_case.output_block_size << ','
74                     << result.benchmark_case.repeat << ','
75                     << backend.name << ','
76                     << (backend.available ? 1 : 0) << ','
77                     << backend.average_us << ','
78                     << backend.max_abs_diff << ','
79                     << backend.checksum << '\n';
80             }
81         }
82         return EXIT_SUCCESS;
83     } catch (const std::exception& error) {
84         std::cerr << "error: " << error.what() << '\n';
85         return EXIT_FAILURE;
86     }
87 }

```

```

83     }
84 }

```

`trace.cpp` 输出 reference decoder checkpoint。它用于定位错误边界，不应该被简化成只打印最终 token。

Listing 5.2 src/trace.cpp

```

1  #include <lcqi/reference_decoder.hpp>
2  #include <lcqi/tokenizer.hpp>
3
4  #include <algorithm>
5  #include <array>
6  #include <cstdlib>
7  #include <exception>
8  #include <filesystem>
9  #include <iostream>
10 #include <span>
11 #include <sstream>
12 #include <string>
13 #include <string_view>
14
15 namespace {
16
17 constexpr std::array<std::int32_t, 3> LCQI_TRACE_TOKEN_IDS{1, 2, 3};
18 constexpr std::string_view LCQI_TRACE_PROMPT = "tok1 tok2 tok3";
19 constexpr std::int32_t LCQI_TRACE_METADATA_LAYER = -1;
20 constexpr std::int32_t LCQI_TRACE_BOS_ID = 0;
21 constexpr std::int32_t LCQI_TRACE_EOS_ID = 4;
22 constexpr std::size_t LCQI_TRACE_VALUE_LIMIT = 8;
23
24 std::filesystem::path tokenizer_fixture_path() {
25     return std::filesystem::path(__FILE__).parent_path().parent_path() /
26         "models" / "tiny_tokenizer.txt";
27 }
28
29 std::vector<std::int32_t> decoder_tokens_from_prompt(const lcqi::TokenizerModel&
    ↪ & tokenizer,
30                                                     std::string_view prompt) {
31     std::vector<std::int32_t> full_tokens = lcqi::encode_prompt(tokenizer,
    ↪ prompt);
32     if (full_tokens.size() != LCQI_TRACE_TOKEN_IDS.size() + 2 ||
33         full_tokens.front() != LCQI_TRACE_BOS_ID ||
34         full_tokens.back() != LCQI_TRACE_EOS_ID) {
35         throw std::runtime_error("unexpected tokenizer fixture ids");
36     }
37
38     std::vector<std::int32_t> decoder_tokens(
39         full_tokens.begin() + 1,
40         full_tokens.end() - 1);
41     if (!std::equal(decoder_tokens.begin(),
42                    decoder_tokens.end(),
43                    LCQI_TRACE_TOKEN_IDS.begin(),

```

```

44         LCQI_TRACE_TOKEN_IDS.end())) {
45             throw std::runtime_error("unexpected decoder token ids");
46         }
47         return decoder_tokens;
48     }
49
50     std::string join_ints(std::span<const std::int32_t> values) {
51         std::ostringstream stream;
52         for (std::size_t index = 0; index < values.size(); ++index) {
53             if (index != 0) {
54                 stream << 'x';
55             }
56             stream << values[index];
57         }
58         return stream.str();
59     }
60
61     std::string join_int_values(std::span<const std::int32_t> values) {
62         std::ostringstream stream;
63         for (std::size_t index = 0; index < values.size(); ++index) {
64             if (index != 0) {
65                 stream << ';';
66             }
67             stream << values[index];
68         }
69         return stream.str();
70     }
71
72     std::int32_t checksum_ints(std::span<const std::int32_t> values) {
73         std::int32_t result = 0;
74         for (const std::int32_t value : values) {
75             result += value;
76         }
77         return result;
78     }
79
80     std::int32_t max_abs_ints(std::span<const std::int32_t> values) {
81         std::int32_t result = 0;
82         for (const std::int32_t value : values) {
83             result = std::max(result, std::abs(value));
84         }
85         return result;
86     }
87
88     std::string join_floats(std::span<const float> values) {
89         std::ostringstream stream;
90         const std::size_t count = std::min(values.size(), LCQI_TRACE_VALUE_LIMIT);
91         for (std::size_t index = 0; index < count; ++index) {
92             if (index != 0) {
93                 stream << ';';
94             }
95             stream << values[index];

```

```

96     }
97     if (values.size() > LCQI_TRACE_VALUE_LIMIT) {
98         stream << ";...";
99     }
100     return stream.str();
101 }
102
103 void print_trace_csv(const lcqi::ReferenceDecodeTraceResult& trace,
104                    const lcqi::TokenizerModel& tokenizer,
105                    std::span<const std::int32_t> full_tokens) {
106     std::cout << "name,token_position,layer_id,shape,stride,dtype,layout,↵
↵ checksum,max_abs,values\n";
107     std::cout << "prompt,0," << LCQI_TRACE_METADATA_LAYER
108         << ",scalar,scalar,utf8,text,0,0," << LCQI_TRACE_PROMPT << '\n';
109     std::cout << "tokenizer,0," << LCQI_TRACE_METADATA_LAYER
110         << ', ' << full_tokens.size() << ",1,i32,contiguous,"
111         << checksum_ints(full_tokens) << ', '
112         << max_abs_ints(full_tokens) << ', '
113         << join_int_values(full_tokens) << '\n';
114     std::cout << "tokenizer_contract,0," << LCQI_TRACE_METADATA_LAYER
115         << ",scalar,scalar,u64,fnv1a,"
116         << tokenizer_contract_hash(tokenizer) << ",0,"
117         << tokenizer.chat_template_hash << '\n';
118     for (const lcqi::ReferenceTensorCheckpoint& checkpoint : trace.checkpoints)↵
↵     {
119         std::cout << checkpoint.name << ', '
120             << checkpoint.token_position << ', '
121             << checkpoint.layer_id << ', '
122             << join_ints(checkpoint.shape) << ', '
123             << join_ints(checkpoint.stride) << ', '
124             << checkpoint.dtype << ', '
125             << checkpoint.layout << ', '
126             << checkpoint.checksum << ', '
127             << checkpoint.max_abs << ', '
128             << join_floats(checkpoint.values) << '\n';
129     }
130     std::cout << "sampler_argmax,"
131         << LCQI_TRACE_TOKEN_IDS.size() - 1 << ', '
132         << LCQI_TRACE_METADATA_LAYER
133         << ', ' << trace.result.logits.size() << ",1,f32,argmax,"
134         << "0,0,token=" << trace.result.predicted_token << '\n';
135     std::cout << "generated_token,"
136         << LCQI_TRACE_TOKEN_IDS.size() << ', '
137         << LCQI_TRACE_METADATA_LAYER
138         << ",scalar,scalar,i32,generated,"
139         << trace.result.predicted_token << ', '
140         << trace.result.predicted_token << ', '
141         << trace.result.predicted_token << '\n';
142 }
143
144 } // namespace
145

```

```

146 int main() {
147     try {
148         const lcqi::TokenizerModel tokenizer =
149             lcqi::load_tokenizer(tokenizer_fixture_path());
150         const std::vector<std::int32_t> full_tokens =
151             lcqi::encode_prompt(tokenizer, LCQI_TRACE_PROMPT);
152         const std::vector<std::int32_t> decoder_tokens =
153             decoder_tokens_from_prompt(tokenizer, LCQI_TRACE_PROMPT);
154         const lcqi::ReferenceDecoderModel model = lcqi::↵
↵ make_tiny_reference_decoder_model();
155         const lcqi::ReferenceDecodeTraceResult trace =
156             lcqi::run_reference_decode_with_trace(model, decoder_tokens);
157         print_trace_csv(trace, tokenizer, full_tokens);
158         return EXIT_SUCCESS;
159     } catch (const std::exception& error) {
160         std::cerr << "error: " << error.what() << '\n';
161         return EXIT_FAILURE;
162     }
163 }

```

`ledger.cpp` 输出 7B 规模账本。它是估算，不是实测吞吐；读者要区分 FLOPs、KV 容量、权重/KV 字节和 bandwidth-only tokens/s 上限。

Listing 5.3 src/ledger.cpp

```

1 #include <cstdlib>
2 #include <stdint>
3 #include <exception>
4 #include <iomanip>
5 #include <iostream>
6 #include <string_view>
7
8 namespace {
9
10 constexpr double LCQI_BYTES_PER_GB = 1000.0 * 1000.0 * 1000.0;
11 constexpr double LCQI_BYTES_PER_MIB = 1024.0 * 1024.0;
12 constexpr double LCQI_FLOPS_PER_MAC = 2.0;
13 constexpr std::int32_t LCQI_SEVENB_LAYER_COUNT = 32;
14 constexpr std::int32_t LCQI_SEVENB_HIDDEN_SIZE = 4096;
15 constexpr std::int32_t LCQI_SEVENB_QUERY_HEADS = 32;
16 constexpr std::int32_t LCQI_SEVENB_KV_HEADS = 8;
17 constexpr std::int32_t LCQI_SEVENB_HEAD_DIM = 128;
18 constexpr std::int32_t LCQI_SEVENB_INTERMEDIATE_SIZE = 11008;
19 constexpr std::int32_t LCQI_SEVENB_CONTEXT_LENGTH = 4096;
20
21 struct DTypeLedger {
22     std::string_view name;
23     double weight_gb_per_token = 0.0;
24     double kv_element_bytes = 0.0;
25 };
26
27 constexpr DTypeLedger LCQI_DTYPE_LEDGERS[] = {
28     {"fp16", 14.0, 2.0},

```

```

29     {"int8", 7.0, 2.0},
30     {"int4", 3.7, 2.0},
31 };
32
33 constexpr double LCQI_BANDWIDTH_GB_PER_S[] = {
34     50.0,
35     100.0,
36     200.0,
37     600.0,
38     1000.0,
39     1600.0,
40 };
41
42 double linear_macs_per_layer() noexcept {
43     const double hidden = LCQI_SEVENB_HIDDEN_SIZE;
44     const double kv_width = LCQI_SEVENB_KV_HEADS * LCQI_SEVENB_HEAD_DIM;
45     const double qkv = hidden * (hidden + 2.0 * kv_width);
46     const double output_projection = hidden * hidden;
47     const double mlp = 3.0 * hidden * LCQI_SEVENB_INTERMEDIATE_SIZE;
48     return qkv + output_projection + mlp;
49 }
50
51 double attention_macs_per_layer() noexcept {
52     return 2.0 * LCQI_SEVENB_QUERY_HEADS * LCQI_SEVENB_CONTEXT_LENGTH *
53           LCQI_SEVENB_HEAD_DIM;
54 }
55
56 double kv_bytes_per_token_all_layers(double element_bytes) noexcept {
57     return static_cast<double>(LCQI_SEVENB_LAYER_COUNT) * 2.0 *
58           LCQI_SEVENB_KV_HEADS * LCQI_SEVENB_HEAD_DIM * element_bytes;
59 }
60
61 double kv_read_gb_per_decode_token(double element_bytes) noexcept {
62     const double bytes = static_cast<double>(LCQI_SEVENB_CONTEXT_LENGTH) *
63           kv_bytes_per_token_all_layers(element_bytes);
64     return bytes / LCQI_BYTES_PER_GB;
65 }
66
67 double kv_capacity_mib(double element_bytes, std::int32_t context_length) ↵
68     ↵ noexcept {
69     const double bytes = static_cast<double>(context_length) *
70           kv_bytes_per_token_all_layers(element_bytes);
71     return bytes / LCQI_BYTES_PER_MIB;
72 }
73
74 void print_model_rows() {
75     const double linear_macs = linear_macs_per_layer();
76     const double attention_macs = attention_macs_per_layer();
77     const double total_flops =
78         (linear_macs + attention_macs) * LCQI_SEVENB_LAYER_COUNT * ↵
79         ↵ LCQI_FLOPS_PER_MAC;
80     std::cout << "section,item,value,unit,notes\n";

```



```

79     std::cout << "model,layers," << LCQI_SEVENB_LAYER_COUNT << ",count,7B ↵
↵ fixture\n";
80     std::cout << "model,hidden," << LCQI_SEVENB_HIDDEN_SIZE << ",elements,H\n";
81     std::cout << "model,query_heads," << LCQI_SEVENB_QUERY_HEADS << ",count,GQA↵
↵ query heads\n";
82     std::cout << "model,kv_heads," << LCQI_SEVENB_KV_HEADS << ",count,GQA KV ↵
↵ heads\n";
83     std::cout << "model,head_dim," << LCQI_SEVENB_HEAD_DIM << ",elements,per ↵
↵ head\n";
84     std::cout << "compute,linear_macs_per_layer," << linear_macs << ",macs,↵
↵ decode token\n";
85     std::cout << "compute,attention_macs_per_layer," << attention_macs
86         << ",macs,context=4096\n";
87     std::cout << "compute,total_decode_flops_per_token," << total_flops
88         << ",flops,linear plus attention\n";
89 }
90
91 void print_kv_rows() {
92     for (const std::int32_t context : {1024, 2048, 4096, 8192}) {
93         std::cout << "kv,fp16_capacity_context_" << context << ','
94             << kv_capacity_mib(2.0, context)
95             << ",MiB,single session\n";
96     }
97 }
98
99 void print_dtype_rows() {
100     for (const DTypeLedger& dtype : LCQI_DTYPE_LEDGERS) {
101         const double kv_read_gb = kv_read_gb_per_decode_token(dtype.↵
↵ kv_element_bytes);
102         const double total_gb = dtype.weight_gb_per_token + kv_read_gb;
103         std::cout << "decode_bytes," << dtype.name << "_weight_gb,"
104             << dtype.weight_gb_per_token << ",GB/token,weight stream\n";
105         std::cout << "decode_bytes," << dtype.name << "_kv_read_gb,"
106             << kv_read_gb << ",GB/token,context=4096\n";
107         std::cout << "decode_bytes," << dtype.name << "_total_gb,"
108             << total_gb << ",GB/token,weight plus KV\n";
109         for (const double bandwidth : LCQI_BANDWIDTH_GB_PER_S) {
110             std::cout << "bandwidth_limit," << dtype.name << "_at_"
111                 << static_cast<std::int32_t>(bandwidth) << "_gbps,"
112                 << bandwidth / total_gb
113                 << ",tokens/s,bandwidth-only upper bound\n";
114         }
115     }
116 }
117
118 } // namespace
119
120 int main() {
121     try {
122         std::cout << std::fixed << std::setprecision(3);
123         print_model_rows();
124         print_kv_rows();

```

```

125     print_dtype_rows();
126     return EXIT_SUCCESS;
127 } catch (const std::exception& error) {
128     std::cerr << "error: " << error.what() << '\n';
129     return EXIT_FAILURE;
130 }
131 }

```

`serving_probe.cpp` 固定短请求、RAG 请求、取消请求和超长请求。它把 TTFT、TPOT、拒绝和取消回收变成可观察字段。

Listing 5.4 src/serving_probe.cpp

```

1  #include <algorithm>
2  #include <cstdlib>
3  #include <stdint>
4  #include <exception>
5  #include <cmath>
6  #include <iomanip>
7  #include <iostream>
8  #include <string_view>
9  #include <vector>
10
11 namespace {
12
13 constexpr double LCQI_BYTES_PER_MIB = 1024.0 * 1024.0;
14 constexpr std::int32_t LCQI_PROBE_LAYER_COUNT = 32;
15 constexpr std::int32_t LCQI_PROBE_KV_HEADS = 8;
16 constexpr std::int32_t LCQI_PROBE_HEAD_DIM = 128;
17 constexpr double LCQI_PROBE_KV_ELEMENT_BYTES = 2.0;
18 constexpr double LCQI_PROBE_KV_BUDGET_MIB = 512.0;
19 constexpr double LCQI_PROBE_WORKSPACE_MIB = 64.0;
20 constexpr double LCQI_PROBE_ARENA_MIB = 32.0;
21 constexpr double LCQI_PROBE_TRACE_MIB = 4.0;
22 constexpr double LCQI_PROBE_QUEUE_WAIT_PER_REQUEST_MS = 8.0;
23 constexpr double LCQI_PROBE_PREFILL_MS_PER_TOKEN = 0.08;
24 constexpr double LCQI_PROBE_DECODE_STEP_MS = 14.0;
25 constexpr double LCQI_PROBE_CANCEL_RECLAIM_MS = 2.0;
26 constexpr std::size_t LCQI_PERCENTILE_MIN_INDEX = 1;
27
28 struct ProbeRequest {
29     std::string_view request_id;
30     std::int32_t prompt_tokens = 0;
31     std::int32_t max_new_tokens = 0;
32     bool cancel_after_prefill = false;
33 };
34
35 struct ProbeResult {
36     std::string_view request_id;
37     std::int32_t accepted = 0;
38     std::string_view rejection_reason;
39     std::int32_t prompt_tokens = 0;
40     std::int32_t reserved_tokens = 0;

```

```

41     double kv_reserved_mib = 0.0;
42     double queue_wait_ms = 0.0;
43     double prefill_ms = 0.0;
44     double ttft_ms = 0.0;
45     double decode_step_p95_ms = 0.0;
46     double tpot_ms = 0.0;
47     std::int32_t completion_tokens = 0;
48     std::string_view finish_reason;
49     double cancel_reclaim_ms = 0.0;
50 };
51
52 const std::vector<ProbeRequest> LCQI_PROBE_REQUESTS{
53     {"req_short", 32, 4, false},
54     {"req_rag", 512, 8, false},
55     {"req_cancel", 128, 16, true},
56     {"req_too_long", 4096, 512, false},
57 };
58
59 double kv_mib_for_tokens(std::int32_t tokens) noexcept {
60     const double bytes = static_cast<double>(tokens) * LCQI_PROBE_LAYER_COUNT *
    ↪ 2.0 *
61         LCQI_PROBE_KV_HEADS * LCQI_PROBE_HEAD_DIM *
62         LCQI_PROBE_KV_ELEMENT_BYTES;
63     return bytes / LCQI_BYTES_PER_MIB;
64 }
65
66 double percentile(std::vector<double> values, double fraction) {
67     if (values.empty()) {
68         return 0.0;
69     }
70     std::sort(values.begin(), values.end());
71     const std::size_t max_index = values.size() - LCQI_PERCENTILE_MIN_INDEX;
72     const std::size_t index =
73         std::min(max_index,
74             static_cast<std::size_t>(
75                 std::ceil(fraction * static_cast<double>(max_index))));
76     return values[index];
77 }
78
79 std::vector<ProbeResult> run_probe() {
80     std::vector<ProbeResult> results;
81     double reserved_kv_mib = 0.0;
82     std::int32_t accepted_before = 0;
83     for (const ProbeRequest& request : LCQI_PROBE_REQUESTS) {
84         ProbeResult result;
85         result.request_id = request.request_id;
86         result.prompt_tokens = request.prompt_tokens;
87         result.reserved_tokens = request.prompt_tokens + request.max_new_tokens
    ↪ ;
88         result.kv_reserved_mib = kv_mib_for_tokens(result.reserved_tokens);
89         const double static_session_mib =
90             result.kv_reserved_mib + LCQI_PROBE_WORKSPACE_MIB +

```

```

91         LCQI_PROBE_ARENA_MIB + LCQI_PROBE_TRACE_MIB;
92         if (reserved_kv_mib + result.kv_reserved_mib > LCQI_PROBE_KV_BUDGET_MIB) {
93             result.accepted = 0;
94             result.rejection_reason = "memory_budget_exceeded";
95             result.finish_reason = "rejected";
96             results.push_back(result);
97             continue;
98         }
99
100         reserved_kv_mib += result.kv_reserved_mib;
101         result.accepted = 1;
102         result.rejection_reason = "none";
103         result.queue_wait_ms = static_cast<double>(accepted_before) *
104             LCQI_PROBE_QUEUE_WAIT_PER_REQUEST_MS;
105         result.prefill_ms = static_cast<double>(request.prompt_tokens) *
106             LCQI_PROBE_PREFILL_MS_PER_TOKEN;
107         result.ttft_ms = result.queue_wait_ms + result.prefill_ms +
108             LCQI_PROBE_DECODE_STEP_MS;
109         result.decode_step_p95_ms = LCQI_PROBE_DECODE_STEP_MS +
110             static_session_mib / 512.0;
111         result.tpot_ms = result.decode_step_p95_ms;
112         if (request.cancel_after_prefill) {
113             result.completion_tokens = 0;
114             result.finish_reason = "cancelled";
115             result.cancel_reclaim_ms = LCQI_PROBE_CANCEL_RECLAIM_MS;
116             reserved_kv_mib -= result.kv_reserved_mib;
117         } else {
118             result.completion_tokens = request.max_new_tokens;
119             result.finish_reason = "max_tokens";
120         }
121         ++accepted_before;
122         results.push_back(result);
123     }
124     return results;
125 }
126
127 void print_request_rows(const std::vector<ProbeResult>& results) {
128     std::cout << "row_type,request_id,accepted,rejection_reason,prompt_tokens,
129     reserved_tokens,
130     kv_reserved_mib,queue_wait_ms,prefill_ms,ttft_ms,
131     decode_step_p95_ms,
132     tpot_ms,completion_tokens,finish_reason,cancel_reclaim_ms,
133     metric_name,
134     metric_value\n";
135     for (const ProbeResult& result : results) {
136         std::cout << "request," << result.request_id << ','
137             << result.accepted << ','
138             << result.rejection_reason << ','
139             << result.prompt_tokens << ','
140             << result.reserved_tokens << ','
141             << result.kv_reserved_mib << ','

```

```

138         << result.queue_wait_ms << ', '
139         << result.prefill_ms << ', '
140         << result.ttft_ms << ', '
141         << result.decode_step_p95_ms << ', '
142         << result.tpot_ms << ', '
143         << result.completion_tokens << ', '
144         << result.finish_reason << ', '
145         << result.cancel_reclaim_ms << ",,\n";
146     }
147 }
148
149 void print_summary_row(const std::vector<ProbeResult>& results) {
150     std::vector<double> ttft_values;
151     std::vector<double> queue_values;
152     double accepted = 0.0;
153     double rejected = 0.0;
154     double cancelled = 0.0;
155     double reserved_peak_mib = 0.0;
156     double running_reserved_mib = 0.0;
157     for (const ProbeResult& result : results) {
158         if (result.accepted == 0) {
159             rejected += 1.0;
160             continue;
161         }
162         accepted += 1.0;
163         ttft_values.push_back(result.ttft_ms);
164         queue_values.push_back(result.queue_wait_ms);
165         running_reserved_mib += result.kv_reserved_mib;
166         reserved_peak_mib = std::max(reserved_peak_mib, running_reserved_mib);
167         if (result.finish_reason == "cancelled") {
168             cancelled += 1.0;
169             running_reserved_mib -= result.kv_reserved_mib;
170         }
171     }
172     const double total = accepted + rejected;
173     const double rejection_rate = total == 0.0 ? 0.0 : rejected / total;
174     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
175         << "accepted_count," << accepted << '\n';
176     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,"
177         << LCQI_PROBE_CANCEL_RECLAIM_MS << ",cancel_count," << cancelled <<
178         << '\n';
179     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
180         << "rejected_count," << rejected << '\n';
181     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
182         << "rejection_rate," << rejection_rate << '\n';
183     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
184         << "kv_reserved_peak_mib," << reserved_peak_mib << '\n';
185     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
186         << "queue_wait_p95_ms," << percentile(queue_values, 0.95) << '\n' <<
187         ;
188     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
189         << "ttft_p95_ms," << percentile(ttft_values, 0.95) << '\n';

```

```

188 }
189
190 } // namespace
191
192 int main() {
193     try {
194         std::cout << std::fixed << std::setprecision(3);
195         const std::vector<ProbeResult> results = run_probe();
196         print_request_rows(results);
197         print_summary_row(results);
198         return EXIT_SUCCESS;
199     } catch (const std::exception& error) {
200         std::cerr << "error: " << error.what() << '\n';
201         return EXIT_FAILURE;
202     }
203 }

```

`onednn_bench.cpp` 是可选外部库对标入口。它只在环境安装 oneDNN 时构建，不能把不可用平台写成已验证性能。

Listing 5.5 src/onednn_bench.cpp

```

1  #include <dnnl.hpp>
2
3  #include <chrono>
4  #include <cstdint>
5  #include <cstdlib>
6  #include <iostream>
7  #include <numeric>
8  #include <stdexcept>
9  #include <string>
10 #include <unordered_map>
11 #include <vector>
12
13 namespace {
14
15 constexpr std::int64_t LCQI_ONEDNN_BATCH = 1;
16 constexpr std::int64_t LCQI_ONEDNN_K = 4096;
17 constexpr std::int64_t LCQI_ONEDNN_O = 4096;
18 constexpr std::int32_t LCQI_ONEDNN_DEFAULT_REPEAT = 20;
19 constexpr std::int32_t LCQI_ONEDNN_WARMUP = 3;
20 constexpr int LCQI_REPEAT_ARG_INDEX = 1;
21 constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
22 constexpr float LCQI_INPUT_SCALE = 0.125F;
23 constexpr std::int32_t LCQI_INPUT_MODULUS = 17;
24 constexpr std::int32_t LCQI_INPUT_CENTER = 8;
25 constexpr std::int32_t LCQI_WEIGHT_MODULUS = 23;
26 constexpr std::int32_t LCQI_WEIGHT_CENTER = 11;
27 constexpr float LCQI_WEIGHT_SCALE = 0.03125F;
28
29 std::int32_t parse_repeat(int argc, char** argv) {
30     if (argc <= LCQI_REPEAT_ARG_INDEX) {
31         return LCQI_ONEDNN_DEFAULT_REPEAT;

```

```

32     }
33     const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[↵
↵ LCQI_REPEAT_ARG_INDEX]));
34     if (repeat <= 0) {
35         throw std::runtime_error("repeat must be positive");
36     }
37     return repeat;
38 }
39
40 std::vector<float> make_input() {
41     std::vector<float> input(static_cast<std::size_t>(LCQI_ONEDNN_K));
42     for (std::int64_t index = 0; index < LCQI_ONEDNN_K; ++index) {
43         input[static_cast<std::size_t>(index)] =
44             static_cast<float>((index % LCQI_INPUT_MODULUS) - LCQI_INPUT_CENTER↵
↵ ) *
45             LCQI_INPUT_SCALE;
46     }
47     return input;
48 }
49
50 std::vector<float> make_weights_kn() {
51     std::vector<float> weights(static_cast<std::size_t>(LCQI_ONEDNN_K * ↵
↵ LCQI_ONEDNN_0));
52     for (std::int64_t k = 0; k < LCQI_ONEDNN_K; ++k) {
53         for (std::int64_t out = 0; out < LCQI_ONEDNN_0; ++out) {
54             const std::int64_t row_major_out_k = out * LCQI_ONEDNN_K + k;
55             const float value =
56                 static_cast<float>((row_major_out_k % LCQI_WEIGHT_MODULUS) -
57                                     LCQI_WEIGHT_CENTER) *
58                 LCQI_WEIGHT_SCALE;
59             weights[static_cast<std::size_t>(k * LCQI_ONEDNN_0 + out)] = value;
60         }
61     }
62     return weights;
63 }
64
65 float checksum(const std::vector<float>& values) {
66     return std::accumulate(values.begin(), values.end(), 0.0F);
67 }
68
69 } // namespace
70
71 int main(int argc, char** argv) {
72     try {
73         const std::int32_t repeat = parse_repeat(argc, argv);
74         std::vector<float> input = make_input();
75         std::vector<float> weights = make_weights_kn();
76         std::vector<float> output(static_cast<std::size_t>(LCQI_ONEDNN_BATCH * ↵
↵ LCQI_ONEDNN_0),
77                                 0.0F);
78
79         dnnl::engine engine(dnnl::engine::kind::cpu, 0);

```

```

80     dnnl::stream stream(engine);
81
82     const dnnl::memory::dims source_dims = {LCQI_ONEDNN_BATCH, ↵
↵ LCQI_ONEDNN_K};
83     const dnnl::memory::dims weight_dims = {LCQI_ONEDNN_K, LCQI_ONEDNN_0};
84     const dnnl::memory::dims output_dims = {LCQI_ONEDNN_BATCH, ↵
↵ LCQI_ONEDNN_0};
85
86     const dnnl::memory::desc source_desc(
87         source_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag↵
↵ ::ab);
88     const dnnl::memory::desc weight_desc(
89         weight_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag↵
↵ ::ab);
90     const dnnl::memory::desc output_desc(
91         output_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag↵
↵ ::ab);
92
93     dnnl::memory source_memory(source_desc, engine, input.data());
94     dnnl::memory weight_memory(weight_desc, engine, weights.data());
95     dnnl::memory output_memory(output_desc, engine, output.data());
96
97     dnnl::matmul::primitive_desc matmul_desc(engine, source_desc, ↵
↵ weight_desc, output_desc);
98     dnnl::matmul matmul(matmul_desc);
99     const std::unordered_map<int, dnnl::memory> arguments = {
100         {DNNL_ARG_SRC, source_memory},
101         {DNNL_ARG_WEIGHTS, weight_memory},
102         {DNNL_ARG_DST, output_memory},
103     };
104
105     for (std::int32_t index = 0; index < LCQI_ONEDNN_WARMUP; ++index) {
106         matmul.execute(stream, arguments);
107         stream.wait();
108     }
109
110     const auto begin = std::chrono::steady_clock::now();
111     for (std::int32_t index = 0; index < repeat; ++index) {
112         matmul.execute(stream, arguments);
113         stream.wait();
114     }
115     const auto end = std::chrono::steady_clock::now();
116     const std::chrono::duration<double> elapsed = end - begin;
117     const double average_us =
118         elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double↵
↵ >(repeat);
119
120     std::cout << "library,input_size,output_size,dtype,repeat,average_us,↵
↵ checksum\n";
121     std::cout << "onednn," << LCQI_ONEDNN_K << ',' << LCQI_ONEDNN_0
122         << ",f32," << repeat << ',' << average_us << ','
123         << checksum(output) << '\n';

```



```

124     return EXIT_SUCCESS;
125 } catch (const std::exception& error) {
126     std::cerr << "error: " << error.what() << '\n';
127     return EXIT_FAILURE;
128 }
129 }

```

5.3 外部模型 Smoke 与对比脚本

`run_smollm2_small_smoke.py` 下载并校验 SmolLM2-135M-Instruct 的 GGUF 量化文件，构建固定 commit 的 `llama-simple-chat`，在 CPU 上运行一次真实开源 small 模型生成，并输出可复查报告。它是外部真实模型 smoke，不替代 LCQI tiny golden trace。

Listing 5.6 `tools/run_smollm2_small_smoke.py`

```

1  #!/usr/bin/env python3
2  from __future__ import annotations
3
4  import argparse
5  import datetime as dt
6  import hashlib
7  import os
8  import re
9  import shlex
10 import shutil
11 import subprocess
12 import sys
13 from dataclasses import dataclass
14 from pathlib import Path
15
16
17 MODEL_SOURCE_ID = "HuggingFaceTB/SmolLM2-135M-Instruct"
18 MODEL_GGUF_REPO_ID = "bartowski/SmolLM2-135M-Instruct-GGUF"
19 MODEL_FILE_NAME = "SmolLM2-135M-Instruct-Q4_K_M.gguf"
20 MODEL_URL = (
21     "https://huggingface.co/bartowski/SmolLM2-135M-Instruct-GGUF"
22     f"/resolve/main/{MODEL_FILE_NAME}"
23 )
24 MODEL_EXPECTED_BYTES = 105454432
25 MODEL_EXPECTED_SHA256 = (
26     "2e8040ceae7815abe0dcb3540b9995eaa1fa0d2ca9e797d0a635ae4433c68c2d"
27 )
28 LLAMA_CPP_REPO_URL = "https://github.com/ggml-org/llama.cpp.git"
29 LLAMA_CPP_COMMIT = "b3fed31b99f9bd37725833674252bccb429bb183"
30 LLAMA_TARGET = "llama-simple-chat"
31 DEFAULT_PROMPT = "What is 2+3? Answer in one short sentence."
32 DEFAULT_CONTEXT_TOKENS = 512
33 DEFAULT_TIMEOUT_SECONDS = 180
34 DEFAULT_BUILD_JOBS = 2
35
36
37 @dataclass(frozen=True)

```

```

38 class CommandResult:
39     args: list[str]
40     returncode: int
41     stdout: str
42     stderr: str
43
44
45 def script_path() -> Path:
46     return Path(__file__).resolve()
47
48
49 def volume_dir() -> Path:
50     return script_path().parents[1]
51
52
53 def repo_dir() -> Path:
54     return script_path().parents[3]
55
56
57 def default_cache_dir() -> Path:
58     explicit_cache = os.environ.get("LCQI_SMOLLM2_CACHE_DIR")
59     if explicit_cache:
60         return Path(explicit_cache).expanduser()
61     xdg_cache = os.environ.get("XDG_CACHE_HOME")
62     cache_root = Path(xdg_cache).expanduser() if xdg_cache else Path.home() / "↵
↵ .cache"
63     return cache_root / "lcqi-smolllm2-small-smoke"
64
65
66 def default_report_path() -> Path:
67     return volume_dir() / "results" / "lcqi-smolllm2-small-model-smoke.txt"
68
69
70 def command_line(args: list[str]) -> str:
71     return " ".join(shlex.quote(arg) for arg in args)
72
73
74 def require_tool(name: str) -> str:
75     path = shutil.which(name)
76     if path is None:
77         raise RuntimeError(f"required tool not found in PATH: {name}")
78     return path
79
80
81 def run_command(
82     args: list[str],
83     *,
84     cwd: Path | None = None,
85     input_text: str | None = None,
86     timeout_seconds: int | None = None,
87 ) -> CommandResult:
88     try:

```

```

89         completed = subprocess.run(
90             args,
91             cwd=str(cwd) if cwd else None,
92             input=input_text,
93             text=True,
94             capture_output=True,
95             timeout=timeout_seconds,
96             check=False,
97         )
98     except subprocess.TimeoutExpired as exc:
99         stdout = exc.stdout if isinstance(exc.stdout, str) else ""
100         stderr = exc.stderr if isinstance(exc.stderr, str) else ""
101         return CommandResult(args=args, returncode=124, stdout=stdout, stderr=↵
↵ stderr)
102
103     return CommandResult(
104         args=args,
105         returncode=completed.returncode,
106         stdout=completed.stdout,
107         stderr=completed.stderr,
108     )
109
110
111 def ensure_success(result: CommandResult, action: str) -> None:
112     if result.returncode == 0:
113         return
114     raise RuntimeError(
115         f"{action} failed with exit code {result.returncode}\n"
116         f"command: {command_line(result.args)}\n"
117         f"stdout tail:\n{tail(result.stdout)}\n"
118         f"stderr tail:\n{tail(result.stderr)}"
119     )
120
121
122 def tail(text: str, max_chars: int = 4000) -> str:
123     if len(text) <= max_chars:
124         return text
125     return text[-max_chars:]
126
127
128 def sha256_file(path: Path) -> str:
129     digest = hashlib.sha256()
130     with path.open("rb") as handle:
131         for block in iter(lambda: handle.read(1024 * 1024), b''):
132             digest.update(block)
133     return digest.hexdigest()
134
135
136 def verify_model_file(path: Path) -> tuple[bool, str, int]:
137     if not path.exists():
138         return False, "", 0
139     actual_bytes = path.stat().st_size

```

```

140     actual_sha = sha256_file(path)
141     return (
142         actual_bytes == MODEL_EXPECTED_BYTES
143         and actual_sha == MODEL_EXPECTED_SHA256,
144         actual_sha,
145         actual_bytes,
146     )
147
148
149 def download_model(model_path: Path, *, force_download: bool, no_download: bool ↵
    ↵ ) -> None:
150     model_path.parent.mkdir(parents=True, exist_ok=True)
151
152     if force_download and model_path.exists():
153         model_path.unlink()
154
155     is_valid, actual_sha, actual_bytes = verify_model_file(model_path)
156     if is_valid:
157         print(f"[lcqi-smoke] model cache hit: {model_path}")
158         return
159
160     if model_path.exists():
161         print(
162             "[lcqi-smoke] cached model hash mismatch, redownloading: "
163             f"bytes={actual_bytes}, sha256={actual_sha}"
164         )
165         model_path.unlink()
166
167     if no_download:
168         raise RuntimeError(
169             f"model file is missing or invalid and --no-download was set: {↵
    ↵ model_path}"
170         )
171
172     require_tool("curl")
173     part_path = model_path.with_suffix(model_path.suffix + ".part")
174     if part_path.exists():
175         part_path.unlink()
176
177     print(f"[lcqi-smoke] downloading small GGUF model: {MODEL_GGUF_REPO_ID}/{↵
    ↵ MODEL_FILE_NAME}")
178     result = run_command(
179         [
180             "curl",
181             "-L",
182             "--fail",
183             "--retry",
184             "3",
185             "-o",
186             str(part_path),
187             MODEL_URL,
188         ]

```

```

189     )
190     ensure_success(result, "download SmolLM2 GGUF")
191
192     downloaded_ok, downloaded_sha, downloaded_bytes = verify_model_file(↵
↵ part_path)
193     if not downloaded_ok:
194         raise RuntimeError(
195             "downloaded model did not match expected identity: "
196             f"bytes={downloaded_bytes}, sha256={downloaded_sha}"
197         )
198     part_path.replace(model_path)
199     print(f"[lcqi-smoke] model verified: sha256={MODEL_EXPECTED_SHA256}")
200
201
202 def ensure_llama_simple_chat(cache_dir: Path, *, jobs: int, skip_build: bool) ↵
↵ -> Path:
203     cached_binary = cache_dir / "llama.cpp" / "build" / "bin" / LLAMA_TARGET
204     source_dir = cache_dir / "llama.cpp"
205     build_dir = source_dir / "build"
206     if cached_binary.exists() and os.access(cached_binary, os.X_OK) and ↵
↵ is_pinned_llama_checkout(source_dir):
207         print(f"[lcqi-smoke] llama.cpp binary cache hit: {cached_binary}")
208         return cached_binary
209
210     if skip_build:
211         raise RuntimeError(f"llama.cpp binary is missing and --skip-build was ↵
↵ set: {cached_binary}")
212
213     require_tool("git")
214     require_tool("cmake")
215
216     cache_dir.mkdir(parents=True, exist_ok=True)
217
218     if not (source_dir / ".git").exists():
219         print(f"[lcqi-smoke] cloning llama.cpp into cache: {source_dir}")
220         result = run_command(
221             [
222                 "git",
223                 "clone",
224                 "--filter=blob:none",
225                 "--no-checkout",
226                 LLAMA_CPP_REPO_URL,
227                 str(source_dir),
228             ]
229         )
230         ensure_success(result, "clone llama.cpp")
231
232     print(f"[lcqi-smoke] checking out pinned llama.cpp commit: {↵
↵ LLAMA_CPP_COMMIT}")
233     fetch = run_command(
234         ["git", "-C", str(source_dir), "fetch", "--depth", "1", "origin", ↵
↵ LLAMA_CPP_COMMIT]

```

```

235 )
236 ensure_success(fetch, "fetch pinned llama.cpp commit")
237 checkout = run_command(["git", "-C", str(source_dir), "checkout", "--detach←
↪ ", LLAMA_CPP_COMMIT])
238 ensure_success(checkout, "checkout pinned llama.cpp commit")
239
240 print(f"[lcqi-smoke] configuring llama.cpp build: {build_dir}")
241 configure = run_command(
242     [
243         "cmake",
244         "-S",
245         str(source_dir),
246         "-B",
247         str(build_dir),
248         "-DCMAKE_BUILD_TYPE=Release",
249         "-DLLAMA_BUILD_TESTS=OFF",
250         "-DLLAMA_BUILD_EXAMPLES=ON",
251         "-DLLAMA_BUILD_SERVER=OFF",
252         "-DGGML_NATIVE=OFF",
253     ]
254 )
255 ensure_success(configure, "configure llama.cpp")
256
257 print(f"[lcqi-smoke] building {LLAMA_TARGET} with -j{jobs}")
258 build = run_command(
259     ["cmake", "--build", str(build_dir), "--target", LLAMA_TARGET, "-j", ←
↪ str(jobs)]
260 )
261 ensure_success(build, f"build {LLAMA_TARGET}")
262
263 if not cached_binary.exists():
264     raise RuntimeError(f"expected binary was not produced: {cached_binary}"←
↪ )
265 return cached_binary
266
267
268 def is_pinned_llama_checkout(source_dir: Path) -> bool:
269     if not (source_dir / ".git").exists():
270         return False
271     result = run_command(["git", "-C", str(source_dir), "rev-parse", "HEAD"])
272     if result.returncode != 0:
273         return False
274     return result.stdout.strip() == LLAMA_CPP_COMMIT
275
276
277 def strip_ansi(text: str) -> str:
278     return re.sub(r"\x1b\[([0-?]*[ -/]*[@-~])", "", text)
279
280
281 def extract_response(stdout: str) -> str:
282     cleaned = strip_ansi(stdout).replace("\r", "")
283     chunks = [chunk.strip() for chunk in re.split(r"(?:^|\n)> ?", cleaned) if ←

```

```

    ↪ chunk.strip()]
284     if not chunks:
285         return ""
286     return chunks[0]
287
288
289 def run_model_smoke(
290     llama_binary: Path,
291     model_path: Path,
292     *,
293     prompt: str,
294     context_tokens: int,
295     timeout_seconds: int,
296 ) -> tuple[CommandResult, str]:
297     command = [
298         str(llvm_binary),
299         "-m",
300         str(model_path),
301         "-ngl",
302         "0",
303         "-c",
304         str(context_tokens),
305     ]
306     print("[lcqi-smoke] running real small model generation")
307     result = run_command(
308         command,
309         input_text=prompt.rstrip("\n") + "\n\n",
310         timeout_seconds=timeout_seconds,
311     )
312     ensure_success(result, "run small model smoke")
313
314     response = extract_response(result.stdout)
315     if not response:
316         raise RuntimeError(
317             "small model smoke exited successfully but produced no assistant ↪
    ↪ text\n"
318             f"stdout:\n{result.stdout}\n"
319             f"stderr:\n{result.stderr}"
320         )
321     return result, response
322
323
324 def current_repo_commit() -> str:
325     result = run_command(["git", "-C", str(repo_dir()), "rev-parse", "--short", ↪
    ↪ "HEAD"])
326     if result.returncode != 0:
327         return "unknown"
328     return result.stdout.strip()
329
330
331 def current_uname() -> str:
332     result = run_command(["uname", "-a"])

```

```

333     if result.returncode != 0:
334         return "unknown"
335     return result.stdout.strip()
336
337
338 def write_report(
339     report_path: Path,
340     *,
341     cache_dir: Path,
342     llama_binary: Path,
343     model_path: Path,
344     prompt: str,
345     result: CommandResult,
346     response: str,
347 ) -> None:
348     report_path.parent.mkdir(parents=True, exist_ok=True)
349     clean_stdout = strip_ansi(result.stdout).strip()
350     report = "\n".join(
351         [
352             "LCQI SmoLLM2 Small Model Smoke",
353             "",
354             f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
355             f"repo_commit={current_repo_commit()}",
356             f"host={current_uname()}",
357             f"python={sys.version.split()[0]}",
358             f"cache_dir={cache_dir}",
359             "",
360             f"model_source_id={MODEL_SOURCE_ID}",
361             f"model_gguf_repo_id={MODEL_GGUF_REPO_ID}",
362             f"model_file={MODEL_FILE_NAME}",
363             f"model_url={MODEL_URL}",
364             f"model_expected_bytes={MODEL_EXPECTED_BYTES}",
365             f"model_expected_sha256={MODEL_EXPECTED_SHA256}",
366             f"model_path={model_path}",
367             "",
368             f"llama_cpp_repo={LLAMA_CPP_REPO_URL}",
369             f"llama_cpp_commit={LLAMA_CPP_COMMIT}",
370             f"llama_target={LLAMA_TARGET}",
371             f"llama_binary={llama_binary}",
372             "",
373             f"prompt={prompt}",
374             f"command={command_line(result.args)}",
375             f"exit_code={result.returncode}",
376             "",
377             "validation=PASS: model hash matched, llama-simple-chat exited 0, "
378             "assistant response was non-empty",
379             "",
380             "assistant_response:",
381             response,
382             "",
383             "stdout_clean_tail:",
384             tail(clean_stdout, 2000),

```



```

385         "",
386         "stderr_tail:",
387         tail(result.stderr.strip(), 2000),
388         "",
389     ]
390 )
391 report_path.write_text(report, encoding="utf-8")
392
393
394 def parse_args() -> argparse.Namespace:
395     parser = argparse.ArgumentParser(
396         description=(
397             "Run a real local small open-source model smoke for CPU Volume 3. "
398             "The script caches llama.cpp and the GGUF model outside the Git ↵
399             ↵ repository."
400         )
401     )
402     parser.add_argument("--cache-dir", type=Path, default=default_cache_dir())
403     parser.add_argument("--model-path", type=Path, default=None)
404     parser.add_argument("--llama-bin", type=Path, default=None)
405     parser.add_argument("--report", type=Path, default=default_report_path())
406     parser.add_argument("--prompt", default=DEFAULT_PROMPT)
407     parser.add_argument("--ctx", type=int, default=DEFAULT_CONTEXT_TOKENS)
408     parser.add_argument("--timeout", type=int, default=DEFAULT_TIMEOUT_SECONDS)
409     parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
410     parser.add_argument("--force-download", action="store_true")
411     parser.add_argument("--no-download", action="store_true")
412     parser.add_argument("--skip-build", action="store_true")
413     return parser.parse_args()
414
415
416 def main() -> int:
417     args = parse_args()
418     cache_dir = args.cache_dir.expanduser().resolve()
419     model_path = (
420         args.model_path.expanduser().resolve()
421         if args.model_path
422         else cache_dir / "models" / MODEL_FILE_NAME
423     )
424     report_path = args.report.expanduser().resolve()
425
426     try:
427         download_model(
428             model_path,
429             force_download=args.force_download,
430             no_download=args.no_download,
431         )
432     if args.llama_bin:
433         llama_binary = args.llama_bin.expanduser().resolve()
434         if not llama_binary.exists() or not os.access(llama_binary, os.X_OK↵
435         ↵ ):
436             raise RuntimeError(f"--llama-bin is not executable: {↵

```

```

↪ llama_binary}")
435     else:
436         llama_binary = ensure_llama_simple_chat(
437             cache_dir,
438             jobs=args.jobs,
439             skip_build=args.skip_build,
440         )
441
442     result, response = run_model_smoke(
443         llama_binary,
444         model_path,
445         prompt=args.prompt,
446         context_tokens=args.ctx,
447         timeout_seconds=args.timeout,
448     )
449     write_report(
450         report_path,
451         cache_dir=cache_dir,
452         llama_binary=llama_binary,
453         model_path=model_path,
454         prompt=args.prompt,
455         result=result,
456         response=response,
457     )
458     except RuntimeError as exc:
459         print(f"[lcqi-smoke] ERROR: {exc}", file=sys.stderr)
460         return 1
461
462     print(f"report={report_path}")
463     print(f"model_sha256={MODEL_EXPECTED_SHA256}")
464     print(f"assistant_response={response}")
465     return 0
466
467
468 if __name__ == "__main__":
469     raise SystemExit(main())

```

`run_gpt2_smoke.py` 下载标准 GPT-2 的 `config.json`、`vocab.json`、`merges.txt` 和 `model.safetensors`，再调用 LCQI 自研 `lcqi_gpt2` reference path 生成 token。它证明自研 safetensors、BPE、GPT-2 forward、KV cache decode 和 greedy generation 已连成闭环。

Listing 5.7 tools/run_gpt2_smoke.py

```

1  #!/usr/bin/env python3
2  from __future__ import annotations
3
4  import argparse
5  import datetime as dt
6  import hashlib
7  import os
8  import shlex
9  import subprocess
10 import sys

```

```

11 import urllib.request
12 from dataclasses import dataclass
13 from pathlib import Path
14
15
16 MODEL_REPO_ID = "openai-community/gpt2"
17 MODEL_REVISION = "main"
18 MODEL_FILE_IDENTITIES = {
19     "config.json": (
20         665,
21         "0daed7749b4f02b8f76240d5444551d7b08712dab4d0adb8239c56ba823bb7b4",
22     ),
23     "vocab.json": (
24         1042301,
25         "196139668be63f3b5d6574427317ae82f612a97c5d1cdaf36ed2256dbf636783",
26     ),
27     "merges.txt": (
28         456318,
29         "1ce1664773c50f3e0cc8842619a93edc4624525b728b188a9e0be33b7726adc5",
30     ),
31     "model.safetensors": (
32         548105171,
33         "248dfc3911869ec493c76e65bf2fcf7f615828b0254c12b473182f0f81d3a707",
34     ),
35 }
36 DEFAULT_PROMPT = "Hello, my name is"
37 DEFAULT_MAX_NEW_TOKENS = 1
38 DEFAULT_TIMEOUT_SECONDS = 300
39 DEFAULT_BUILD_JOBS = 2
40
41
42 @dataclass(frozen=True)
43 class CommandResult:
44     args: list[str]
45     returncode: int
46     stdout: str
47     stderr: str
48
49
50 def script_path() -> Path:
51     return Path(__file__).resolve()
52
53
54 def volume_dir() -> Path:
55     return script_path().parents[1]
56
57
58 def repo_dir() -> Path:
59     return script_path().parents[3]
60
61
62 def default_cache_dir() -> Path:

```

```

63     explicit_cache = os.environ.get("LCQI_GPT2_CACHE_DIR")
64     if explicit_cache:
65         return Path(explicit_cache).expanduser()
66     xdg_cache = os.environ.get("XDG_CACHE_HOME")
67     cache_root = Path(xdg_cache).expanduser() if xdg_cache else Path.home() / "←
    ↪ .cache"
68     return cache_root / "lcqi-gpt2-smoke" / MODEL_REPO_ID.replace("/", "--")
69
70
71 def default_build_dir() -> Path:
72     return volume_dir() / "build" / "lcqi-release"
73
74
75 def default_report_path() -> Path:
76     return volume_dir() / "results" / "lcqi-gpt2-smoke.txt"
77
78
79 def command_line(args: list[str]) -> str:
80     return " ".join(shlex.quote(arg) for arg in args)
81
82
83 def run_command(
84     args: list[str],
85     *,
86     cwd: Path | None = None,
87     timeout_seconds: int | None = None,
88 ) -> CommandResult:
89     try:
90         completed = subprocess.run(
91             args,
92             cwd=str(cwd) if cwd else None,
93             text=True,
94             capture_output=True,
95             timeout=timeout_seconds,
96             check=False,
97         )
98     except subprocess.TimeoutExpired as exc:
99         stdout = exc.stdout if isinstance(exc.stdout, str) else ""
100         stderr = exc.stderr if isinstance(exc.stderr, str) else ""
101         return CommandResult(args=args, returncode=124, stdout=stdout, stderr=←
    ↪ stderr)
102     return CommandResult(
103         args=args,
104         returncode=completed.returncode,
105         stdout=completed.stdout,
106         stderr=completed.stderr,
107     )
108
109
110 def ensure_success(result: CommandResult, action: str) -> None:
111     if result.returncode == 0:
112         return

```

```

113     raise RuntimeError(
114         f"{action} failed with exit code {result.returncode}\n"
115         f"command: {command_line(result.args)}\n"
116         f"stdout tail:\n{tail(result.stdout)}\n"
117         f"stderr tail:\n{tail(result.stderr)}"
118     )
119
120
121 def tail(text: str, max_chars: int = 4000) -> str:
122     if len(text) <= max_chars:
123         return text
124     return text[-max_chars:]
125
126
127 def sha256_file(path: Path) -> str:
128     digest = hashlib.sha256()
129     with path.open("rb") as handle:
130         for block in iter(lambda: handle.read(1024 * 1024), b''):
131             digest.update(block)
132     return digest.hexdigest()
133
134
135 def model_url(file_name: str) -> str:
136     return (
137         f"https://huggingface.co/{MODEL_REPO_ID}/resolve/"
138         f"{MODEL_REVISION}/{file_name}"
139     )
140
141
142 def download_file(url: str, target: Path, timeout_seconds: int) -> None:
143     part_path = target.with_suffix(target.suffix + ".part")
144     if part_path.exists():
145         part_path.unlink()
146     request = urllib.request.Request(url, headers={"User-Agent": "lcqi-gpt2-↵
↵ smoke"})
147     with urllib.request.urlopen(request, timeout=timeout_seconds) as response:
148         with part_path.open("wb") as output:
149             while True:
150                 block = response.read(1024 * 1024)
151                 if not block:
152                     break
153                 output.write(block)
154     part_path.replace(target)
155
156
157 def file_identity_matches(path: Path, expected_bytes: int, expected_sha256: str↵
↵ ) -> bool:
158     return (
159         path.exists()
160         and path.stat().st_size == expected_bytes
161         and sha256_file(path) == expected_sha256
162     )

```

```

163
164
165 def ensure_model_files(cache_dir: Path, *, no_download: bool, timeout_seconds: ←
    ↪ int) -> None:
166     cache_dir.mkdir(parents=True, exist_ok=True)
167     missing = [
168         name
169         for name, (expected_bytes, expected_sha256) in MODEL_FILE_IDENTITIES.←
    ↪ items()
170         if not file_identity_matches(cache_dir / name, expected_bytes, ←
    ↪ expected_sha256)
171     ]
172     if not missing:
173         print(f"[lcqi-gpt2] model cache hit: {cache_dir}")
174         return
175     if no_download:
176         raise RuntimeError(
177             f"missing GPT-2 model files and --no-download was set: {missing}"
178         )
179     for file_name in missing:
180         target = cache_dir / file_name
181         if target.exists():
182             target.unlink()
183         print(f"[lcqi-gpt2] downloading {MODEL_REPO_ID}/{file_name}")
184         download_file(model_url(file_name), target, timeout_seconds)
185         expected_bytes, expected_sha256 = MODEL_FILE_IDENTITIES[file_name]
186         if not file_identity_matches(target, expected_bytes, expected_sha256):
187             raise RuntimeError(
188                 f"downloaded GPT-2 file identity mismatch: {file_name}"
189             )
190
191
192 def build_lcqi_gpt2(build_dir: Path, jobs: int) -> Path:
193     configure = run_command(
194         [
195             "cmake",
196             "-S",
197             str(volume_dir()),
198             "-B",
199             str(build_dir),
200             "-DCMAKE_BUILD_TYPE=Release",
201         ],
202         timeout_seconds=DEFAULT_TIMEOUT_SECONDS,
203     )
204     ensure_success(configure, "configure LCQI")
205     build = run_command(
206         [
207             "cmake",
208             "--build",
209             str(build_dir),
210             "--target",
211             "lcqi_gpt2",

```

```

212         "-j",
213         str(jobs),
214     ],
215     timeout_seconds=DEFAULT_TIMEOUT_SECONDS,
216 )
217 ensure_success(build, "build lcqi_gpt2")
218 binary = build_dir / "labs" / "linux_cpu_inference" / "lcqi_gpt2"
219 if not binary.exists():
220     raise RuntimeError(f"lcqi_gpt2 binary not found: {binary}")
221 return binary
222
223
224 def write_report(
225     report_path: Path,
226     *,
227     cache_dir: Path,
228     binary: Path,
229     prompt: str,
230     max_new_tokens: int,
231     engine: str,
232     result: CommandResult,
233 ) -> None:
234     report_path.parent.mkdir(parents=True, exist_ok=True)
235     lines = [
236         f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
237         f"model_repo_id={MODEL_REPO_ID}",
238         f"model_revision={MODEL_REVISION}",
239         f"model_cache_dir={cache_dir}",
240     ]
241     for file_name, (expected_bytes, expected_sha256) in MODEL_FILE_IDENTITIES.items():
242         path = cache_dir / file_name
243         lines.append(f"file.{file_name}.bytes={path.stat().st_size}")
244         lines.append(f"file.{file_name}.sha256={sha256_file(path)}")
245         lines.append(f"file.{file_name}.expected_bytes={expected_bytes}")
246         lines.append(f"file.{file_name}.expected_sha256={expected_sha256}")
247     lines.extend(
248         [
249             f"binary={binary}",
250             f"prompt={prompt}",
251             f"max_new_tokens={max_new_tokens}",
252             f"engine={engine}",
253             f"command={command_line(result.args)}",
254             f"exit_code={result.returncode}",
255             "stdout_begin",
256             result.stdout.rstrip(),
257             "stdout_end",
258             "stderr_begin",
259             tail(result.stderr).rstrip(),
260             "stderr_end",
261         ]
262     )

```

```

263     passed = result.returncode == 0 and "generated_text " in result.stdout
264     lines.append(f"validation={'PASS' if passed else 'FAIL'}")
265     report_path.write_text("\n".join(lines) + "\n", encoding="utf-8")
266
267
268 def parse_args() -> argparse.Namespace:
269     parser = argparse.ArgumentParser(
270         description="Run LCQI self-owned GPT-2 smoke on a HuggingFace ↵
↵ safetensors directory."
271     )
272     parser.add_argument("--cache-dir", type=Path, default=default_cache_dir())
273     parser.add_argument("--build-dir", type=Path, default=default_build_dir())
274     parser.add_argument("--report", type=Path, default=default_report_path())
275     parser.add_argument("--prompt", default=DEFAULT_PROMPT)
276     parser.add_argument("--max-new-tokens", type=int, default=↵
↵ DEFAULT_MAX_NEW_TOKENS)
277     parser.add_argument("--engine", choices=("cached", "full"), default="cached↵
↵ ")
278     parser.add_argument("--benchmark", action="store_true")
279     parser.add_argument("--timeout-seconds", type=int, default=↵
↵ DEFAULT_TIMEOUT_SECONDS)
280     parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
281     parser.add_argument("--no-download", action="store_true")
282     return parser.parse_args()
283
284
285 def main() -> int:
286     args = parse_args()
287     try:
288         if args.max_new_tokens < 0:
289             raise RuntimeError("--max-new-tokens cannot be negative")
290         ensure_model_files(
291             args.cache_dir,
292             no_download=args.no_download,
293             timeout_seconds=args.timeout_seconds,
294         )
295         binary = build_lcqi_gpt2(args.build_dir, args.jobs)
296         command = [
297             str(binary),
298             "--engine",
299             args.engine,
300             str(args.cache_dir),
301             args.prompt,
302             str(args.max_new_tokens),
303         ]
304         if args.benchmark:
305             command.insert(1, "--benchmark")
306         result = run_command(command, cwd=repo_dir(), timeout_seconds=args.↵
↵ timeout_seconds)
307         write_report(
308             args.report,
309             cache_dir=args.cache_dir,

```



```

310         binary=binary,
311         prompt=args.prompt,
312         max_new_tokens=args.max_new_tokens,
313         engine=args.engine,
314         result=result,
315     )
316     print(f"[lcqi-gpt2] report: {args.report}")
317     print(tail(result.stdout, max_chars=2000))
318     return 0 if result.returncode == 0 else result.returncode
319 except Exception as error:
320     print(f"[lcqi-gpt2] error: {error}", file=sys.stderr)
321     return 1
322
323
324 if __name__ == "__main__":
325     raise SystemExit(main())

```

`run_gpt2_benchmark_compare.py` 用同一个 GPT-2 F32 safetensors 模型分别运行 LCQI full-prefix 与 cached-KV 路径，并在本机存在 llama.cpp/SmolLM2 缓存时附带成熟引擎参考。它的报告说明算法差距、kernel 差距和模型口径差异。

Listing 5.8 tools/run_gpt2_benchmark_compare.py

```

1  #!/usr/bin/env python3
2  from __future__ import annotations
3
4  import argparse
5  import datetime as dt
6  import os
7  import re
8  import shlex
9  import subprocess
10 import sys
11 from dataclasses import dataclass
12 from pathlib import Path
13
14 import run_gpt2_smoke
15
16
17 DEFAULT_PROMPT = "Hello, my name is"
18 DEFAULT_MAX_NEW_TOKENS = 4
19 DEFAULT_BUILD_JOBS = 2
20 DEFAULT_TIMEOUT_SECONDS = 300
21 DEFAULT_SMOLLM2_CACHE_DIR = Path.home() / ".cache" / "lcqi-smolllm2-small-smoke"
22 SMOLLM2_GGUF = "SmolLM2-135M-Instruct-Q4_K_M.gguf"
23
24
25 @dataclass(frozen=True)
26 class EngineRun:
27     name: str
28     command: list[str]
29     returncode: int
30     stdout: str

```

```

31     stderr: str
32     metrics: dict[str, str]
33
34
35 def volume_dir() -> Path:
36     return Path(__file__).resolve().parents[1]
37
38
39 def repo_dir() -> Path:
40     return Path(__file__).resolve().parents[3]
41
42
43 def default_report_path() -> Path:
44     return volume_dir() / "results" / "lcqi-gpt2-benchmark-compare.txt"
45
46
47 def default_build_dir() -> Path:
48     return volume_dir() / "build" / "lcqi-release"
49
50
51 def command_line(args: list[str]) -> str:
52     return " ".join(shlex.quote(arg) for arg in args)
53
54
55 def tail(text: str, max_chars: int = 4000) -> str:
56     return text if len(text) <= max_chars else text[-max_chars:]
57
58
59 def run_command(args: list[str], *, timeout_seconds: int) -> tuple[int, str, ←
    ↪ str]:
60     try:
61         completed = subprocess.run(
62             args,
63             cwd=repo_dir(),
64             text=True,
65             capture_output=True,
66             timeout=timeout_seconds,
67             check=False,
68         )
69     except subprocess.TimeoutExpired as exc:
70         stdout = exc.stdout if isinstance(exc.stdout, str) else ""
71         stderr = exc.stderr if isinstance(exc.stderr, str) else ""
72         return 124, stdout, stderr
73     return completed.returncode, completed.stdout, completed.stderr
74
75
76 def parse_lcqi_metrics(stdout: str) -> dict[str, str]:
77     metrics: dict[str, str] = {}
78     for line in stdout.splitlines():
79         if not line:
80             continue
81         key, _, value = line.partition(" ")

```

```

82     if key in {
83         "engine",
84         "generated_text",
85         "benchmark_load_ms",
86         "benchmark_tokenize_ms",
87         "benchmark_prefill_ms",
88         "benchmark_decode_ms",
89         "benchmark_generate_ms",
90         "benchmark_total_ms",
91         "benchmark_prompt_tokens",
92         "benchmark_generated_tokens",
93         "benchmark_prefill_steps",
94         "benchmark_decode_steps",
95         "benchmark_generate_tokens_per_second",
96         "benchmark_decode_tokens_per_second",
97         "benchmark_kv_cache_bytes",
98     }:
99         metrics[key] = value
100     return metrics
101
102
103 def run_lcqi(
104     binary: Path,
105     model_dir: Path,
106     *,
107     engine: str,
108     prompt: str,
109     max_new_tokens: int,
110     timeout_seconds: int,
111 ) -> EngineRun:
112     command = [
113         str(binary),
114         "--benchmark",
115         "--engine",
116         engine,
117         str(model_dir),
118         prompt,
119         str(max_new_tokens),
120     ]
121     returncode, stdout, stderr = run_command(command, timeout_seconds=↵
↵ timeout_seconds)
122     return EngineRun(
123         name=f"lcqi_{engine}",
124         command=command,
125         returncode=returncode,
126         stdout=stdout,
127         stderr=stderr,
128         metrics=parse_lcqi_metrics(stdout),
129     )
130
131
132 def llama_bench_binary(cache_dir: Path) -> Path:

```

```

133     return cache_dir / "llama.cpp" / "build" / "bin" / "llama-bench"
134
135
136 def smollm2_model_path(cache_dir: Path) -> Path:
137     return cache_dir / "models" / SMOLLM2_GGUF
138
139
140 def parse_llama_bench_metrics(stdout: str) -> dict[str, str]:
141     metrics: dict[str, str] = {}
142     lines = [line.strip() for line in stdout.splitlines() if line.strip()]
143     for line in lines:
144         if not line.startswith("|"):
145             continue
146         columns = [column.strip() for column in line.strip("|").split("|")]
147         if len(columns) != 7 or columns[0] == "model" or columns[0].startswith(↵
↵ "-"):
148             continue
149         metrics["llama_model"] = columns[0]
150         metrics["llama_size"] = columns[1]
151         metrics["llama_params"] = columns[2]
152         metrics["llama_backend"] = columns[3]
153         metrics["llama_threads"] = columns[4]
154         test_name = columns[5]
155         tokens_per_second = re.sub(r"\s+.*$", "", columns[6])
156         if test_name.startswith("pp"):
157             metrics["llama_prefill_test"] = test_name
158             metrics["llama_prefill_tps"] = tokens_per_second
159         elif test_name.startswith("tg"):
160             metrics["llama_decode_test"] = test_name
161             metrics["llama_decode_tps"] = tokens_per_second
162     return metrics
163
164
165 def run_llama_bench(cache_dir: Path, *, timeout_seconds: int) -> EngineRun | ↵
↵ None:
166     binary = llama_bench_binary(cache_dir)
167     model = smollm2_model_path(cache_dir)
168     if not binary.exists() or not os.access(binary, os.X_OK) or not model.↵
↵ exists():
169         return None
170     command = [
171         str(binary),
172         "-m",
173         str(model),
174         "-ngl",
175         "0",
176         "-p",
177         "5",
178         "-n",
179         "4",
180         "-r",
181         "1",

```

```

182     ]
183     returncode, stdout, stderr = run_command(command, timeout_seconds=↵
↵ timeout_seconds)
184     return EngineRun(
185         name="llama_cpp_smolLM2_q4_k_m",
186         command=command,
187         returncode=returncode,
188         stdout=stdout,
189         stderr=stderr,
190         metrics=parse_llama_bench_metrics(stdout),
191     )
192
193
194 def write_report(path: Path, runs: list[EngineRun], *, prompt: str, ↵
↵ max_new_tokens: int) -> None:
195     path.parent.mkdir(parents=True, exist_ok=True)
196     lines = [
197         "LCQI GPT-2 Benchmark Compare",
198         "",
199         f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
200         f"repo_commit={current_commit()}",
201         f"working_tree_dirty={working_tree_dirty()}",
202         f"python={sys.version.split()[0]}",
203         f"prompt={prompt}",
204         f"max_new_tokens={max_new_tokens}",
205         "",
206         "scope=LCQI full_prefix and cached_kv run the same GPT-2 F32 ↵
↵ safetensors model. "
207         "llama.cpp uses SmolLM2-135M-Instruct Q4_K_M as an external mature-↵
↵ engine reference, "
208         "so its numbers are engineering context, not same-model quality ranking↵
↵ .",
209         "",
210     ]
211     for run in runs:
212         lines.extend(
213             [
214                 f"[{run.name}]",
215                 f"returncode={run.returncode}",
216                 f"command={command_line(run.command)}",
217             ]
218         )
219     for key in sorted(run.metrics):
220         lines.append(f"{key}={run.metrics[key]}")
221     lines.extend(
222         [
223             "stdout_tail:",
224             tail(run.stdout.strip(), 2000),
225             "stderr_tail:",
226             tail(run.stderr.strip(), 2000),
227             "",
228         ]

```

```

229     )
230     while lines and lines[-1] == "":
231         lines.pop()
232     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
233
234
235 def current_commit() -> str:
236     result = subprocess.run(
237         ["git", "-C", str(repo_dir()), "rev-parse", "--short", "HEAD"],
238         text=True,
239         capture_output=True,
240         check=False,
241     )
242     return result.stdout.strip() if result.returncode == 0 else "unknown"
243
244
245 def working_tree_dirty() -> str:
246     result = subprocess.run(
247         ["git", "-C", str(repo_dir()), "status", "--porcelain"],
248         text=True,
249         capture_output=True,
250         check=False,
251     )
252     if result.returncode != 0:
253         return "unknown"
254     return "true" if result.stdout.strip() else "false"
255
256
257 def parse_args() -> argparse.Namespace:
258     parser = argparse.ArgumentParser(
259         description="Compare LCQI GPT-2 full-prefix and KV-cache paths, with ↵
↵ optional llama.cpp context."
260     )
261     parser.add_argument("--cache-dir", type=Path, default=run_gpt2_smoke.↵
↵ default_cache_dir())
262     parser.add_argument("--build-dir", type=Path, default=default_build_dir())
263     parser.add_argument("--report", type=Path, default=default_report_path())
264     parser.add_argument("--prompt", default=DEFAULT_PROMPT)
265     parser.add_argument("--max-new-tokens", type=int, default=↵
↵ DEFAULT_MAX_NEW_TOKENS)
266     parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
267     parser.add_argument("--timeout-seconds", type=int, default=↵
↵ DEFAULT_TIMEOUT_SECONDS)
268     parser.add_argument("--no-download", action="store_true")
269     parser.add_argument("--skip-llama", action="store_true")
270     parser.add_argument("--smollm2-cache-dir", type=Path, default=↵
↵ DEFAULT_SMOLLM2_CACHE_DIR)
271     return parser.parse_args()
272
273
274 def main() -> int:
275     args = parse_args()

```

```

276     if args.max_new_tokens < 0:
277         print("[lcqi-compare] --max-new-tokens cannot be negative", file=sys.↵
↵ stderr)
278         return 1
279     try:
280         run_gpt2_smoke.ensure_model_files(
281             args.cache_dir,
282             no_download=args.no_download,
283             timeout_seconds=args.timeout_seconds,
284         )
285         binary = run_gpt2_smoke.build_lcqi_gpt2(args.build_dir, args.jobs)
286         runs = [
287             run_lcqi(
288                 binary,
289                 args.cache_dir,
290                 engine="full",
291                 prompt=args.prompt,
292                 max_new_tokens=args.max_new_tokens,
293                 timeout_seconds=args.timeout_seconds,
294             ),
295             run_lcqi(
296                 binary,
297                 args.cache_dir,
298                 engine="cached",
299                 prompt=args.prompt,
300                 max_new_tokens=args.max_new_tokens,
301                 timeout_seconds=args.timeout_seconds,
302             ),
303         ]
304         if not args.skip_llama:
305             llama_run = run_llama_bench(
306                 args.smollm2_cache_dir.expanduser().resolve(),
307                 timeout_seconds=args.timeout_seconds,
308             )
309             if llama_run is not None:
310                 runs.append(llama_run)
311         write_report(args.report, runs, prompt=args.prompt, max_new_tokens=args.↵
↵ .max_new_tokens)
312     except Exception as error:
313         print(f"[lcqi-compare] error: {error}", file=sys.stderr)
314         return 1
315
316     for run in runs:
317         print(f"{run.name}: returncode={run.returncode}")
318         for key in sorted(run.metrics):
319             print(f"    {key}={run.metrics[key]}")
320     print(f"report={args.report}")
321     return 0 if all(run.returncode == 0 for run in runs) else 1
322
323
324 if __name__ == "__main__":
325     raise SystemExit(main())

```

`run_lcqi_benchmark.sh` 是本地 benchmark 收集脚本。它适合在固定机器上反复运行，把同一组 shape 和 backend 的结果落到报告文件。

Listing 5.9 tools/run_lcqi_benchmark.sh

```

1 #!/usr/bin/env bash
2 set -euo pipefail
3
4 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
5 VOLUME_DIR="$(cd "${SCRIPT_DIR}/.." && pwd)"
6 REPO_DIR="$(cd "${VOLUME_DIR}/../.." && pwd)"
7 BUILD_DIR="${VOLUME_DIR}/build/lcqi-release"
8 RESULT_DIR="${VOLUME_DIR}/results"
9 REPEAT="${1:-200}"
10 STAMP="$(date -u +%Y%m%dT%H%M%S)"
11 CSV_PATH="${RESULT_DIR}/lcqi-bench-${STAMP}.csv"
12 META_PATH="${RESULT_DIR}/lcqi-bench-${STAMP}.meta.txt"
13 PERF_PATH="${RESULT_DIR}/lcqi-bench-${STAMP}.perf.txt"
14
15 mkdir -p "${RESULT_DIR}"
16
17 cmake -S "${VOLUME_DIR}" -B "${BUILD_DIR}" -DCMAKE_BUILD_TYPE=Release
18 cmake --build "${BUILD_DIR}"
19 ctest --test-dir "${BUILD_DIR}" --output-on-failure
20
21 {
22     echo "timestamp_utc=${STAMP}"
23     echo "repeat=${REPEAT}"
24     echo "commit=$(git -C "${REPO_DIR}" rev-parse --short HEAD)"
25     echo "uname=$(uname -a)"
26     echo "compiler=$((${CXX:-c++} --version | sed -n '1p')"
27     echo "cmake=$(cmake --version | sed -n '1p')"
28     echo "bench=${BUILD_DIR}/labs/linux_cpu_inference/lcqi_bench"
29 } > "${META_PATH}"
30
31 "${BUILD_DIR}/labs/linux_cpu_inference/lcqi_bench" "${REPEAT}" > "${CSV_PATH}"
32
33 if command -v perf >/dev/null 2>&1; then
34     perf stat \
35         -e cycles,instructions,cache-misses,branches,branch-misses \
36         "${BUILD_DIR}/labs/linux_cpu_inference/lcqi_bench" "${REPEAT}" \
37         > /dev/null 2> "${PERF_PATH}" || true
38 else
39     echo "perf_unavailable=1" > "${PERF_PATH}"
40 fi
41
42 echo "csv=${CSV_PATH}"
43 echo "meta=${META_PATH}"
44 echo "perf=${PERF_PATH}"

```


5.4 完整测试

`lcqi_tests.cpp` 把 `safetensors`、`tokenizer`、`tiny MLP`、`int8 kernel`、`reference decoder`、`GPT-2` `tiny path`、`trace`、`ledger` 和 `serving probe` 串成验收网。新增优化或 `shape` 时，必须先扩展这里的 `golden` 和边界样例。

Listing 5.10 tests/lcqi_tests.cpp

```

1 #include <lcqi/gpt2_reference.hpp>
2 #include <lcqi/inference.hpp>
3 #include <lcqi/int8_kernels.hpp>
4 #include <lcqi/model.hpp>
5 #include <lcqi/reference_decoder.hpp>
6 #include <lcqi/safetensors.hpp>
7 #include <lcqi/tokenizer.hpp>
8
9 #include <array>
10 #include <cmath>
11 #include <cstring>
12 #include <cstdlib>
13 #include <filesystem>
14 #include <fstream>
15 #include <iostream>
16 #include <span>
17 #include <sstream>
18 #include <stdexcept>
19 #include <string>
20 #include <utility>
21 #include <vector>
22
23 namespace {
24
25 constexpr float LCQI_TEST_EPSILON = 1.0e-5F;
26 constexpr std::int32_t LCQI_TEST_INPUT_SIZE = 4;
27 constexpr std::int32_t LCQI_TEST_HIDDEN_SIZE = 3;
28 constexpr std::int32_t LCQI_TEST_OUTPUT_SIZE = 2;
29 constexpr std::int32_t LCQI_TEST_PREDICTED_CLASS = 1;
30 constexpr float LCQI_TEST_LOGIT_0 = -0.48F;
31 constexpr float LCQI_TEST_LOGIT_1 = 1.06F;
32 constexpr float LCQI_TEST_HIDDEN_0 = 0.0F;
33 constexpr float LCQI_TEST_HIDDEN_1 = 0.4F;
34 constexpr float LCQI_TEST_HIDDEN_2 = 1.4F;
35 constexpr float LCQI_RMS_EXPECTED_0 = 0.848528F;
36 constexpr float LCQI_RMS_EXPECTED_1 = 0.565685F;
37 constexpr float LCQI_ATTENTION_EXPECTED_0 = 2.0F;
38 constexpr float LCQI_ATTENTION_EXPECTED_1 = 3.0F;
39 constexpr float LCQI_KERNEL_MAX_DIFF = 1.0e-5F;
40 constexpr std::int32_t LCQI_SIMD_TEST_BLOCK = 8;
41 constexpr std::int32_t LCQI_REFERENCE_DECODER_PREDICTED_TOKEN = 0;
42 constexpr std::size_t LCQI_REFERENCE_DECODER_VOCAB_SIZE = 5;
43 constexpr std::array<float, LCQI_REFERENCE_DECODER_VOCAB_SIZE> ←
    ↪ LCQI_REFERENCE_DECODER_LOGITS{
44     0.352516979F,

```

```

45     -0.126884326F,
46     0.0797837451F,
47     -0.29797405F,
48     0.127030417F,
49 };
50 constexpr std::int32_t LCQI_REFERENCE_TRACE_TOKEN_COUNT = 3;
51 constexpr std::int32_t LCQI_REFERENCE_TRACE_HIDDEN_SIZE = 4;
52 constexpr std::int32_t LCQI_REFERENCE_TRACE_FIRST_TOKEN = 0;
53 constexpr std::uint64_t LCQI_TOKENIZER_HASH_EXPECTED = 13452902845388333734ULL;
54 constexpr std::int32_t LCQI_SAFETENSORS_PREFIX_BYTES = 8;
55 constexpr std::int32_t LCQI_TEST_BYTE_BITS = 8;
56 constexpr std::uint64_t LCQI_SAFETENSORS_VALID_PAYLOAD_BYTES = 48;
57 constexpr std::uint64_t LCQI_SAFETENSORS_EMBED_BEGIN = 0;
58 constexpr std::uint64_t LCQI_SAFETENSORS_EMBED_END = 24;
59 constexpr std::uint64_t LCQI_SAFETENSORS_QPROJ_BEGIN = 24;
60 constexpr std::uint64_t LCQI_SAFETENSORS_QPROJ_END = 48;
61 constexpr unsigned int LCQI_BYTE_MASK = 0xFFU;
62 constexpr std::int32_t LCQI_GPT2_PREDICTED_TOKEN = 3;
63 constexpr std::size_t LCQI_GPT2_VOCAB_SIZE = 6;
64 constexpr std::array<float, LCQI_GPT2_VOCAB_SIZE> LCQI_GPT2_LOGITS{
65     0.407358F,
66     -0.504049F,
67     -0.107834F,
68     0.840337F,
69     -0.450123F,
70     -0.156763F,
71 };
72 constexpr std::int32_t LCQI_GPT2_GENERATED_LENGTH = 5;
73 constexpr std::int32_t LCQI_GPT2_TOKENIZER_HELLO_ID = 7;
74 constexpr std::uint16_t LCQI_TEST_F16_ONE = 0x3C00U;
75 constexpr std::uint16_t LCQI_TEST_F16_NEGATIVE_TWO = 0xC000U;
76 constexpr std::uint16_t LCQI_TEST_BF16_ONE_POINT_FIVE = 0x3FC0U;
77 constexpr std::uint16_t LCQI_TEST_BF16_NEGATIVE_HALF = 0xBF00U;
78
79 struct KernelShapeCase {
80     std::int32_t input_size = 0;
81     std::int32_t output_size = 0;
82     std::int32_t output_block_size = 0;
83 };
84
85 struct RawTensorFixture {
86     std::string name;
87     std::string dtype;
88     std::vector<std::int64_t> shape;
89     std::vector<std::uint8_t> bytes;
90 };
91
92 struct FloatTensorFixture {
93     std::string name;
94     std::vector<std::int64_t> shape;
95     std::vector<float> values;
96 };

```

```

97
98 constexpr std::array<KernelShapeCase, 4> LCQI_KERNEL_SHAPE_CASES{{
99     {17, 19, 8},
100     {33, 7, 8},
101     {64, 32, 16},
102     {65, 31, 16},
103 }};
104
105 void require(bool condition, const char* message) {
106     if (!condition) {
107         throw std::runtime_error(message);
108     }
109 }
110
111 void require_close(float actual, float expected, const char* message) {
112     if (std::fabs(actual - expected) > LCQI_TEST_EPSILON) {
113         throw std::runtime_error(message);
114     }
115 }
116
117 std::filesystem::path test_model_path() {
118     return std::filesystem::path(__FILE__).parent_path().parent_path() /
119         "models" / "tiny_mlp_i8.txt";
120 }
121
122 std::filesystem::path test_tokenizer_path() {
123     return std::filesystem::path(__FILE__).parent_path().parent_path() /
124         "models" / "tiny_tokenizer.txt";
125 }
126
127 std::filesystem::path temp_file_path(const std::string& name) {
128     return std::filesystem::temp_directory_path() / name;
129 }
130
131 void write_le_u64(std::ofstream& output, std::uint64_t value) {
132     for (std::int32_t i = 0; i < LCQI_SAFETENSORS_PREFIX_BYTES; ++i) {
133         const char byte = static_cast<char>(
134             (value >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK);
135         output.write(&byte, 1);
136     }
137 }
138
139 void append_le_u16(std::vector<std::uint8_t>& output, std::uint16_t value) {
140     output.push_back(static_cast<std::uint8_t>(value & LCQI_BYTE_MASK));
141     output.push_back(static_cast<std::uint8_t>((value >> LCQI_TEST_BYTE_BITS) &
142         ↪ LCQI_BYTE_MASK));
143 }
144
145 void append_le_f32(std::vector<std::uint8_t>& output, float value) {
146     std::uint32_t bits = 0;
147     std::memcpy(&bits, &value, sizeof(bits));
148     for (std::int32_t i = 0; i < 4; ++i) {

```

```

148     output.push_back(static_cast<std::uint8_t>(
149         (bits >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK));
150 }
151 }
152
153 void write_safetensors_fixture(const std::filesystem::path& path,
154                               const std::string& header,
155                               std::uint64_t payload_bytes) {
156     std::ofstream output(path, std::ios::binary);
157     if (!output) {
158         throw std::runtime_error("cannot create safetensors test fixture");
159     }
160     write_le_u64(output, static_cast<std::uint64_t>(header.size()));
161     output.write(header.data(), static_cast<std::streamsize>(header.size()));
162     for (std::uint64_t i = 0; i < payload_bytes; ++i) {
163         const char byte = static_cast<char>(i & 0xFFU);
164         output.write(&byte, 1);
165     }
166 }
167
168 std::string json_shape(std::span<const std::int64_t> shape) {
169     std::ostringstream stream;
170     stream << '[';
171     for (std::size_t index = 0; index < shape.size(); ++index) {
172         if (index != 0) {
173             stream << ',';
174         }
175         stream << shape[index];
176     }
177     stream << ']';
178     return stream.str();
179 }
180
181 void write_safetensors_raw_fixture(const std::filesystem::path& path,
182                                    std::span<const RawTensorFixture> tensors) {
183     std::ostringstream header;
184     header << "{\"__metadata__\":{\"format\":\"lcqi-test\"}";
185     std::uint64_t offset = 0;
186     for (const RawTensorFixture& tensor : tensors) {
187         header << ",\"\" << tensor.name << "\":{\"dtype\":\"\" << tensor.dtype
188             << "\",\"shape\":\"\" << json_shape(tensor.shape)
189             << "\",\"data_offsets\":[\" << offset << ', '
190             << offset + tensor.bytes.size() << "]}\"";
191         offset += tensor.bytes.size();
192     }
193     header << '}'';
194
195     std::ofstream output(path, std::ios::binary);
196     if (!output) {
197         throw std::runtime_error("cannot create safetensors raw test fixture");
198     }
199     const std::string header_text = header.str();

```

```

200     write_le_u64(output, static_cast<std::uint64_t>(header_text.size()));
201     output.write(header_text.data(), static_cast<std::streamsize>(header_text.↵
↵ size()));
202     for (const RawTensorFixture& tensor : tensors) {
203         output.write(reinterpret_cast<const char*>(tensor.bytes.data()),
204                     static_cast<std::streamsize>(tensor.bytes.size()));
205     }
206 }
207
208 std::vector<std::uint8_t> f32_payload(std::span<const float> values) {
209     std::vector<std::uint8_t> bytes;
210     bytes.reserve(values.size() * 4);
211     for (const float value : values) {
212         append_le_f32(bytes, value);
213     }
214     return bytes;
215 }
216
217 std::vector<float> transpose_linear_to_hf_conv1d(const lcqi::Gpt2LinearF32& ↵
↵ linear) {
218     std::vector<float> transposed(
219         static_cast<std::size_t>(linear.input_size) *
220         static_cast<std::size_t>(linear.output_size),
221         0.0F);
222     for (std::int32_t out = 0; out < linear.output_size; ++out) {
223         for (std::int32_t in = 0; in < linear.input_size; ++in) {
224             transposed[static_cast<std::size_t>(in) *
225                     static_cast<std::size_t>(linear.output_size) +
226                     static_cast<std::size_t>(out)] =
227                 linear.weights[static_cast<std::size_t>(out) *
228                             static_cast<std::size_t>(linear.input_size) ↵
↵ +
229                             static_cast<std::size_t>(in)];
230         }
231     }
232     return transposed;
233 }
234
235 void add_f32_tensor(std::vector<RawTensorFixture>& tensors,
236                   std::string name,
237                   std::vector<std::int64_t> shape,
238                   std::span<const float> values) {
239     tensors.push_back(RawTensorFixture{
240         std::move(name),
241         "F32",
242         std::move(shape),
243         f32_payload(values),
244     });
245 }
246
247 void write_text_file(const std::filesystem::path& path, const std::string& text↵
↵ ) {

```

```

248     std::ofstream output(path);
249     if (!output) {
250         throw std::runtime_error("cannot create text fixture");
251     }
252     output << text;
253 }
254
255 void write_tiny_gpt2_safetensors(const std::filesystem::path& path,
256                                 const lcqi::Gpt2ReferenceModel& model) {
257     std::vector<RawTensorFixture> tensors;
258     add_f32_tensor(tensors,
259                   "transformer.wte.weight",
260                   {model.config.vocab_size, model.config.hidden_size},
261                   model.token_embedding);
262     add_f32_tensor(tensors,
263                   "transformer.wpe.weight",
264                   {model.config.max_positions, model.config.hidden_size},
265                   model.position_embedding);
266     for (std::int32_t layer_id = 0; layer_id < model.config.layer_count; ++layer_id) {
267         const lcqi::Gpt2LayerWeightsF32& layer =
268             model.layers[static_cast<std::size_t>(layer_id)];
269         const std::string prefix = "transformer.h." + std::to_string(layer_id) +
270             ".";
271         add_f32_tensor(tensors, prefix + "ln_1.weight", {model.config.hidden_size, layer.ln_1_weight});
272         add_f32_tensor(tensors, prefix + "ln_1.bias", {model.config.hidden_size, layer.ln_1_bias});
273         const std::vector<float> c_attn_hf = transpose_linear_to_hf_conv1d(layer.c_attn);
274         add_f32_tensor(tensors,
275                       prefix + "attn.c_attn.weight",
276                       {model.config.hidden_size, model.config.hidden_size * 3},
277                       c_attn_hf);
278         add_f32_tensor(tensors,
279                       prefix + "attn.c_attn.bias",
280                       {model.config.hidden_size * 3},
281                       layer.c_attn.bias);
282         const std::vector<float> c_proj_hf = transpose_linear_to_hf_conv1d(layer.c_proj);
283         add_f32_tensor(tensors,
284                       prefix + "attn.c_proj.weight",
285                       {model.config.hidden_size, model.config.hidden_size},
286                       c_proj_hf);
287         add_f32_tensor(tensors,
288                       prefix + "attn.c_proj.bias",
289                       {model.config.hidden_size},
290                       layer.c_proj.bias);
291         add_f32_tensor(tensors, prefix + "ln_2.weight", {model.config.hidden_size, layer.ln_2_weight});
292         add_f32_tensor(tensors, prefix + "ln_2.bias", {model.config.hidden_size, layer.ln_2_bias});

```

```

↪ }, layer.ln_2_bias);
293     const std::vector<float> c_fc_hf = transpose_linear_to_hf_conv1d(layer.↪
↪ c_fc);
294     add_f32_tensor(tensors,
295                     prefix + "mlp.c_fc.weight",
296                     {model.config.hidden_size, model.config.↪
↪ intermediate_size},
297                     c_fc_hf);
298     add_f32_tensor(tensors,
299                     prefix + "mlp.c_fc.bias",
300                     {model.config.intermediate_size},
301                     layer.c_fc.bias);
302     const std::vector<float> mlp_proj_hf =
303         transpose_linear_to_hf_conv1d(layer.mlp_c_proj);
304     add_f32_tensor(tensors,
305                     prefix + "mlp.c_proj.weight",
306                     {model.config.intermediate_size, model.config.↪
↪ hidden_size},
307                     mlp_proj_hf);
308     add_f32_tensor(tensors,
309                     prefix + "mlp.c_proj.bias",
310                     {model.config.hidden_size},
311                     layer.mlp_c_proj.bias);
312 }
313 add_f32_tensor(tensors,
314                 "transformer.ln_f.weight",
315                 {model.config.hidden_size},
316                 model.final_ln_weight);
317 add_f32_tensor(tensors,
318                 "transformer.ln_f.bias",
319                 {model.config.hidden_size},
320                 model.final_ln_bias);
321 write_safetensors_raw_fixture(path, tensors);
322 }
323
324 void test_tiny_mlp() {
325     const lcqi::TinyMlpModel model = lcqi::load_model(test_model_path());
326     require(model.input_size == LCQI_TEST_INPUT_SIZE, "input size mismatch");
327     require(model.hidden_size == LCQI_TEST_HIDDEN_SIZE, "hidden size mismatch")↪
↪ ;
328     require(model.output_size == LCQI_TEST_OUTPUT_SIZE, "output size mismatch")↪
↪ ;
329
330     const std::vector<float> input{1.0F, 2.0F, -1.0F, 0.5F};
331     const lcqi::InferenceResult result = lcqi::run_inference(model, input);
332
333     require(result.predicted_class == LCQI_TEST_PREDICTED_CLASS, "unexpected ↪
↪ predicted class");
334     require(result.logits.size() == static_cast<std::size_t>(↪
↪ LCQI_TEST_OUTPUT_SIZE),
335             "unexpected logits size");
336     require_close(result.logits[0], LCQI_TEST_LOGIT_0, "logit 0 mismatch");

```

```

337     require_close(result.logits[1], LCQI_TEST_LOGIT_1, "logit 1 mismatch");
338
339     std::vector<float> hidden(static_cast<std::size_t>(LCQI_TEST_HIDDEN_SIZE), ←
    ↪ 0.0F);
340     lcqi::linear_i8(model.hidden, input, hidden);
341     lcqi::relu_inplace(hidden);
342     require_close(hidden[0], LCQI_TEST_HIDDEN_0, "hidden 0 mismatch");
343     require_close(hidden[1], LCQI_TEST_HIDDEN_1, "hidden 1 mismatch");
344     require_close(hidden[2], LCQI_TEST_HIDDEN_2, "hidden 2 mismatch");
345 }
346
347 void test_tokenizer_contract() {
348     const lcqi::TokenizerModel tokenizer = lcqi::load_tokenizer(←
    ↪ test_tokenizer_path());
349     require(tokenizer.bos_id == 0, "tokenizer BOS id mismatch");
350     require(tokenizer.eos_id == 4, "tokenizer EOS id mismatch");
351     require(tokenizer.unk_id == -1, "tokenizer UNK id mismatch");
352     require(tokenizer.chat_template_hash == "tiny-chat-template-v1",
353             "tokenizer template hash mismatch");
354
355     const std::vector<std::int32_t> token_ids =
356         lcqi::encode_prompt(tokenizer, "tok1 tok2 tok3");
357     const std::vector<std::int32_t> expected{0, 1, 2, 3, 4};
358     require(token_ids == expected, "tokenizer prompt ids mismatch");
359     require(lcqi::tokenizer_contract_hash(tokenizer) == ←
    ↪ LCQI_TOKENIZER_HASH_EXPECTED,
360             "tokenizer contract hash mismatch");
361 }
362
363 void test_tokenizer_rejects_unknown_without_unk() {
364     const lcqi::TokenizerModel tokenizer = lcqi::load_tokenizer(←
    ↪ test_tokenizer_path());
365     bool threw = false;
366     try {
367         static_cast<void>(lcqi::encode_prompt(tokenizer, "tok1 missing_token"))←
    ↪ ;
368     } catch (const std::runtime_error&) {
369         threw = true;
370     }
371     require(threw, "tokenizer should reject unknown token when unk id is absent←
    ↪ ");
372 }
373
374 void test_safetensors_manifest_contract() {
375     const std::filesystem::path path =
376         temp_file_path("lcqi_safetensors_manifest_contract.safetensors");
377     const std::string header =
378         R("{\"__metadata__\":{\"format\":\"lcqi-fixture\"},\"model.embed_tokens.weight←
    ↪ \":{\"dtype\":\"F32\",\"shape\":[2,3],\"data_offsets\":[0,24]},\"model.layers.0.←
    ↪ attn.q_proj.weight\":{\"dtype\":\"F16\",\"shape\":[4,3],\"data_offsets\"←
    ↪ :[24,48]}}");
379     write_safetensors_fixture(path, header, ←

```



```

↪ LCQI_SAFETENSORS_VALID_PAYLOAD_BYTES);
380
381 const lcqi::SafeTensorManifest manifest = lcqi::load_safetensors_manifest(↪
↪ path);
382 std::filesystem::remove(path);
383
384 require(manifest.header_size == static_cast<std::uint64_t>(header.size()),
385         "safetensors header size mismatch");
386 require(manifest.data_start_offset ==
387         static_cast<std::uint64_t>(LCQI_SAFETENSORS_PREFIX_BYTES) +
388         static_cast<std::uint64_t>(header.size()),
389         "safetensors data start mismatch");
390 require(manifest.metadata.size() == 1, "safetensors metadata count mismatch↪
↪ ");
391 require(manifest.metadata[0].first == "format" &&
392         manifest.metadata[0].second == "lcqi-fixture",
393         "safetensors metadata mismatch");
394 require(manifest.tensors.size() == 2, "safetensors tensor count mismatch");
395
396 const lcqi::SafeTensorEntry* embedding =
397     manifest.find_tensor("model.embed_tokens.weight");
398 require(embedding != nullptr, "safetensors embedding tensor missing");
399 require(embedding->dtype == "F32", "safetensors embedding dtype mismatch");
400 require(embedding->shape == std::vector<std::int64_t>({2, 3}),
401         "safetensors embedding shape mismatch");
402 require(embedding->data_begin == LCQI_SAFETENSORS_EMBED_BEGIN &&
403         embedding->data_end == LCQI_SAFETENSORS_EMBED_END,
404         "safetensors embedding offset mismatch");
405 require(embedding->byte_size() ==
406         LCQI_SAFETENSORS_EMBED_END - LCQI_SAFETENSORS_EMBED_BEGIN,
407         "safetensors embedding byte size mismatch");
408
409 const lcqi::SafeTensorEntry* q_proj =
410     manifest.find_tensor("model.layers.0.attn.q_proj.weight");
411 require(q_proj != nullptr, "safetensors q projection tensor missing");
412 require(q_proj->dtype == "F16", "safetensors q projection dtype mismatch");
413 require(q_proj->data_begin == LCQI_SAFETENSORS_QPROJ_BEGIN &&
414         q_proj->data_end == LCQI_SAFETENSORS_QPROJ_END,
415         "safetensors q projection offset mismatch");
416 }
417
418 void test_safetensors_rejects_bad_offsets() {
419     const std::filesystem::path path =
420         temp_file_path("lcqi_safetensors_bad_offsets.safetensors");
421     const std::string header =
422         R"({"tensor":{"dtype":"F32","shape":[4],"data_offsets":[0,32]}})";
423     write_safetensors_fixture(path, header, 4);
424
425     bool threw = false;
426     try {
427         static_cast<void>(lcqi::load_safetensors_manifest(path));
428     } catch (const std::runtime_error&) {

```

```

429     threw = true;
430 }
431 std::filesystem::remove(path);
432 require(threw, "safetensors parser should reject offsets beyond payload");
433 }
434
435 void test_safetensors_rejects_bad_dtype_shape_size() {
436     const std::filesystem::path path =
437         temp_file_path("lcqi_safetensors_bad_dtype_shape_size.safetensors");
438     const std::string header =
439         R"({"tensor":{"dtype":"F32","shape":[4],"data_offsets":[0,8]}})";
440     write_safetensors_fixture(path, header, 8);
441
442     bool threw = false;
443     try {
444         static_cast<void>(lcqi::load_safetensors_manifest(path));
445     } catch (const std::runtime_error&) {
446         threw = true;
447     }
448     std::filesystem::remove(path);
449     require(threw, "safetensors parser should reject dtype/shape byte mismatch"↵
↵ );
450 }
451
452 void test_safetensors_reads_float_tensors() {
453     const std::filesystem::path path =
454         temp_file_path("lcqi_safetensors_float_read.safetensors");
455     std::vector<std::uint8_t> f16_bytes;
456     append_le_u16(f16_bytes, LCQI_TEST_F16_ONE);
457     append_le_u16(f16_bytes, LCQI_TEST_F16_NEGATIVE_TWO);
458     std::vector<std::uint8_t> bf16_bytes;
459     append_le_u16(bf16_bytes, LCQI_TEST_BF16_ONE_POINT_FIVE);
460     append_le_u16(bf16_bytes, LCQI_TEST_BF16_NEGATIVE_HALF);
461
462     const std::array<RawTensorFixture, 3> tensors{{
463         {"f32", "F32", {2}, f32_payload(std::array<float, 2>{1.25F, -2.5F})},
464         {"f16", "F16", {2}, f16_bytes},
465         {"bf16", "BF16", {2}, bf16_bytes},
466     }};
467     write_safetensors_raw_fixture(path, tensors);
468
469     const lcqi::SafeTensorFile file = lcqi::load_safetensors_file(path);
470     std::filesystem::remove(path);
471     const std::vector<float> f32 = lcqi::read_safetensor_f32_tensor(file, "f32"↵
↵ );
472     const std::vector<float> f16 = lcqi::read_safetensor_f32_tensor(file, "f16"↵
↵ );
473     const std::vector<float> bf16 = lcqi::read_safetensor_f32_tensor(file, "↵
↵ bf16");
474
475     require_close(f32[0], 1.25F, "safetensors F32 value 0 mismatch");
476     require_close(f32[1], -2.5F, "safetensors F32 value 1 mismatch");

```

```

477     require_close(f16[0], 1.0F, "safetensors F16 value 0 mismatch");
478     require_close(f16[1], -2.0F, "safetensors F16 value 1 mismatch");
479     require_close(bf16[0], 1.5F, "safetensors BF16 value 0 mismatch");
480     require_close(bf16[1], -0.5F, "safetensors BF16 value 1 mismatch");
481 }
482
483 void test_gpt2_tiny_forward_and_generation() {
484     const lcqi::Gpt2ReferenceModel model = lcqi::make_tiny_gpt2_reference_model↵
↵ ();
485     const std::vector<std::int32_t> tokens{1, 2, 3};
486     const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(model, tokens↵
↵ );
487
488     require(result.logits.size() == LCQI_GPT2_VOCAB_SIZE, "GPT-2 logits size ↵
↵ mismatch");
489     require(result.predicted_token == LCQI_GPT2_PREDICTED_TOKEN,
490             "GPT-2 predicted token mismatch");
491     for (std::size_t index = 0; index < LCQI_GPT2_LOGITS.size(); ++index) {
492         require_close(result.logits[index], LCQI_GPT2_LOGITS[index], "GPT-2 ↵
↵ logit mismatch");
493     }
494
495     const std::vector<std::int32_t> generated =
496         lcqi::gpt2_generate_greedy(model, tokens, 2);
497     const std::vector<std::int32_t> expected{1, 2, 3, 3, 3};
498     require(generated == expected, "GPT-2 greedy generation mismatch");
499     require(static_cast<std::int32_t>(generated.size()) == ↵
↵ LCQI_GPT2_GENERATED_LENGTH,
500             "GPT-2 generated length mismatch");
501
502     lcqi::Gpt2KvCache cache(model.config);
503     lcqi::Gpt2ForwardResult cached_result;
504     for (const std::int32_t token : tokens) {
505         cached_result = lcqi::run_gpt2_forward_cached(model, cache, token);
506     }
507     require(cache.filled_tokens() == static_cast<std::int32_t>(tokens.size()),
508             "GPT-2 KV cache filled token count mismatch");
509     require(cache.byte_size() > 0, "GPT-2 KV cache byte size should be non-zero↵
↵ ");
510     require(cached_result.predicted_token == result.predicted_token,
511             "cached GPT-2 predicted token mismatch");
512     require(cached_result.logits.size() == result.logits.size(),
513             "cached GPT-2 logits size mismatch");
514     for (std::size_t index = 0; index < result.logits.size(); ++index) {
515         require_close(cached_result.logits[index],
516                       result.logits[index],
517                       "cached GPT-2 logit mismatch");
518     }
519
520     const std::vector<std::int32_t> cached_generated =
521         lcqi::gpt2_generate_greedy_cached(model, tokens, 2);
522     require(cached_generated == expected, "cached GPT-2 greedy generation ↵

```

```

    ↪ mismatch");
523 }
524
525 void test_gpt2_byte_bpe_tokenizer() {
526     const std::filesystem::path vocab_path = temp_file_path("lcqi_gpt2_vocab.↪
    ↪ json");
527     const std::filesystem::path merges_path = temp_file_path("lcqi_gpt2_merges.↪
    ↪ txt");
528     const std::string space_marker = "\xC4\xA0";
529     const std::string vocab =
530         "{ \"Hello\":7, \" \":8, \"\" + space_marker + "world\":9, \"!\":10}";
531     const std::string merges =
532         "#version: 0.2\n"
533         "H e\n"
534         "He l\n"
535         "Hel l\n"
536         "Hell o\n" +
537         space_marker + " w\n" +
538         space_marker + "w o\n" +
539         space_marker + "wo r\n" +
540         space_marker + "wor l\n" +
541         space_marker + "worl d\n";
542     write_text_file(vocab_path, vocab);
543     write_text_file(merges_path, merges);
544
545     const lcqi::Gpt2Tokenizer tokenizer =
546         lcqi::load_gpt2_tokenizer(vocab_path, merges_path, -1, -1);
547     std::filesystem::remove(vocab_path);
548     std::filesystem::remove(merges_path);
549
550     const std::vector<std::int32_t> ids =
551         lcqi::gpt2_encode(tokenizer, "Hello, world!");
552     const std::vector<std::int32_t> expected{
553         LCQI_GPT2_TOKENIZER_HELLO_ID,
554         8,
555         9,
556         10,
557     };
558     require(ids == expected, "GPT-2 BPE ids mismatch");
559     require(lcqi::gpt2_decode(tokenizer, ids) == "Hello, world!",
560         "GPT-2 BPE decode mismatch");
561 }
562
563 void test_gpt2_loads_hf_style_tiny_checkpoint() {
564     const std::filesystem::path directory =
565         temp_file_path("lcqi_gpt2_tiny_checkpoint_dir");
566     std::filesystem::remove_all(directory);
567     std::filesystem::create_directories(directory);
568
569     const lcqi::Gpt2ReferenceModel original = lcqi::↪
    ↪ make_tiny_gpt2_reference_model();
570     write_text_file(

```

```

571     directory / "config.json",
572     R("{\"model_type\":\"gpt2\",\"n_embd\":4,\"n_head\":2,\"n_layer\":1,\"n_positions\"↵
↵ :8,\"n_ctx\":8,\"vocab_size\":6,\"n_inner\":8,\"layer_norm_epsilon\":0.00001,\"↵
↵ activation_function\":\"gelu_new\",\"bos_token_id\":0,\"eos_token_id\":5}");
573 write_tiny_gpt2_safetensors(directory / "model.safetensors", original);
574
575 const lcqi::Gpt2ReferenceModel loaded = lcqi::load_gpt2_from_directory(↵
↵ directory);
576 const std::vector<std::int32_t> tokens{1, 2, 3};
577 const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(loaded, ↵
↵ tokens);
578 std::filesystem::remove_all(directory);
579
580 require(result.predicted_token == LCQI_GPT2_PREDICTED_TOKEN,
581         "loaded GPT-2 predicted token mismatch");
582 for (std::size_t index = 0; index < LCQI_GPT2_LOGITS.size(); ++index) {
583     require_close(result.logits[index],
584                   LCQI_GPT2_LOGITS[index],
585                   "loaded GPT-2 logit mismatch");
586 }
587 }
588
589 void test_gpt2_loader_rejects_bad_shape() {
590     const std::filesystem::path directory =
591         temp_file_path("lcqi_gpt2_bad_shape_dir");
592     std::filesystem::remove_all(directory);
593     std::filesystem::create_directories(directory);
594
595     const lcqi::Gpt2ReferenceModel original = lcqi::↵
↵ make_tiny_gpt2_reference_model();
596     write_text_file(
597         directory / "config.json",
598         R("{\"model_type\":\"gpt2\",\"n_embd\":5,\"n_head\":1,\"n_layer\":1,\"n_positions\"↵
↵ :8,\"n_ctx\":8,\"vocab_size\":6,\"n_inner\":8,\"layer_norm_epsilon\":0.00001,\"↵
↵ activation_function\":\"gelu_new\",\"bos_token_id\":0,\"eos_token_id\":5}");
599     write_tiny_gpt2_safetensors(directory / "model.safetensors", original);
600
601     bool threw = false;
602     try {
603         static_cast<void>(lcqi::load_gpt2_from_directory(directory));
604     } catch (const std::runtime_error&) {
605         threw = true;
606     }
607     std::filesystem::remove_all(directory);
608     require(threw, "GPT-2 loader should reject bad tensor shape");
609 }
610
611 void test_reference_rms_norm() {
612     const std::vector<float> input{3.0F, 4.0F};
613     const std::vector<float> weight{1.0F, 0.5F};
614     std::vector<float> output(2, 0.0F);
615     lcqi::rms_norm(input, weight, 0.0F, output);

```

```

616     require_close(output[0], LCQI_RMS_EXPECTED_0, "RMSNorm output 0 mismatch");
617     require_close(output[1], LCQI_RMS_EXPECTED_1, "RMSNorm output 1 mismatch");
618 }
619
620 void test_reference_rope() {
621     std::vector<float> heads{1.0F, 0.0F, 0.0F, 1.0F};
622     lcqi::apply_rope(heads, 1, 4, 1, 10000.0F);
623     require_close(heads[0], std::cos(1.0F), "RoPE pair 0 cosine mismatch");
624     require_close(heads[1], std::sin(1.0F), "RoPE pair 0 sine mismatch");
625     require_close(heads[2], -std::sin(0.01F), "RoPE pair 1 negative sine ←
    ↪ mismatch");
626     require_close(heads[3], std::cos(0.01F), "RoPE pair 1 cosine mismatch");
627 }
628
629 void test_reference_kv_attention() {
630     lcqi::DecoderConfig config;
631     config.hidden_size = 4;
632     config.query_heads = 2;
633     config.kv_heads = 1;
634     config.head_dim = 2;
635     config.intermediate_size = 4;
636     config.vocab_size = 4;
637     config.max_context = 4;
638
639     lcqi::ReferenceKVCache cache(config, 1);
640     const std::vector<float> key{1.0F, 0.0F};
641     const std::vector<float> value{LCQI_ATTENTION_EXPECTED_0, ←
    ↪ LCQI_ATTENTION_EXPECTED_1};
642     cache.append(0, 0, key, value);
643     require(cache.filled_tokens() == 1, "KV filled token count mismatch");
644     require_close(cache.key(0, 0, 0)[0], 1.0F, "KV key read mismatch");
645
646     const std::vector<float> query{1.0F, 0.0F, 0.0F, 1.0F};
647     std::vector<float> output(4, 0.0F);
648     lcqi::attention_decode(config, cache, 0, 0, query, output);
649     require_close(output[0], LCQI_ATTENTION_EXPECTED_0, "attention head 0 dim 0 ←
    ↪ mismatch");
650     require_close(output[1], LCQI_ATTENTION_EXPECTED_1, "attention head 0 dim 1 ←
    ↪ mismatch");
651     require_close(output[2], LCQI_ATTENTION_EXPECTED_0, "attention head 1 dim 0 ←
    ↪ mismatch");
652     require_close(output[3], LCQI_ATTENTION_EXPECTED_1, "attention head 1 dim 1 ←
    ↪ mismatch");
653 }
654
655 void test_reference_decoder_end_to_end() {
656     const lcqi::ReferenceDecoderModel model = lcqi::←
    ↪ make_tiny_reference_decoder_model();
657     const std::vector<std::int32_t> tokens{1, 2, 3};
658     const lcqi::ReferenceDecodeResult result = lcqi::run_reference_decode(model←
    ↪ , tokens);
659     require(result.logits.size() == LCQI_REFERENCE_DECODER_VOCAB_SIZE,

```

```

660         "reference decoder logits size mismatch");
661     require(result.predicted_token == LCQI_REFERENCE_DECODER_PREDICTED_TOKEN,
662         "reference decoder predicted token mismatch");
663     for (std::size_t index = 0; index < LCQI_REFERENCE_DECODER_LOGITS.size(); ↵
        ↵ ++index) {
664         require_close(result.logits[index],
665             LCQI_REFERENCE_DECODER_LOGITS[index],
666             "reference decoder golden logit mismatch");
667     }
668 }
669
670 void test_reference_decoder_trace_contract() {
671     const lcqi::ReferenceDecoderModel model = lcqi::↵
        ↵ make_tiny_reference_decoder_model();
672     const std::vector<std::int32_t> tokens{1, 2, 3};
673     const lcqi::ReferenceDecodeTraceResult trace =
674         lcqi::run_reference_decode_with_trace(model, tokens);
675
676     require(trace.result.predicted_token == ↵
        ↵ LCQI_REFERENCE_DECODER_PREDICTED_TOKEN,
677         "reference trace predicted token mismatch");
678     require(!trace.checkpoints.empty(), "reference trace checkpoints missing");
679
680     const lcqi::ReferenceTensorCheckpoint& first = trace.checkpoints.front();
681     require(first.name == "embedding", "first checkpoint should be embedding");
682     require(first.token_position == LCQI_REFERENCE_TRACE_FIRST_TOKEN,
683         "first checkpoint token position mismatch");
684     require(first.dtype == "f32", "first checkpoint dtype mismatch");
685     require(first.layout == "row_major", "first checkpoint layout mismatch");
686     require(first.shape.size() == 1 &&
687         first.shape[0] == LCQI_REFERENCE_TRACE_HIDDEN_SIZE,
688         "first checkpoint shape mismatch");
689     require(first.stride.size() == 1 && first.stride[0] == 1,
690         "first checkpoint stride mismatch");
691
692     bool saw_logits = false;
693     bool saw_kv_slot = false;
694     for (const lcqi::ReferenceTensorCheckpoint& checkpoint : trace.checkpoints)↵
        ↵ {
695         if (checkpoint.name == "kv_cache_key_slot") {
696             saw_kv_slot = true;
697             require(checkpoint.shape.size() == 4, "KV checkpoint rank mismatch"↵
        ↵ );
698             require(checkpoint.layout == "kv_contiguous", "KV checkpoint layout↵
        ↵ mismatch");
699         }
700         if (checkpoint.name == "logits") {
701             saw_logits = true;
702             require(checkpoint.token_position == ↵
        ↵ LCQI_REFERENCE_TRACE_TOKEN_COUNT - 1,
703                 "logits checkpoint token position mismatch");
704             require(checkpoint.shape.size() == 1 &&

```



```

705         checkpoint.shape[0] ==
706             static_cast<std::int32_t>(<
    ↪ LCQI_REFERENCE_DECODER_VOCAB_SIZE),
707             "logits checkpoint shape mismatch");
708         require(checkpoint.values.size() == LCQI_REFERENCE_DECODER_LOGITS.<
    ↪ size(),
709             "logits checkpoint value size mismatch");
710         for (std::size_t index = 0; index < LCQI_REFERENCE_DECODER_LOGITS.<
    ↪ size(); ++index) {
711             require_close(checkpoint.values[index],
712                 LCQI_REFERENCE_DECODER_LOGITS[index],
713                 "trace logits checkpoint mismatch");
714         }
715     }
716 }
717 require(saw_kv_slot, "KV checkpoint missing");
718 require(saw_logits, "logits checkpoint missing");
719 }
720
721 void test_packed_i8_kernel_matches_scalar() {
722     for (const KernelShapeCase& shape_case : LCQI_KERNEL_SHAPE_CASES) {
723         const lcqi::QuantizedLinearLayer layer =
724             lcqi::make_deterministic_i8_layer(shape_case.input_size,
725                 shape_case.output_size,
726                 0.03125F);
727         const lcqi::PackedLinearI8 packed =
728             lcqi::pack_linear_i8(layer, shape_case.output_block_size);
729         const std::vector<float> input = lcqi::make_deterministic_input(layer.<
    ↪ input_size);
730         std::vector<float> scalar_output(static_cast<std::size_t>(layer.<
    ↪ output_size), 0.0F);
731         std::vector<float> packed_output(static_cast<std::size_t>(layer.<
    ↪ output_size), 0.0F);
732         lcqi::linear_i8(layer, input, scalar_output);
733         lcqi::linear_i8_packed(packed, input, packed_output);
734         require(lcqi::max_abs_diff(scalar_output, packed_output) <= <
    ↪ LCQI_KERNEL_MAX_DIFF,
735             "packed int8 kernel output differs from scalar");
736
737         const lcqi::PackedLinearI8 simd_packed =
738             lcqi::pack_linear_i8(layer, LCQI_SIMD_TEST_BLOCK);
739         if (lcqi::linear_i8_packed_avx2_available()) {
740             std::vector<float> avx2_output(static_cast<std::size_t>(layer.<
    ↪ output_size), 0.0F);
741             lcqi::linear_i8_packed_avx2(simd_packed, input, avx2_output);
742             require(lcqi::max_abs_diff(scalar_output, avx2_output) <= <
    ↪ LCQI_KERNEL_MAX_DIFF,
743                 "AVX2 int8 kernel output differs from scalar");
744         }
745     }
746 }
747

```



```
748 } // namespace
749
750 int main() {
751     try {
752         test_tiny_mlp();
753         test_tokenizer_contract();
754         test_tokenizer_rejects_unknown_without_unk();
755         test_safetensors_manifest_contract();
756         test_safetensors_rejects_bad_offsets();
757         test_safetensors_rejects_bad_dtype_shape_size();
758         test_safetensors_reads_float_tensors();
759         test_gpt2_tiny_forward_and_generation();
760         test_gpt2_byte_bpe_tokenizer();
761         test_gpt2_loads_hf_style_tiny_checkpoint();
762         test_gpt2_loader_rejects_bad_shape();
763         test_reference_rms_norm();
764         test_reference_rope();
765         test_reference_kv_attention();
766         test_reference_decoder_end_to_end();
767         test_reference_decoder_trace_contract();
768         test_packed_i8_kernel_matches_scalar();
769
770         std::cout << "lcqi tests passed\n";
771         return EXIT_SUCCESS;
772     } catch (const std::exception& error) {
773         std::cerr << "lcqi test failure: " << error.what() << "\n";
774         return EXIT_FAILURE;
775     }
776 }
```

读码检查清单

构建入口 是否能从 CMake 文件看出每个可执行目标依赖哪些源码，哪些目标属于默认 CTest，哪些目标依赖外部环境。

资产边界 模型文件、tokenizer、safetensors header 和外部下载模型是否都有校验或拒绝路径。

公共合同 头文件是否把 shape、dtype、token、trace checkpoint、KV cache 地址和返回字段说清楚。

正确性路径 tiny MLP、reference decoder 和 GPT-2 tiny fixture 是否有可复查 golden。

性能口径 benchmark 是否同时报告 backend、available、耗时、误差和 checksum，避免只有速度没有正确性。

真实模型证据 GPT-2 smoke 和 SmolLM2 smoke 是否记录模型文件 hash、命令、输出和验证结论。

回归门禁 CTest 是否覆盖坏资产、坏 shape、tokenizer 合同、trace 关键字段、ledger 固定公式和 serving probe 拒绝路径。

如果某段新代码不能回答上面的检查项，它就还不能进入第三册主线工程。源码卷的作用是把这条标准显式化：代码不只是能编译，还要能解释、能测量、能拒绝坏输入、能被长期回归。